The System Design Interview Playbook

The System Design Interview Playbook

A Step-by-Step Framework, 20 Worked Designs, Tested Building Blocks and Mock Interviews to Prepare for Senior Engineering Interviews at Top Tech Companies

Aadhavan Muthurengan

The System Design Interview Playbook
A Step-by-Step Framework, 20 Worked Designs, Tested Building Blocks and Mock Interviews to Prepare for Senior Engineering Interviews at Top Tech Companies

Copyright © 2026 Aadhavan Muthurengan. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the author, except for brief quotations in reviews and certain other noncommercial uses permitted by copyright law. Readers may freely use and adapt the example code and designs in this book for their own study, practice and work.

Disclaimer. This book is provided for educational and informational purposes only. AI products, features, and policies change frequently; verify current details with each provider. The author makes no guarantees about results and is not liable for any losses arising from the use of this information. Nothing in this book is legal, medical, financial, or other professional advice.

Trademarks. Google, Meta, Amazon, Twitter and other company names are trademarks of their respective owners; they are used only to describe publicly stated or publicly reported hiring practices. Product names such as Redis, Memcached, Kafka, Cassandra, PostgreSQL, MySQL, ZooKeeper, etcd and Python are trademarks of their respective owners and are mentioned only to describe widely used technologies. This book is independent and is not affiliated with, sponsored by, or endorsed by any company it mentions. The designs are generic teaching designs and do not describe any company's actual systems unless a published source is cited.

Examples. Every building block and design model in this book is runnable Python, tested on Python 3.12, 3.13, 3.14 and 3.15. Interview formats described were checked against companies' own published guidance where available, as of October 2026, and change over time.

First edition, 2026.

To my father and my mother,
for your love, your sacrifices, and every lesson you taught me.

To my wife,
for your patience, your encouragement, and your belief in me.

And to our three children,
may you always build the things you dream of.

This book is for you.

Contents

Introduction 11

Part I: The Interview and the Method 15

1. What the System Design Interview Tests . . . 17
2. A Framework for 45 Minutes . . . 23
3. Back-of-the-Envelope Estimation . . . 29
4. Networking and APIs for Designers . . . 35

Part II: Building Blocks 41

5. Scaling Out: Load Balancers and Stateless Services . . . 43
6. Caching . . . 49
7. Databases and Storage Engines . . . 55
8. Replication and Consistency . . . 61
9. Partitioning and Consistent Hashing . . . 67
10. Queues, Logs and Delivery Guarantees . . . 71
11. Rate Limiting . . . 75
12. Unique IDs and Time . . . 81
13. Probabilistic Data Structures . . . 87
14. Reliability: Timeouts, Retries and Circuit Breakers . . . 93
15. Observability and Security Essentials . . . 99

Part III: Twenty Worked Designs 103

16. Link Shortener 105
17. Distributed Rate Limiter 111
18. Key-Value Store 117
19. Unique ID Generator 123
20. Web Crawler 129
21. Notification Service 135
22. News Feed 141
23. Chat and Messaging 147
24. Search Autocomplete 153
25. Video Streaming 159
26. File Storage and Sync 165
27. Nearby Places 171
28. Ride Matching 177
29. Payment System 183
30. Ticket Booking 189
31. Metrics and Alerting 195
32. Distributed Job Scheduler 201
33. Real-Time Leaderboard 207
34. Collaborative Document Editing 213
35. Distributed Message Queue 219

Part IV: Practice **225**

36. Four Full Mock Interviews 227
37. Common Mistakes and How to Recover 237
38. Your Study Plan 241

Appendix A: Numbers Every Designer Should Know **245**

Appendix B: Building Blocks Reference 249

Appendix C: One-Page Design Template 253

Appendix D: Design Index 257

Appendix E: Glossary 259

About the Author 263

Introduction

A system design interview starts with a sentence like "Design a chat app" or "Design a service that shortens links." There is no single right answer, no compiler, and no test that passes or fails. For 45 minutes you and an experienced engineer sketch a system that could serve millions of people, and talk through the choices: where the data lives, what happens when a machine dies, how the system grows when traffic grows tenfold.

For many engineers, this is the most intimidating interview of all. Coding questions can be practiced by solving hundreds of problems; system design feels open-ended, and it's often the round that decides the level of an offer, and with it the salary. This book is designed to make it learnable.

What Makes This Book Different

A method, not a list of answers. Interviewers rarely ask exactly the design you memorized, and they change the requirements halfway through. Chapter 2 gives a step-by-step framework for any design question, and every worked design in Part III follows it, so by the end of the book the framework is a habit.

Every building block is real, tested code. Most system design books describe ideas such as consistent hashing, quorums, rate limiters and Bloom filters in words and diagrams. Here, each one is also a short Python model you can run, and each model is tested: on Python 3.12, 3.13, 3.14 and 3.15, usually against a simpler version on thousands of random inputs. The outputs printed in this book were produced by running that code. When the book says

that adding a server to a consistent-hashing ring moves about 9% of keys while a naive scheme moves 91%, that's a measurement, not a claim.

Every estimate is computed. Each worked design starts with back-of-the-envelope numbers. They are computed by the same small helper functions you'll learn in Chapter 3, so they are consistent and checkable, and you can change an assumption and rerun them.

Twenty complete designs. Part III works through twenty common interview prompts, from a link shortener to a payment system and a collaborative editor, each with requirements, estimates, an API, a data model, a high-level diagram, deep dives, trade-offs and the follow-up questions interviewers like to ask.

Practice like the real thing. Part IV contains four full mock interviews, written as conversations between candidate and interviewer, with the interviewer's notes at the end of each.

How the Book Is Organized

- **Part I, The Interview and the Method:** what the interview tests, a framework for 45 minutes, estimation, and the networking and API basics every design relies on.
- **Part II, Building Blocks:** load balancing, caching, databases and storage engines, replication, partitioning, queues and logs, rate limiting, IDs and time, probabilistic data structures, reliability patterns, and observability and security.
- **Part III, Twenty Worked Designs.**
- **Part IV, Practice:** four mock interviews, the common mistakes and how to recover from them, and study plans.
- **Appendices:** the numbers every designer should know, a building-block reference, a one-page design template, an index of the designs, and a glossary.

How to Use the Code

You don't need to run the code to follow the book; every listing is short, explained in the text, and shown with its output. But running and changing it is the fastest way to build intuition. Change the number of virtual nodes in the hashing ring, or the quorum sizes, and watch what happens. All models use only Python's standard library.

> **Note:** The models are teaching tools that capture the essential behavior of each technique in a few dozen lines. Production systems add networking, persistence, monitoring and many edge cases. The book points out the most important differences as they come up.

A Note on Companies, Products and Dates

The book refers to companies such as Google, Meta and Amazon when describing interviews, and to widely used technologies such as Redis, Kafka and PostgreSQL when describing building blocks. Where a company publishes its own guidance, the book relies on it and says so; where information comes from candidates' reports, the book says that too. The designs in Part III are generic teaching designs, not descriptions of any company's internal systems. Interview formats change, so always confirm the details with your recruiter. The descriptions in this book reflect what was published as of **October 2026**.

If you have read *The Coding Interview Playbook*, you will recognize the structure: the same emphasis on method, on testing, and on explaining your thinking out loud. If you haven't, nothing in this book depends on it.

Let's begin with what the interviewer is actually looking for.

PART I

The Interview and the Method

CHAPTER 1

What the System Design Interview Tests

In a coding interview, the interviewer can check your answer against a known solution. In a system design interview, they can't, because there isn't one. A good design for a startup's first million users is a bad design for a company with a billion, and both can be defended. So the interviewer isn't grading the design itself. They are grading how you got there: the questions you asked, the numbers you estimated, the trade-offs you saw, and how you responded when they pushed back.

This chapter explains what that means in practice, how the interview is scored at different levels, and what large companies say about it.

The Format

Most system design interviews follow the same shape:

- **About 45 to 60 minutes**, in person on a whiteboard or remotely with a shared drawing tool.
- **One open-ended prompt**, usually a product you know: "Design a news feed", "Design a ride-sharing service", "Design a rate limiter."
- **A conversation, not a presentation.** The interviewer answers questions, adds constraints ("now make it work across three regions"), and chooses where to go deep.

- **Little or no code.** You may be asked to write an API signature, a schema or a short piece of pseudocode, but the focus is architecture.

Some companies run a variant focused on product and APIs rather than infrastructure. Candidates describe Meta's design rounds, for example, as either a "system design" round, centered on backend infrastructure, or a "product architecture" round, centered on APIs, data models and client-server interaction for a user-facing product. Both use the same skills; the emphasis differs. Ask your recruiter which one you'll get.

Who Gets a System Design Interview

The round is usually part of the loop for mid-level engineers and above, and it carries more weight the more senior the role. Amazon is unusually explicit: its interview prep page for senior software engineers (SDE III) says that half of the interview time is spent on Leadership Principles questions and the other half on coding problems and system design. Candidates report similar patterns elsewhere: a design round or two for mid-level and senior roles, and often none for new graduates, who may get an extra coding round instead.

Because the design round often decides the level of an offer, it's worth preparing for even if you are early in your career. A mid-level candidate who designs like a senior one can be hired at the higher level.

What Interviewers Evaluate

Rubrics differ between companies, but the same five areas appear everywhere:

Area	What strong looks like	What weak looks like
Problem exploration	Asks about users, scale and the most important features; agrees on scope before drawing	Starts drawing boxes immediately; designs features nobody asked for
Estimation	Turns requirements into numbers (requests per second, storage) and uses them to choose a design	Skips numbers, or computes them and never uses them

Area	What strong looks like	What weak looks like
Architecture	A clear, simple high-level design; explains how one request flows through it	A diagram full of fashionable components with no explanation of why each is there
Depth and trade-offs	Goes deep on the hardest part; compares at least two options and explains the choice	Names technologies instead of explaining mechanisms; "we'll use Kafka" with no reason
Communication	Drives the conversation, checks in with the interviewer, adapts to new constraints	Monologues; ignores hints; argues instead of reasoning

> **Interviewer's View:** The most common reason strong engineers fail this round isn't a lack of knowledge. It's a lack of structure: they spend 20 minutes on one detail, run out of time, and never show a complete design. The framework in Chapter 2 exists to prevent exactly that.

How Expectations Change With Level

The same prompt is graded differently depending on the level you're interviewing for:

- **Mid-level:** a working, sensible design with a clear data model and API, correct use of common building blocks (load balancers, caches, a database, a queue), and reasonable answers when asked about scale. The interviewer may lead the deep dives.
- **Senior:** you drive. You identify the hardest parts yourself, quantify them, compare alternatives, and discuss failure modes: what happens when a node, a region or a dependency fails. You know where the bottlenecks will appear as traffic grows.
- **Staff and above:** everything above, plus judgment about the broader picture: operational cost, team boundaries, migration from an existing system, how to roll out safely, and what not to build. Breadth across the whole system and depth in two or three areas.

What Companies Say

Companies publish less about system design than about coding interviews, so treat any detailed description of "the format at company X" with care.

Company	What is published or reported	Type of source
Amazon	For senior software engineers, half of the interview time goes to Leadership Principles and half to coding and system design	Amazon's SDE III interview prep page
Meta	Design rounds described as "system design" or "product architecture", about 45 minutes, depending on role and level; select roles also include an AI-assisted coding round	Candidate reports and preparation guides; Meta's hiring page (AI-assisted round)
Google	Design rounds of about 45 minutes, mainly for mid-level and senior candidates; feedback reviewed by a hiring committee	Candidate reports; Google's general candidate guidance to think out loud, clarify and discuss trade-offs

Across all of them, the advice converges: clarify before you design, think out loud, use numbers, and discuss trade-offs honestly.

Key Takeaways

- System design interviews grade your process: exploration, estimation, architecture, trade-offs and communication.
- There's no single right answer; a defensible design with clear reasoning beats a fashionable one.
- Expectations rise with level: seniors drive the conversation and discuss failure modes; staff engineers add cost, operations and rollout.
- Ask your recruiter whether your round is infrastructure-focused or product-focused.

Sources

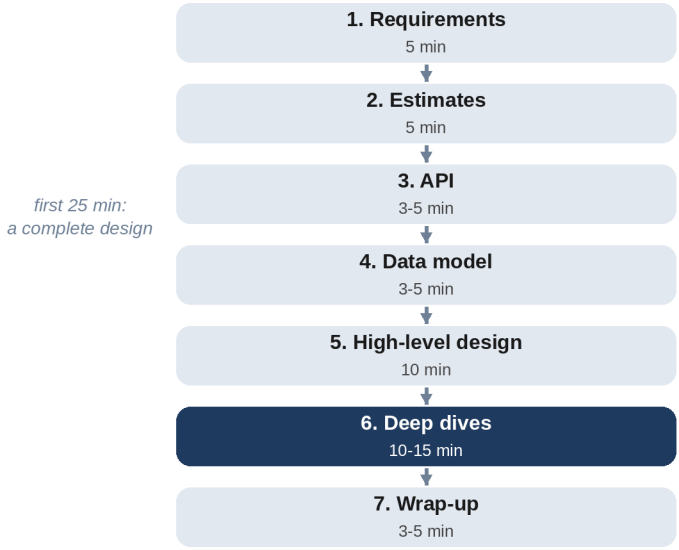
- Amazon, "SDE III Interview Prep", amazon.jobs (half Leadership Principles, half coding and system design).

- Meta Careers, hiring process page (authorized AI assistant for select roles); descriptions of design rounds are from candidates' reports and preparation guides published in 2025 and 2026.
- Google, "Preparing for Google technical internship interviews" (candidate guide: think out loud, clarify, state assumptions, discuss trade-offs); descriptions of design rounds are from candidates' reports.

CHAPTER 2

A Framework for 45 Minutes

The biggest risk in a system design interview isn't a wrong answer; it's running out of time with half a design. This chapter gives you a seven-step framework with a time budget. It works for any prompt, and every design in Part III follows it, so you'll practice it twenty times before your interview.



The seven steps of a system design interview with a rough time budget: requirements, estimates, API, data model, high-level design, deep dives and wrap-up.

Step	Time	Output on the whiteboard
1. Requirements	5 min	A short list of functional and non-functional requirements, and what's out of scope
2. Estimates	5 min	Requests per second, storage, bandwidth; read-heavy or write-heavy
3. API	3 to 5 min	The main endpoints or functions, with parameters
4. Data model	3 to 5 min	The main entities and how they're stored
5. High-level design	10 min	Boxes and arrows; one request traced end to end
6. Deep dives	10 to 15 min	Two or three hard parts, with options and a choice
7. Wrap-up	3 to 5 min	Bottlenecks, failure modes, what you'd do next

The times are a guide, not a rule. If the interviewer wants to skip the API and go straight to scaling, follow them. But keep an eye on the clock: when 25 minutes have passed, you should have a complete high-level design on the board.

Step 1: Requirements

Prompts are deliberately vague. "Design Instagram" could mean photo upload, feeds, stories, direct messages, search, ads and more; nobody can design all of that in 45 minutes. Your first job is to agree on a scope.

Functional requirements are what the system does. Aim for three to five, phrased as user actions: "users post a photo with a caption", "users see a feed of posts from accounts they follow", "users like a post." Then say explicitly what you'll leave out: "I'll skip stories, messaging and ads unless you'd like me to include them."

Non-functional requirements are the qualities the system must have. The useful ones to ask about:

- **Scale:** how many users, how many daily active users, how much data?

- **Latency:** what must feel instant (loading a feed), and what can be slower (uploading a video)?
- **Availability versus consistency:** is it worse to show slightly stale data or to show an error? A like count can be stale; a bank balance can't.
- **Durability:** can we ever lose data? Usually no for user content; often yes for caches and analytics samples.
- **Geography:** one region or global?

> **Tip:** If the interviewer says "you decide", make a reasonable assumption out loud and write it down: "I'll assume 100 million daily active users and a global audience." A clear assumption is better than a question left hanging.

Step 2: Estimates

Turn the requirements into numbers: requests per second for reads and writes, storage per year, bandwidth. Chapter 3 shows how to do this quickly and how far to round. The point isn't precision; it's to discover the shape of the problem:

- Is it **read-heavy** (a link shortener, a news site) or **write-heavy** (metrics ingestion, chat)?
- Does the data fit on **one machine**, or must it be **partitioned**?
- Is the bottleneck **requests**, **storage**, or **bandwidth**?

Then use the answers. "We need about 4,000 reads per second and 40 writes per second, so caching will matter much more than write throughput" is a sentence that earns credit. Numbers that are computed and then ignored earn none.

Step 3: API

Define the main operations precisely enough that someone could implement a client. For a web service, that's usually a few HTTP endpoints:

```
POST /v1/photos            body: image, caption    returns:
  photo_id
GET  /v1/feed?cursor=...   returns: posts, next_cursor
POST /v1/photos/{id}/likes returns: 204
```

Think about the details that matter at scale: pagination with cursors rather than page numbers (Chapter 4 shows why), idempotency keys for operations that must not happen twice, and which calls must be fast.

Step 4: Data Model

List the main entities, their key fields, and how they'll be looked up. The access patterns decide the storage:

- **Users, posts, follows** with lookups by ID and by user: tables keyed by ID, with an index on user.
- **A huge volume of small writes read by key** (messages, metrics): a key-value or wide-column store partitioned by key.
- **Large binary objects** (photos, videos): object storage, with only the metadata in a database.

Say which store you'd use and why. Chapter 7 covers the choices.

Step 5: High-Level Design

Draw the main components and the arrows between them: clients, load balancers, services, databases, caches, queues, object storage. Keep it simple: five to ten boxes. Then **trace one request** through the diagram, for example a feed load from the user's phone to the response, and one write, such as posting a photo. Tracing exposes missing pieces and shows the interviewer you understand how the parts connect.

> **Common Mistake:** Drawing a "microservices" diagram with fifteen services before anything works. Start with the minimum that satisfies the requirements; split services only when you can say why.

Step 6: Deep Dives

Now spend the most time on the two or three hardest parts of this particular system. For a news feed, that's how posts reach followers' feeds; for a chat app, it's message delivery and ordering; for a payment system, it's never charging twice. Often the interviewer chooses; if not, propose: "The interesting parts are feed generation and the hot-key problem for celebrity accounts. Shall I start with feed generation?"

For each deep dive, follow a mini-pattern:

1. State the problem precisely, with numbers.
2. Give two or three options.
3. Compare them on the requirements that matter here.
4. Choose one, and say what you'd monitor to know if the choice stops working.

This is where the building blocks of Part II earn their keep.

Step 7: Wrap-Up

In the last few minutes, step back:

- **Bottlenecks:** what breaks first at ten times the traffic?
- **Failure modes:** what happens if the cache, a database replica, a whole region goes down?
- **Operations:** what would you monitor and alert on?
- **Extensions:** what would you build next, given more time?

A short, honest wrap-up ("the weakest part of this design is the single database for writes; with more time I'd partition it by user ID") leaves a strong final impression.

Handling Curveballs

Interviewers deliberately change the problem: "Now 10 times the traffic." "Now it must work across continents." "What if this service is down?" These are not attacks; they test whether your design and your reasoning adapt. Go back to the numbers, find which component is now under pressure, and adjust it, rather than redrawing everything.

If you don't know something, say so and reason from first principles: "I haven't used that database, but if it's a log-structured store, writes should be cheap and reads may need several lookups, so..." Interviewers value honest reasoning far more than a confidently wrong fact.

Key Takeaways

- Follow seven steps: requirements, estimates, API, data model, high-level design, deep dives, wrap-up.
- Agree on scope early and write down your assumptions.
- Use estimates to choose the design, not as a ritual.
- Have a complete high-level design on the board by about 25 minutes.
- Spend the remaining time going deep on the hardest parts, comparing options each time.

CHAPTER 3

Back-of-the-Envelope Estimation

Estimation sounds intimidating, but in a design interview it takes five minutes and uses only multiplication and division. The goal is not an exact answer; it's the right **order of magnitude**: is this 10 requests per second or 10,000? Gigabytes or petabytes? Those differences decide the architecture.

Numbers to Remember

You need very few facts:

Quantity	Value	Shortcut
Seconds in a day	86,400	about 100,000
1 million per day	about 12 per second	divide by 100,000
1 billion per day	about 12,000 per second	
1 KB, 1 MB, 1 GB, 1 TB, 1 PB	10^3, 10^6, 10^9, 10^{12}, 10^{15} bytes	each step is ×1,000
Peak traffic	2 to 10 times the average	state your assumption

And a sense of how big things are:

Item	Typical size
A short text field (a name, a URL)	tens to a few hundred bytes
A database row with a few fields	a few hundred bytes to 1 KB
A compressed photo	100 KB to a few MB
A minute of HD video	tens of MB

Appendix A lists more, including latency figures such as the time for a memory read, a network round trip and a disk seek.

The Helper Functions

This book computes every estimate with a handful of tiny functions, so the numbers are consistent and you can rerun them with your own assumptions:

```
def per_second(per_day):
    """Average rate per second for a daily total."""
    return per_day / SECONDS_PER_DAY


def peak(rate, factor=3):
    """Peak rate, assuming peak traffic is `factor` times the average."""
    return rate * factor


def storage(item_bytes, items_per_day, days, copies=1):
    """Bytes needed to keep `days` of items, with `copies` replicas."""
    return item_bytes * items_per_day * days * copies


def bandwidth(rate, item_bytes):
    """Bytes per second for `rate` items per second."""
    return rate * item_bytes


def servers(rate, per_server, headroom=0.7):
    """Machines needed so each runs at most `headroom` of its capacity."""
    needed = rate / (per_server * headroom)
    return max(1, -(-needed // 1))  # round up


def in_flight(rate, latency_s):
    """Little's law: requests in progress = arrival rate x time in
    ↪ system."""
    return rate * latency_s
```

Two of them deserve a word:

- `servers` divides the load by what one machine can handle, but plans for each machine to run at only 70% of its capacity. That headroom absorbs bursts and lets you lose a machine without overloading the rest.
- `in_flight` is **Little's law**: the average number of requests in a system equals the arrival rate times the average time each spends there. With 1,000 requests per second and 200 ms per request, about 200 requests are in progress at any moment. That tells you how many connections, threads or worker processes you need.

The functions are checked against simple cases:

```
>>> daily_to_qps(86400)
1.0
>>> daily_to_qps(1000000)
11.6
>>> daily_to_qps(1000000000)
11574.1
```

A Worked Estimate

Suppose a photo-sharing app has 10 million uploads a day, each about 2 MB, with 100 views per upload. Photos are kept for 5 years with 3 copies. Each web server handles about 1,000 requests per second, and requests take about 200 ms.

```
def demo():
    # A photo-sharing app: 10 million uploads a day, 2 MB each,
    # kept for 5 years with 3 copies; 100 views per upload.
    uploads = per_second(10_000_000)
    views = uploads * 100
    table(
        [
            ("uploads per second (avg)", human(uploads)),
            ("uploads per second (peak x3)", human(peak(uploads))),
            ("views per second (avg)", human(views)),
            (
                "upload bandwidth",
                human(bandwidth(uploads, 2 * 10**6), "B") + "/s",
            ),
            (
                "view bandwidth",
                human(bandwidth(views, 2 * 10**6), "B") + "/s",
            ),
            (
                "storage, 5 years x 3",
```

```
                human(storage(2 * 10**6, 10**7, 5 * 365, 3), "B"),
            ),
            (
                "web servers (1,000 rps each)",
                int(servers(peak(views), 1000)),
            ),
            (
                "requests in flight (200 ms)",
                human(in_flight(peak(views), 0.2)),
            ),
        ]
    )
```

```
uploads per second (avg)      116
uploads per second (peak x3)  347
views per second (avg)        11.6K
upload bandwidth              231 MB/s
view bandwidth                23.1 GB/s
storage, 5 years x 3          110 PB
web servers (1,000 rps each)  50
requests in flight (200 ms)   6.94K
```

Read the results like an interviewer would:

- **Uploads are modest** (about 116 per second); **views are 100 times more frequent.** This is a read-heavy system, so caching and a content delivery network (CDN) will matter most.
- **View bandwidth is enormous**, about 23 GB per second. No single data center link serves that comfortably; a CDN is not optional.
- **Storage reaches about 110 PB** over five years with three copies. That rules out a database for the images themselves: they belong in object storage, with only metadata in a database. It also suggests cheaper storage tiers or erasure coding instead of three full copies for old photos (Chapter 26).
- **Fifty web servers** for the peak, and about 7,000 requests in flight at any moment.

Each line points to a design decision. That's what makes an estimate worth its five minutes.

Rounding Like an Engineer

Round aggressively and say so:

- 86,400 seconds becomes 100,000 when you're in a hurry; 10 million a day is "about 100 per second".
- Multiply mantissas and add exponents: 2×10^6 bytes $\times$ 10^7 photos = 2×10^{13} bytes = 20 TB per day.
- Keep one or two significant figures. "About 20 TB a day" is better than "19.3 TB".

> **Common Mistake:** Mixing bits and bytes. Network speeds are usually quoted in bits per second (Mbps, Gbps); storage in bytes. Divide bits by 8 to get bytes: a 1 Gbps link moves about 125 MB per second.

What to Estimate, and When to Stop

Estimate what can change the design:

1. **Read and write rates**, average and peak.
2. **Storage** over the data's lifetime, including replication.
3. **Bandwidth** for large objects.
4. **Memory** for anything you plan to cache: will the hot set fit in RAM on one machine, or must the cache be distributed?
5. **Machines** or **connections**, when relevant (chat servers holding millions of open connections, for example).

Stop as soon as the numbers have told you what you need. Five lines on the whiteboard are usually enough.

> **Interviewer's View:** Saying "this fits in memory on one machine, so a distributed cache would be overkill" is as valuable as discovering that you need petabytes. Estimates that make the design *simpler* show good judgment.

Key Takeaways

- Estimates exist to choose the architecture: read-heavy or write-heavy, one machine or many, where the bottleneck is.
- Remember a handful of numbers: 86,400 seconds a day, 1 million a day is about 12 per second, and the powers of 1,000.
- Little's law turns rates and latencies into concurrency: requests in progress = rate × time.
- Round aggressively, keep one or two significant figures, and state your assumptions.

CHAPTER 4

Networking and APIs for Designers

Every design in this book starts with a client sending a request over a network. You don't need to be a networking expert for a system design interview, but you do need to know what happens between a tap on a phone and a response, which protocol fits which job, and how to design an API that still works when there are millions of clients and things go wrong.

The Life of a Request

When a user opens `https://app.example/feed`:

1. **DNS** translates `app.example` into an IP address. Results are cached by the operating system, the browser and resolvers, which is why DNS changes take time to spread, and why DNS can also route users to the nearest region.
2. **TCP** opens a connection with a three-way handshake: one network round trip.
3. **TLS** negotiates encryption: one more round trip with TLS 1.3.
4. **HTTP** sends the request; the response comes back.
5. Usually the request first reaches a **CDN** edge server or a **load balancer**, not the application directly.

Each round trip costs time proportional to distance: within a data center, well under a millisecond; across a continent, tens of milliseconds; around the world, 150 milliseconds or more. That's why reusing connections, serving content from nearby edge servers, and reducing the number of sequential calls matter so much for latency.

HTTP Versions in One Paragraph Each

- **HTTP/1.1** sends one request at a time per connection; browsers open several connections to work around it.
- **HTTP/2** multiplexes many requests over one TCP connection, with compressed headers. A lost packet still stalls every request on that connection, because TCP delivers bytes in order.
- **HTTP/3** runs over **QUIC**, a transport built on UDP (standardized in RFC 9000 in 2021, with HTTP/3 in RFC 9114 in 2022). Each request stream is independent, so one lost packet stalls only its own stream, and connections can survive a change of network, such as walking from Wi-Fi to mobile data.

For interview purposes: HTTP/2 or HTTP/3 between clients and the edge; HTTP/2-based protocols such as gRPC between internal services.

Choosing an API Style

Style	Strengths	Weaknesses	Typical use
REST over HTTP + JSON	Simple, cacheable, universal tooling	Chatty for complex screens; loose contracts	Public APIs, web and mobile clients
gRPC (HTTP/2 + Protocol Buffers)	Compact, fast, typed contracts, streaming	Less friendly for browsers; harder to inspect by hand	Service-to-service calls
GraphQL	Client asks for exactly the fields it needs in one round trip	Caching and rate limiting are harder; expensive queries possible	Complex client screens with many data sources

In an interview, REST is the default for client APIs unless there's a reason to choose otherwise; gRPC is a good default between internal services.

Real-Time Updates: Polling, SSE and WebSockets

Many designs push data to clients: chat messages, notifications, live scores, a driver's location.

- **Short polling:** the client asks "anything new?" every few seconds. Simple, but wasteful and slow to notice changes.
- **Long polling:** the server holds the request open until there's news or a timeout, then the client immediately asks again. Works everywhere, but each message costs a new request.
- **Server-Sent Events (SSE):** one long-lived HTTP response through which the server streams events. One-way (server to client) and simple.
- **WebSockets (RFC 6455):** a persistent, two-way connection upgraded from HTTP. The usual choice for chat and collaborative editing.

Persistent connections change the architecture: a server holding 100,000 open connections must be tracked ("which gateway is user 42 connected to?"), and deploying new code means draining connections gracefully. Chapter 23 covers this in detail.

Designing APIs That Scale

Pagination: Use Cursors

Lists are returned in pages. The obvious design, `?page=3` or `?offset=60`, breaks when the list changes between requests: if new items arrive at the top of a feed, every item shifts down, and the next page repeats items the user has already seen (or, for deletions, skips some). A **cursor** fixes this by asking for "the items after the last one I saw":

```
def offset_pages(feed_versions, page_size=3):
    """feed_versions[i] is the feed (newest first) when page i is read."""
    seen = []
    for i, feed in enumerate(feed_versions):
        seen.extend(feed[i * page_size : (i + 1) * page_size])
    return seen
```

```
def cursor_pages(feed_versions, page_size=3):
    seen, cursor = [], None  # cursor: the ID of the last item seen
    for feed in feed_versions:
        older = [x for x in feed if cursor is None or x < cursor]
        page = older[:page_size]
        seen.extend(page)
        if page:
            cursor = page[-1]
    return seen
```

Here, two new items arrive between each page load:

```
offset pages: [20, 19, 18, 19, 18, 17, 18, 17, 16]
cursor pages: [20, 19, 18, 17, 16, 15, 14, 13, 12]
```

The offset version shows 19, 18 and 17 more than once; the cursor version shows each item exactly once. Cursors are also faster for databases: "WHERE id < cursor ORDER BY id DESC LIMIT 20" uses an index, while a large OFFSET forces the database to count past all the skipped rows. In a real API, encode the cursor as an opaque string so clients don't depend on its contents.

Idempotency

Networks fail in the worst possible place: after the server has done the work but before the client receives the answer. The client can't tell, so it retries. For reads that's harmless; for "charge the card" it's a disaster. The fix is an **idempotency key**: the client sends a unique ID with each logical operation, and the server stores the result under that key and returns the same result on a retry instead of repeating the work. Chapter 29 implements this for payments.

Other Details Worth Mentioning

- **Versioning** (`/v1/...`) so old clients keep working when the API changes.
- **Rate limits**, with responses that tell clients when to retry (HTTP 429 with a `Retry-After` header). Chapter 11.
- **Timeouts** on every call, and clear error codes that distinguish "retry later" (503) from "your request is wrong" (400).
- **Authentication** with short-lived tokens; never send secrets in URLs, which end up in logs.

Content Delivery Networks

A CDN is a network of edge servers close to users that caches static content: images, video segments, scripts, style sheets. Users get the content from a server a few milliseconds away instead of crossing an ocean, and your origin servers see a fraction of the traffic. The trade-off is **staleness**: when content changes, cached copies remain until they expire or you purge them. Use long cache lifetimes for content that never changes (give each version a new URL, such as `logo.3f9a.png`) and short ones for content that does.

Key Takeaways

- Latency is dominated by round trips and distance; reuse connections and serve content from the edge.
- REST for public client APIs, gRPC between services, GraphQL when clients need flexible queries.
- WebSockets for two-way real-time features; SSE for one-way streams; long polling as a fallback.
- Paginate with cursors, not offsets; offsets repeat or skip items when the data changes.
- Make every non-read operation idempotent with a client-supplied key.

PART II

Building Blocks

CHAPTER 5

Scaling Out: Load Balancers and Stateless Services

The first scaling decision in almost every design is how to serve more requests than one machine can handle. You can buy a bigger machine (**vertical scaling**) or add more machines (**horizontal scaling**). Vertical scaling is simple and surprisingly powerful, a single modern server has dozens of cores and hundreds of gigabytes of memory, but it has a ceiling, and that one machine is a single point of failure. Horizontal scaling has no practical ceiling, but it needs two things: a way to spread requests across machines, and services that don't care which machine handles a request.

Stateless Services

A service is **stateless** when any instance can handle any request because it keeps nothing important in its own memory between requests. Session data, uploads in progress and user preferences live in shared stores: a database, a cache, object storage. Stateless services are easy to scale (add instances), easy to deploy (replace instances one by one), and easy to recover (a crashed instance loses nothing).

Some components are inherently **stateful**: databases, caches, and servers holding long-lived connections such as WebSockets. A good design keeps state in as few, well-understood places as possible, and makes everything else stateless.

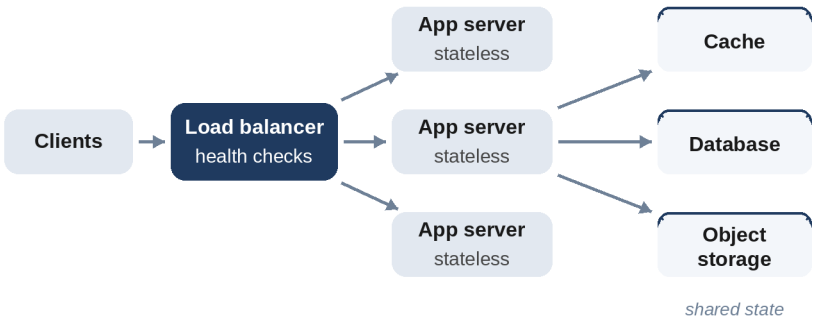
Load Balancers

A load balancer receives client requests and forwards each to one of several backend instances. It also performs **health checks**, regularly probing each backend and removing unhealthy ones from rotation, which is what turns "we have several servers" into "we survive losing one".

Load balancers work at two levels:

- **Layer 4 (transport):** forwards TCP or UDP connections based on addresses and ports, without looking inside. Very fast, and protocol-agnostic.
- **Layer 7 (application):** understands HTTP, so it can route by path or header (`/api/*` to one service, `/images/*` to another), terminate TLS, and retry failed requests.

A typical web design has DNS pointing to a few layer-7 load balancers (themselves redundant), which spread traffic across stateless application servers.



A load balancer with health checks in front of stateless application servers; state lives in shared stores behind them.

Choosing a Balancing Strategy

How the balancer picks a backend matters more than it seems, because requests are not all equal: most are quick, a few are slow. The model below sends the same stream of requests, most taking 1 to 4 time units and one in five taking 120, to ten servers with four strategies, and records the most requests any one server held at once:

```
def simulate(strategy, servers=10, requests=20_000, seed=1):
    rng = random.Random(seed)
    finish = [[] for _ in range(servers)]  # end ticks of active requests
    turn = 0
    worst = 0
    for tick in range(requests):
        for active in finish:  # drop finished requests
            active[:] = [t for t in active if t > tick]
        load = [len(active) for active in finish]
        if strategy == "round robin":
            s = turn % servers
            turn += 1
        elif strategy == "random":
            s = rng.randrange(servers)
        elif strategy == "two choices":  # pick the lighter of two
            a, b = rng.randrange(servers), rng.randrange(servers)
            s = a if load[a] <= load[b] else b
        else:  # least connections: needs every server's load
            s = min(range(servers), key=load.__getitem__)
        finish[s].append(tick + rng.choice([1, 2, 3, 4, 120]))
        worst = max(worst, len(finish[s]))
    return worst
```

```
random       busiest server held 12 requests
round robin  busiest server held 9 requests
two choices  busiest server held 6 requests
least        busiest server held 5 requests
```

- **Random** is the worst: by chance, some servers collect several slow requests.
- **Round robin** spreads requests evenly by count, but not by work: a server can still receive a quick request right behind three slow ones.
- **Least connections** sends each request to the server with the fewest active requests. It's the best here, but the balancer must know every server's load, which is easy for one balancer and harder for many balancers sharing the work.
- **Power of two choices** picks two servers at random and sends the request to the less busy one. With almost no coordination, it gets most of the

benefit of least connections. It's a surprisingly powerful idea, and the default in several modern load balancers and service meshes.

> **Interviewer's View:** You rarely need to choose a balancing algorithm in an interview. But if asked, "least connections, or power of two choices when there are many balancers", together with the reason, uneven request costs, shows real understanding.

Sticky Sessions and Why to Avoid Them

Sticky sessions send all requests from one user to the same backend, usually because that server keeps session data in memory. They make scaling and failover harder: if that server dies, the user's session is lost, and load can't be rebalanced freely. Prefer storing sessions in a shared cache or in signed tokens held by the client, and keep servers stateless. The exception is long-lived connections, such as WebSockets, where a client naturally stays on one server for the life of the connection.

Autoscaling and Capacity

Stateless services can grow and shrink automatically, typically by watching CPU use or request rate. Two cautions:

- **Scaling takes time.** New instances need minutes to start and warm up, so autoscaling smooths daily patterns but doesn't absorb a sudden spike. Keep headroom (the 70% rule from Chapter 3) and use queues to absorb bursts.
- **The bottleneck moves.** Doubling application servers doubles the load on the database behind them. Always ask what each layer can handle.

Key Takeaways

- Scale horizontally with stateless services; keep state in a few shared stores.
- Load balancers spread requests and remove unhealthy backends using health checks.

- Request costs vary, so balancing by load (least connections, or the power of two choices) beats round robin.
- Avoid sticky sessions except for long-lived connections.
- When you add capacity to one layer, check the next one down.

CHAPTER 6

Caching

A cache keeps a copy of data somewhere faster or closer than its source: in memory instead of on disk, in the same data center instead of across the world, in the application instead of behind a network call. Caching is the single most effective tool for read-heavy systems, and also a reliable source of bugs, because every cache creates a second copy of the truth that can fall out of date.

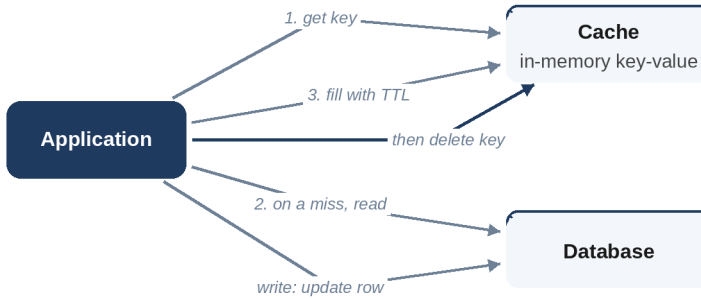
Where Caches Live

- **Client:** the browser or app keeps responses, controlled by HTTP cache headers.
- **CDN:** edge servers cache static content close to users (Chapter 4).
- **Application memory:** each server keeps a small local cache. Very fast, but every server has its own copy, so invalidation is harder.
- **Distributed cache:** a shared in-memory store such as Redis or Memcached, used by all application servers.
- **Database:** databases cache pages and query results internally.

Reading and Writing Through a Cache

Pattern	How it works	Good for	Watch out for
Cache-aside (lazy loading)	App reads the cache; on a miss, reads the database and fills the cache	Most read-heavy systems	Stale data until expiry or invalidation; first reads are slow
Read-through	The cache itself loads from the database on a miss	Simplifying application code	Needs a cache that knows the database
Write-through	Writes go to the cache and the database together	Data that's read right after being written	Every write pays both costs
Write-behind (write-back)	Writes go to the cache; the database is updated later	Absorbing write bursts	Data loss if the cache fails before flushing

Cache-aside is the default answer in interviews. On a write, update the database and then **delete** the cache entry rather than updating it; the next read reloads a fresh value, which avoids races where two writers update the cache in the wrong order.



Cache-aside: on a hit, the app reads from the cache; on a miss, it reads the database and fills the cache; on a write, it updates the database and deletes the cache entry.

Eviction and Expiry

A cache is smaller than the data it caches, so it must decide what to keep. **Least recently used (LRU)** eviction drops the entry unused for the longest time. A **time to live (TTL)** makes entries expire after a fixed time, which bounds how stale they can get. Real caches combine both:

```
class TTLCache:
    def __init__(self, capacity, ttl, clock):
        self.capacity, self.ttl, self.clock = capacity, ttl, clock
        self.data = OrderedDict()  # key -> (value, expires_at)

    def get(self, key):
        item = self.data.get(key)
        if item is None:
            return None
        value, expires = item
        if self.clock() >= expires:  # expired: treat as a miss
            del self.data[key]
            return None
        self.data.move_to_end(key)  # most recently used
        return value

    def put(self, key, value):
        self.data[key] = (value, self.clock() + self.ttl)
        self.data.move_to_end(key)
        if len(self.data) > self.capacity:
            self.data.popitem(last=False)  # evict least recently used
```

The test replays random sequences of reads, writes and clock ticks against a deliberately simple version that searches a list, and checks that both give the same answers. A short session shows both rules at work: `b` is evicted to make room for `c`, and `a` expires after 10 seconds:

```
>>> session(
...     2,
...     10,
...     [('put', 'a', 1),
...      ('put', 'b', 2),
...      ('get', 'a'),
...      ('put', 'c', 3),
...      ('get', 'b'),
...      ('tick', 10),
...      ('get', 'a')],
... )
[1, None, None]
```

How Big Should a Cache Be?

Access patterns on real systems are usually skewed: a few items are very popular and most are rarely requested. The model below draws requests from 100,000 keys with a Zipf-like distribution, where key *k* is requested with weight 1/*k*, and measures the LRU hit rate for different cache sizes:

```
def hit_rate(cache_size, keys=100_000, requests=200_000, seed=7):
    rng = random.Random(seed)
    weights = [1 / k for k in range(1, keys + 1)]
    stream = rng.choices(range(keys), weights=weights, k=requests)
    cache, hits = OrderedDict(), 0
    for key in stream:
        if key in cache:
            hits += 1
            cache.move_to_end(key)
        else:
            cache[key] = True
            if len(cache) > cache_size:
                cache.popitem(last=False)
    return hits / requests
```

```
cache holds  0.1% of keys -> hit rate 29%
cache holds  1.0% of keys -> hit rate 51%
cache holds 10.0% of keys -> hit rate 73%
```

A cache holding just 1% of the keys serves about half of all requests, and 10% serves nearly three quarters. That's why caches pay off so well, and why it's worth estimating the size of the "hot set" rather than trying to cache everything.

The Hard Parts

- **Invalidation.** When the source changes, every cached copy must be removed or allowed to expire. A short TTL is the simplest safety net, even when you also invalidate explicitly.
- **Thundering herd (cache stampede).** When a popular entry expires, thousands of requests miss at once and all hit the database. Fixes: let one request rebuild the entry while others wait or get the stale value ("request coalescing"); add random jitter to TTLs so entries don't expire together; refresh hot entries before they expire.

- **Hot keys.** One extremely popular key (a celebrity's profile) can overload the single cache server that holds it. Replicate hot keys across several servers, or add a small local cache in front of the distributed one.
- **Cold start.** An empty cache after a restart sends all traffic to the database. Warm caches gradually, or restart them one at a time.

> **Interviewer's View:** Saying "cache-aside with a TTL, delete on write, and request coalescing for hot entries" covers what most interviewers want to hear about caching. Then quantify: what's the hit rate, and how much load does it take off the database?

Key Takeaways

- Cache-aside is the default: read the cache, fall back to the database, delete the entry on writes.
- Combine LRU eviction with a TTL to bound both memory and staleness.
- Skewed access means a small cache gives a large hit rate; estimate the hot set.
- Plan for stampedes, hot keys and cold starts.

CHAPTER 7

Databases and Storage Engines

"Which database would you use?" is one of the most common questions in a design interview, and "it depends" is only a good answer if you say what it depends on. This chapter compares the main families of databases, explains the two storage engines underneath most of them, and gives a short checklist for choosing.

Relational Databases

Relational databases such as PostgreSQL and MySQL store data in tables with a fixed schema, support joins between tables, and provide **transactions**: a group of changes either all happen or none do, and concurrent transactions don't see each other's half-finished work. That's summarized by the acronym **ACID**: atomicity, consistency, isolation and durability.

They are the right default when data is structured, relationships matter (users, orders, products), and correctness matters more than extreme write throughput. A single well-tuned relational database handles a great deal; read replicas (Chapter 8) scale reads further. Their limit is scaling *writes* beyond one machine, which requires partitioning (Chapter 9), and with it the loss of easy joins and transactions across partitions.

NoSQL Families

"NoSQL" covers several quite different designs:

Family	Data model	Strengths	Examples of use
Key-value	A value looked up by a key	Very fast, simple, scales out easily	Sessions, caches, user settings
Document	JSON-like documents, queried by fields	Flexible schema, natural for nested data	Product catalogs, user profiles
Wide-column	Rows with many columns, partitioned by key, sorted within a partition	Huge write volumes, time-ordered data	Messages, metrics, activity logs
Graph	Nodes and edges	Queries that follow relationships many hops deep	Social graphs, fraud detection
Time-series	Timestamped measurements	Compression, downsampling, time-range queries	Metrics and monitoring (Chapter 31)
Search	Inverted index over text	Full-text search, relevance ranking	Product search, logs

Many large systems use several: a relational database for accounts and payments, a wide-column store for messages, a search index for queries, object storage for files. This is sometimes called **polyglot persistence**. In an interview, choose each store for a specific access pattern and say why.

Indexes

An index is an extra data structure that lets the database find rows without scanning the whole table, at the cost of extra storage and slower writes, since every write must update every index. Index the columns you filter and sort by in hot queries, and no more. A **composite index** on (user_id, created_at) serves "this user's posts, newest first" with one lookup.

Two Storage Engines

Underneath, most databases use one of two families of storage engine.

B-trees (Bayer and McCreight, 1972) keep data in fixed-size pages organized as a balanced tree, and update pages in place. Reads are predictable, a few page reads per lookup, which is why B-trees power most relational databases.

Log-structured merge trees (LSM-trees, described by O'Neil and colleagues in 1996) never update in place. Writes go to an in-memory table; when it fills up, it's written to disk as an immutable, sorted file. Reads check memory first, then the files from newest to oldest. Background **compaction** merges files and discards overwritten values. Sequential writes make LSM-trees excellent for write-heavy workloads; the cost is that a read may check several files, which is mitigated with Bloom filters (Chapter 13). Wide-column stores such as Cassandra and many key-value stores use LSM-trees.

The model below captures the essential mechanism in a few lines:

```
class LSMStore:
    def __init__(self, memtable_limit=4):
        self.memtable = {}
        self.runs = []  # list of sorted (key, value) lists, oldest first
        self.limit = memtable_limit

    def put(self, key, value):
        self.memtable[key] = value
        if len(self.memtable) >= self.limit:
            self.runs.append(sorted(self.memtable.items()))
            self.memtable = {}

    def delete(self, key):
        self.put(key, DELETED)

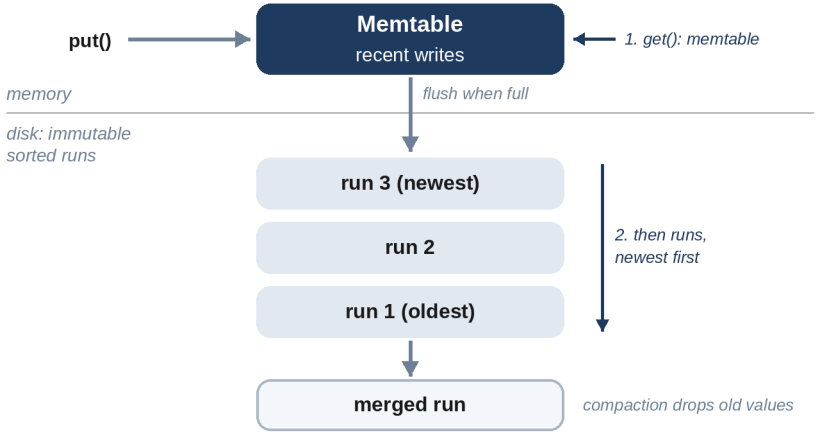
    def get(self, key):
        if key in self.memtable:
            value = self.memtable[key]
        else:
            value = None
            for run in reversed(self.runs):  # newest run first
                i = bisect_left(run, (key,))
                if i < len(run) and run[i][0] == key:
                    value = run[i][1]
                    break
        return None if value is DELETED else value

    def compact(self):
        merged = {}
        for run in self.runs:  # later runs overwrite earlier ones
            merged.update(run)
        live = [
            (k, v) for k, v in sorted(merged.items()) if v is not DELETED
```

```
        ]
        self.runs = [live] if live else []
```

A **tombstone** marks a deleted key, because an older file may still contain a value for it; compaction finally removes both. The tests compare the store with a plain dictionary over thousands of random sequences of writes, deletes, reads and compactions:

```
>>> session(
...     [('put', 'k1', 1),
...      ('put', 'k2', 2),
...      ('put', 'k1', 3),
...      ('del', 'k2'),
...      ('put', 'k3', 4),
...      ('get', 'k1'),
...      ('get', 'k2'),
...      ('compact',),
...      ('get', 'k3')],
... )
[3, None, 4]
```



An LSM-tree: writes go to an in-memory table, which is flushed to immutable sorted runs on disk; reads check memory, then runs from newest to oldest; compaction merges runs.

	B-tree	LSM-tree
Writes	In place; random I/O	Appends; sequential I/O; very fast
Reads	Predictable, a few page reads	May check several files; Bloom filters help
Space	Some fragmentation	Old versions until compaction
Typical use	Relational databases, read-heavy data	Write-heavy data: logs, messages, metrics

Object Storage

Large binary objects (photos, videos, backups, files) go in **object storage**: a flat namespace of keys mapping to immutable blobs, built for durability and cheap capacity rather than low latency. Store the object there and its metadata (owner, size, location key) in a database. Serve popular objects through a CDN.

Choosing a Database: a Checklist

1. **What are the access patterns?** Lookups by key, range scans, joins, full-text search, graph traversals?
2. **What's the read/write ratio and volume?** From your estimates.
3. **Do you need transactions across records?** Money and inventory usually do.
4. **How much data, and how fast does it grow?** Does it fit on one machine for the next few years?
5. **What consistency do readers need?** Chapter 8.

> **Common Mistake:** Choosing a NoSQL database "because it scales" when the data fits comfortably on one relational server. Name the access pattern that the chosen store serves better, or choose the simpler option.

Key Takeaways

- Relational databases are the default for structured data that needs transactions; their limit is write scaling.
- NoSQL families serve specific access patterns; choose by pattern, not by fashion.
- B-trees update in place and read predictably; LSM-trees append and excel at writes.
- Put large objects in object storage and their metadata in a database.

CHAPTER 8

Replication and Consistency

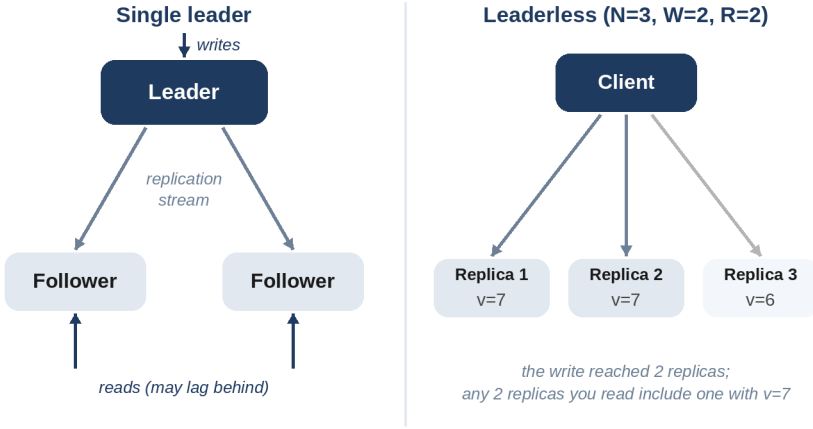
Replication means keeping copies of the same data on several machines. It serves three goals: **availability** (if one copy is lost, others remain), **read scaling** (spread reads across copies), and **latency** (keep a copy near users in each region). Its cost is that copies can disagree, at least for a while, and every design must decide how much disagreement its users can tolerate.

Three Ways to Replicate

Single leader. One replica, the leader, accepts writes and streams them to followers, which serve reads. Simple, and the most common setup for relational databases. The leader is a bottleneck for writes and a single point of failure until a follower is promoted (failover).

Multi-leader. Several replicas accept writes, usually one per region, and exchange changes. Writes are fast everywhere, but two leaders can change the same record at the same time, so conflicts must be resolved: last write wins, a merge function, or keeping both versions for the application to reconcile.

Leaderless. Any replica accepts writes; clients write to and read from several replicas at once, using **quorums** to make sure reads see recent writes. This is the approach described in Amazon's Dynamo paper (DeCandia and colleagues, 2007) and used by stores such as Cassandra.



Single-leader replication: the leader takes writes and streams them to followers that serve reads. Leaderless replication: clients write to W replicas and read from R replicas.

Synchronous or Asynchronous?

A leader can wait for followers to confirm a write before acknowledging it (**synchronous**), or acknowledge immediately and replicate in the background (**asynchronous**). Synchronous replication means no acknowledged write is lost when the leader dies, but every write waits for the slowest follower. Asynchronous replication is fast but can lose recent writes on failover, and followers lag behind the leader, so a user may not see their own update if their next read goes to a follower. A common compromise is one synchronous follower and the rest asynchronous.

Quorums

With **N** replicas, a write succeeds once **W** of them acknowledge it, and a read asks **R** replicas and takes the newest value it gets. If **R + W > N**, every set of R replicas overlaps every set of W replicas in at least one machine, so every read reaches at least one replica with the latest acknowledged write.

The model below writes a new version to W random replicas, then reads R random replicas, many thousands of times:

```
def stale_reads(n, w, r, trials=20_000, seed=3):
    """Fraction of reads that miss the latest acknowledged write."""
    rng = random.Random(seed)
    replicas = [0] * n  # version stored on each replica
    version, stale = 0, 0
    for _ in range(trials):
        version += 1
        for i in rng.sample(range(n), w):  # the write reaches W replicas
            replicas[i] = version
        seen = max(replicas[i] for i in rng.sample(range(n), r))
        stale += seen < version
    return stale / trials
```

```
N=3 W=1 R=1 (R + W <= N): 67% of reads are stale
N=3 W=2 R=1 (R + W <= N): 33% of reads are stale
N=3 W=1 R=3 (R + W > N): 0% of reads are stale
N=3 W=2 R=2 (R + W > N): 0% of reads are stale
N=3 W=3 R=1 (R + W > N): 0% of reads are stale
```

With N = 3, W = 2 and R = 2, reads are never stale; with W = 1 and R = 1, two thirds are. The test checks the rule for every combination of N, W and R up to 5: zero stale reads whenever R + W > N, and some stale reads otherwise.

Choosing the numbers is a trade-off:

- **W = N, R = 1:** fast reads, but a write fails if any replica is down.
- **W = 1, R = N:** fast writes, slow reads.
- **W = R = majority** (2 of 3, 3 of 5): balanced, and tolerates a minority of failures for both reads and writes. The usual choice.

> **Warning:** R + W > N is necessary but not sufficient for strong consistency in real systems. Concurrent writes, clock-based conflict resolution and replicas that are replaced can still produce surprises. For "exactly one winner" decisions, such as leader election or locks, use a consensus protocol.

Consistency Models

From strongest to weakest, the guarantees you'll be asked about:

- **Linearizable (strong):** the system behaves as if there were a single copy; once a write is acknowledged, every later read sees it. Needed for locks, unique usernames, account balances.
- **Read-your-writes:** a user always sees their own updates, though others may see them later. Implemented by routing a user's reads to the leader for a short time after they write, or by tracking the version they last wrote.
- **Monotonic reads:** a user never sees data go backwards in time, for example by always reading from the same replica.
- **Eventual:** if writes stop, all replicas eventually agree. Fine for likes, view counts and recommendations.

Choose per feature, not per system. A social app can show eventually consistent like counts while requiring strong consistency for changing a password.

CAP and PACELC

The **CAP theorem** (conjectured by Eric Brewer in 2000, proven by Gilbert and Lynch in 2002) says that when a **network partition** splits a distributed system, it must choose between **consistency** (refuse requests that might return stale data) and **availability** (answer anyway, possibly with stale data). Since partitions do happen, the practical question is what each part of your system should do during one.

PACELC (Daniel Abadi, 2012) adds the more common case: *else*, when there's no partition, there's still a trade-off between **latency** and **consistency**, because waiting for replicas to agree takes time. It's a more useful lens for everyday design decisions.

Consensus, Briefly

When replicas must agree on one value despite failures, such as which node is the leader, or whether a lock is held, systems use a **consensus** protocol: Paxos (Lamport, published 1998) or Raft (Ongaro and Ousterhout, 2014). They

work as long as a majority of nodes can communicate, which is why clusters have an odd number of members, 3 or 5. In interviews, you rarely need the details; you need to know when to use a consensus-backed service (such as etcd or ZooKeeper) instead of inventing your own coordination.

Key Takeaways

- Replication buys availability, read scaling and locality, at the cost of possible disagreement between copies.
- Single-leader is simplest; multi-leader and leaderless trade simplicity for write availability.
- Quorums guarantee overlap when R + W > N; majorities for both are the usual choice.
- Pick consistency per feature: strong for money and identity, eventual for counts and feeds.
- Use an existing consensus service for coordination rather than building one.

CHAPTER 9

Partitioning and Consistent Hashing

Replication copies all the data to several machines; **partitioning** (also called **sharding**) splits the data so that each machine holds only part of it. Partitioning is how a system stores more data, or handles more writes, than one machine can. Most large designs use both: data is split into partitions, and each partition is replicated.

Choosing a Partition Key

The **partition key** decides which partition a record lives on. A good key:

- **Spreads load evenly**, so no partition is much busier than the others.
- **Keeps related data together**, so common queries touch one partition. All of a user's messages on one partition makes "load this user's inbox" a single lookup.

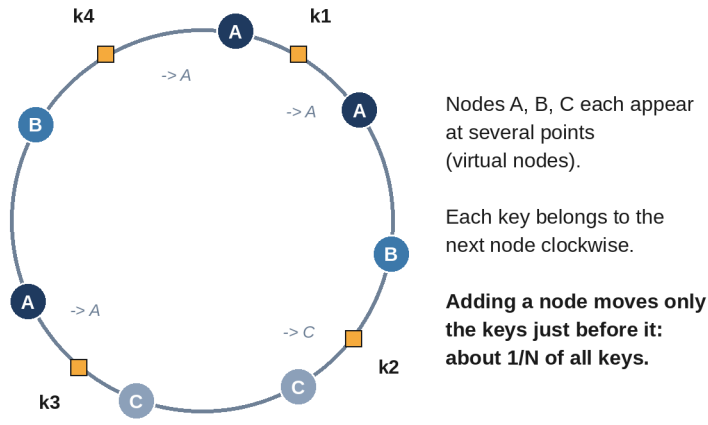
Two ways to map keys to partitions:

- **Range partitioning:** each partition holds a contiguous range of keys (A to F, G to M, ...). Range scans are efficient, but sequential keys such as timestamps all land on the newest partition, creating a hot spot.
- **Hash partitioning:** hash the key and use the hash to choose a partition. Load spreads evenly, but range scans across keys must ask every partition.

The Problem With Hash Mod N

The obvious hash scheme is `partition = hash(key) % N` for N servers. It works perfectly until N changes. Add one server and almost every key maps to a different partition, so nearly all data must move at once.

Consistent hashing (Karger and colleagues, 1997) fixes this. Servers and keys are hashed onto the same circle of values. Each key belongs to the first server found clockwise from its position. When a server joins, it takes over only the keys between itself and its predecessor; when a server leaves, only its keys move, to its successor.



Consistent hashing: servers and keys are placed on a ring by their hashes; each key belongs to the next server clockwise. Each server appears at many points (virtual nodes) to spread load evenly.

With one point per server, the arcs between servers vary a lot in size, so load is uneven. The fix is **virtual nodes**: each server appears at many points on the ring, so its total share averages out. Here is a complete implementation:

```
def h(text):
    return int.from_bytes(hashlib.md5(text.encode()).digest()[:8], "big")


class HashRing:
    def __init__(self, nodes, vnodes=100):
        self.vnodes = vnodes
```

```
        self.points = []  # sorted (hash, node) pairs around the ring
        for node in nodes:
            self.add(node)

    def add(self, node):
        for i in range(self.vnodes):
            self.points.append((h(f"{node}#{i}"), node))
        self.points.sort()

    def remove(self, node):
        self.points = [p for p in self.points if p[1] != node]

    def owner(self, key):
        i = bisect(self.points, (h(key), "")) % len(self.points)
        return self.points[i][1]  # first point clockwise from the key
```

The model places 20,000 keys on 10 servers, then adds an 11th:

```
adding an 11th node: ring moves 9% of keys,
                     mod-N moves 91% of keys
  1 virtual nodes each: busiest node has 3.38x its share
 10 virtual nodes each: busiest node has 1.41x its share
100 virtual nodes each: busiest node has 1.16x its share
```

Consistent hashing moves 9% of keys, close to the ideal 1/11; hash mod N moves 91%. With one virtual node per server, the busiest server holds over three times its fair share; with 100, about 16% more than its share. The test also checks the key property directly: when a server is added, every key either stays where it was or moves to the new server, and removing the server returns every key to its old place.

> **Interviewer's View:** "Consistent hashing with virtual nodes, so adding a node moves about 1/N of the keys" is the expected answer whenever a design must add or remove storage or cache servers. Being able to explain *why* virtual nodes help is what distinguishes a strong answer.

Hot Partitions

Even a perfect hash can't spread load that is concentrated on one key: a celebrity's account, a viral post, a flash sale on one product. Options:

- **Split the hot key:** append a suffix (`post:123#0` to `post:123#9`) to spread writes across partitions, and combine the parts on read.
- **Cache aggressively** in front of the hot partition.

- **Treat big accounts differently**, as the news feed design does for celebrities (Chapter 22).

Rebalancing and Routing

When partitions move, clients must find them. Three approaches: clients ask any node, which forwards the request; a routing tier knows the partition map; or clients cache the map themselves. The partition map itself is usually kept in a consensus-backed configuration service so all components agree on it.

Moving a partition means copying its data while it keeps serving traffic, then switching ownership. Do it gradually and throttle it, so rebalancing doesn't overload the cluster it's meant to help.

Queries Across Partitions

Partitioning makes some operations expensive:

- **Secondary indexes:** "find posts by hashtag" when posts are partitioned by user. Either every partition keeps a local index and queries ask all of them (scatter-gather), or a global index is partitioned by the indexed value and updated asynchronously.
- **Joins and transactions across partitions** require coordination, which is slow. Choose partition keys so the most common operations stay within one partition.

Key Takeaways

- Partition to handle more data or writes than one machine can; replicate each partition for availability.
- Choose a partition key that spreads load and keeps related data together.
- Hash mod N moves almost all data when N changes; consistent hashing moves about 1/N.
- Virtual nodes even out the load on a hashing ring.
- Plan for hot keys and for the cost of cross-partition queries.

CHAPTER 10

Queues, Logs and Delivery Guarantees

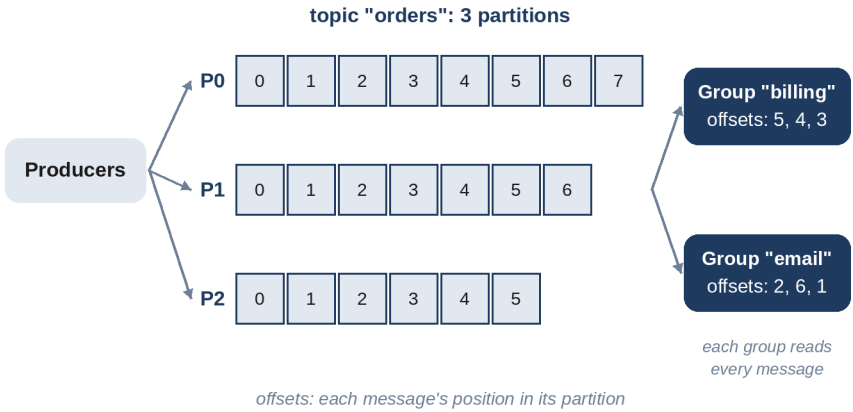
Synchronous calls tie services together: if the email service is slow, the sign-up request that waits for it is slow too, and if it's down, sign-up fails. Putting a **message queue** between them breaks that tie. The sign-up service records "send a welcome email" and responds immediately; the email service works through messages at its own pace. Queues absorb bursts, isolate failures and let each side scale independently, and they appear in almost every design in Part III.

Queues and Logs

There are two main families:

- **Message queues** (such as RabbitMQ or Amazon SQS) deliver each message to one consumer and delete it once it's acknowledged. They are ideal for distributing tasks among workers.
- **Logs** (such as Kafka, described by Kreps, Narkhede and Rao in 2011) append messages to an ordered, durable log split into **partitions**. Messages aren't deleted when read; each consumer group tracks its own position, its **offset**. Several independent consumers can read the same stream, and a consumer can replay history.

Use a queue for "do this task once"; use a log when several systems need the same stream of events, or when order and replay matter. Chapter 35 designs a distributed log from scratch.



Producers append messages to a partitioned log; each consumer group reads every partition and stores its own offsets.

Delivery Guarantees

Networks and processes fail, so a message system can promise one of three things:

- **At most once:** a message may be lost, but never delivered twice. Acknowledge before processing.
- **At least once:** a message is never lost, but may be delivered more than once. Acknowledge after processing; if the acknowledgement is lost, the message is delivered again.
- **Exactly once:** each message affects the system exactly once. Hard in general; in practice it's built from at-least-once delivery plus **idempotent processing**.

The model below sends 1,000 payments of 25 through a queue in which 20% of acknowledgements are lost, so those messages are redelivered:

```
def run(messages, idempotent, ack_loss=0.2, seed=5):
    rng = random.Random(seed)
    queue = [(f"m{i}", amount) for i, amount in enumerate(messages)]
    balance, seen, deliveries = 0, set(), 0
    while queue:
        msg_id, amount = queue.pop(0)
        deliveries += 1
        if not (idempotent and msg_id in seen):
            balance += amount  # the side effect
            seen.add(msg_id)
        if rng.random() < ack_loss:  # ack lost: the broker redelivers
            queue.append((msg_id, amount))
    return balance, deliveries
```

```
naive consumer     : 1273 deliveries, total 31825 (expected 25000)
idempotent consumer: 1273 deliveries, total 25000 (expected 25000)
```

Both consumers received 1,273 deliveries. The naive one applied every delivery and overstated the total by more than a quarter; the idempotent one remembered message IDs and applied each payment exactly once. The tests check that the idempotent consumer's total always equals the true total on thousands of random message sequences.

> **Common Mistake:** Designing a consumer that assumes each message arrives once. With at-least-once delivery, the most common choice, every consumer must be idempotent: record processed message IDs, use database upserts keyed by message ID, or make the operation naturally repeatable ("set status to shipped" rather than "add one to the shipped count").

Ordering

Logs guarantee order **within a partition**, not across partitions. If the events for one order must be processed in sequence, use the order ID as the partition key, so all its events land in the same partition. Global ordering across all events requires a single partition, which limits throughput; it's rarely needed.

Backpressure and Dead Letters

- **Backpressure:** when consumers fall behind, the queue grows. Monitor **consumer lag** (how far behind the newest message each consumer is) and scale consumers, or slow producers, before the backlog becomes hours long.
- **Poison messages:** a message that always makes the consumer crash will be redelivered forever. After a few failed attempts, move it to a **dead-letter queue** for inspection, so it doesn't block the rest.
- **Retries with backoff:** when a downstream dependency is failing, retrying immediately makes it worse (Chapter 14).

The Transactional Outbox

A common bug: a service updates its database and then publishes an event, and crashes between the two. The database says the order was placed, but no event was sent. The **outbox pattern** fixes it: write the event into an "outbox" table in the same database transaction as the change, and have a separate process publish rows from the outbox to the queue. The event may be published twice, but never lost, and idempotent consumers handle the duplicates.

Key Takeaways

- Queues decouple services, absorb bursts and isolate failures.
- Use a task queue for work distribution and a partitioned log for event streams that several consumers read or replay.
- At-least-once delivery plus idempotent consumers is the practical route to exactly-once effects.
- Order is guaranteed only within a partition; partition by the entity whose events must stay in order.
- Monitor consumer lag, use dead-letter queues, and publish events with the outbox pattern.

CHAPTER 11

Rate Limiting

A rate limiter caps how many requests a client can make in a period: 100 requests per minute per API key, five login attempts per hour per account. It protects services from overload, from abusive or buggy clients, and from runaway costs, and it's a classic interview topic both as a building block and as a complete design (Chapter 17).

Where to Limit

- **At the edge** (API gateway or load balancer): stops excess traffic before it costs anything. The usual place.
- **In each service:** protects a service from its own callers, including internal ones.
- **On the client:** well-behaved clients slow themselves down when told to, using the `Retry-After` header from a 429 response.

Limits are keyed by something that identifies the caller: API key, user ID, IP address, or a combination. Different endpoints usually have different limits; a search endpoint is more expensive than a profile read.

Four Algorithms

All four limiters below have the same interface: `allow(t)` returns whether a request at time `t` is allowed, for a limit of `limit` requests per `window` seconds.

Fixed window counts requests in each calendar window (each minute, say) and resets the count when a new window starts. It's the simplest and uses one counter, but it has a flaw at window boundaries.

```
class FixedWindow:
    def __init__(self, limit, window):
        self.limit, self.window = limit, window
        self.current, self.count = None, 0

    def allow(self, t):
        w = int(t // self.window)  # which window t falls in
        if w != self.current:
            self.current, self.count = w, 0
        if self.count < self.limit:
            self.count += 1
            return True
        return False
```

Sliding log remembers the time of every allowed request in the last window, and allows a new one only if fewer than `limit` remain. It's exact, but memory grows with the limit.

```
class SlidingLog:
    def __init__(self, limit, window):
        self.limit, self.window = limit, window
        self.log = deque()  # times of allowed requests

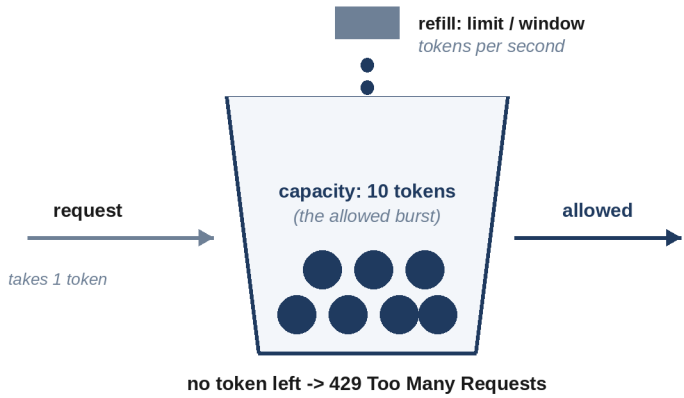
    def allow(self, t):
        while self.log and self.log[0] <= t - self.window:
            self.log.popleft()
        if len(self.log) < self.limit:
            self.log.append(t)
            return True
        return False
```

Sliding window counter approximates the sliding log with two counters: the current window's count, plus the previous window's count weighted by how much of it still overlaps the sliding window. Cheap, and accurate enough for most uses.

Token bucket holds up to `limit` tokens and refills continuously at `limit / window` tokens per second; each request takes a token. It allows short bursts up to the bucket size while enforcing the average rate, which is often exactly what an API wants.

```
class TokenBucket:
    def __init__(self, limit, window):
        self.capacity, self.rate = limit, limit / window
        self.tokens, self.last = float(limit), None

    def allow(self, t):
        if self.last is not None:
            self.tokens = min(
                self.capacity, self.tokens + (t - self.last) * self.rate
            )
        self.last = t
        if self.tokens >= 1:
            self.tokens -= 1
            return True
        return False
```



A token bucket: tokens drip in at a fixed rate up to the bucket's capacity; each request removes one token; with no tokens left, requests are rejected.

The sliding log is tested against a brute-force version that recounts recent requests from scratch, on thousands of random request sequences:

```
>>> sliding_log(2, 10, [0, 1, 2, 10, 10.5, 11])
[True, True, False, True, False, True]
```

The Boundary Problem

Now the flaw. With a limit of 10 requests per 60 seconds, a client sends 10 requests at 0:59 and 10 more at 1:00:

```
def demo():
    # 10 requests per 60 s. A client sends 10 at 0:59 and 10 at 1:00.
    times = [59.0] * 10 + [60.0] * 10
    for cls in [FixedWindow, SlidingLog, SlidingCounter, TokenBucket]:
        lim = cls(10, 60)
        allowed = sum(lim.allow(t) for t in times)
        print(
            f"{cls.__name__:<14} allows {allowed} of 20 requests in 2
            ↪ seconds"
        )
```

```
FixedWindow    allows 20 of 20 requests in 2 seconds
SlidingLog     allows 10 of 20 requests in 2 seconds
SlidingCounter allows 10 of 20 requests in 2 seconds
TokenBucket    allows 10 of 20 requests in 2 seconds
```

The fixed window allows all 20 in two seconds, twice the intended rate, because the requests straddle a window boundary. The other three allow 10. In an interview, mentioning this weakness when someone proposes a fixed window is a quick way to show depth.

Algorithm	Memory per key	Accuracy	Bursts
Fixed window	1 counter	Up to 2x at boundaries	Uncontrolled at boundaries
Sliding log	One timestamp per request	Exact	None beyond the limit
Sliding window counter	2 counters	Close approximation	Smoothed
Token bucket	2 numbers (tokens, last time)	Exact average rate	Allowed up to bucket size

> **Tip:** A token bucket is the best default. It's cheap, it allows the short bursts that real clients produce (a page load fires several API calls at once), and it's easy to explain.

Limiting Across Many Servers

With one gateway, the limiter's state lives in its memory. With twenty gateways, a client's requests are spread across all of them, so the state must be shared, typically in a fast store such as Redis, and updated **atomically**. Chapter 17 shows what goes wrong without atomic updates, and how to keep the extra latency to about a millisecond.

Responding to Limited Clients

Return HTTP **429 Too Many Requests** with a `Retry-After` header, and ideally headers showing the limit and how much remains, so clients can slow down. Decide whether to **fail open** (allow traffic) or **fail closed** (reject) if the limiter's store is unavailable; for most APIs, failing open briefly is better than an outage caused by the limiter itself.

Key Takeaways

- Rate limit at the edge, keyed by API key, user or IP, with per-endpoint limits.
- Fixed windows are simple but allow double the rate across a boundary.
- Sliding logs are exact but memory-hungry; sliding counters are cheap approximations.
- Token buckets enforce an average rate while allowing bounded bursts: the usual default.
- Across many servers, keep the state in a shared store and update it atomically.

CHAPTER 12

Unique IDs and Time

Every record needs an identifier, and in a distributed system generating them is surprisingly subtle. A single database's auto-increment counter becomes a bottleneck and a single point of failure. Random IDs are easy to generate anywhere, but they're long and don't sort by time. And ordering events by time is itself tricky, because the clocks on different machines never agree exactly. This chapter covers the standard solutions.

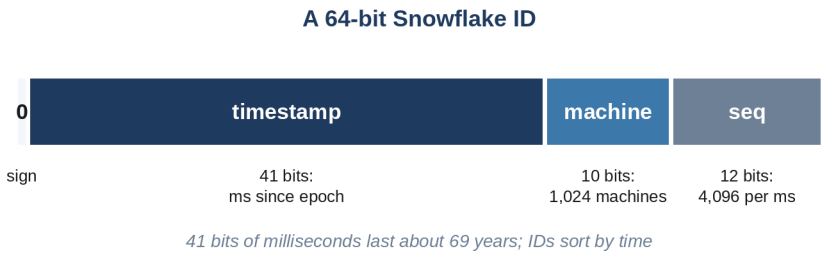
Options for Unique IDs

Approach	Size	Sortable by time	Coordination	Notes
Database auto-increment	64 bits	Yes	Every ID	Simple; a bottleneck at scale
UUID version 4 (random)	128 bits	No	None	Collisions are practically impossible; poor index locality
UUID version 7 (RFC 9562, 2024)	128 bits	Yes, by millisecond	None	Starts with a 48-bit Unix timestamp in milliseconds
Snowflake-style	64 bits	Yes, by millisecond	Assign machine IDs once	Compact; depends on reasonable clocks
Range allocation	64 bits	Roughly	One call per block	A central service hands out blocks of IDs (Chapter 19)

Sortable IDs matter more than they might seem: databases insert them at the end of their indexes instead of at random positions, which keeps writes fast, and "newest first" queries can sort by ID alone.

Snowflake IDs

The Snowflake format, announced by Twitter in 2010, packs three fields into a 64-bit integer:



A Snowflake ID: one unused sign bit, 41 bits of milliseconds since a custom epoch, 10 bits of machine ID and 12 bits of sequence number.

- **41 bits of time** in milliseconds since a chosen epoch: enough for about 69 years.
- **10 bits of machine ID**: up to 1,024 generators.
- **12 bits of sequence**: up to 4,096 IDs per millisecond per machine.

Each machine generates IDs alone, with no coordination after its machine ID is assigned:

```
class SnowflakeGenerator:
    def __init__(self, machine_id, clock_ms):
        if not 0 <= machine_id < 1024:
            raise ValueError("machine_id must fit in 10 bits")
        self.machine, self.clock = machine_id, clock_ms
        self.last_ms, self.sequence = -1, 0

    def next_id(self):
        now = self.clock()
        if now < self.last_ms:
            raise RuntimeError(
                "clock moved backwards; refusing to reuse IDs"
            )
        if now == self.last_ms:
            self.sequence = (self.sequence + 1) & 0xFFF
            if self.sequence == 0:  # 4,096 IDs used this millisecond
```

```
                while now <= self.last_ms:
                    now = self.clock()  # wait for the next millisecond
        else:
            self.sequence = 0
        self.last_ms = now
        return (now - EPOCH_MS) << 22 | self.machine << 12 | self.sequence
```

Two details matter. When a machine uses all 4,096 sequence numbers in one millisecond, it waits for the next millisecond. And if the clock moves backwards, for example after a time-synchronization correction, it refuses to generate IDs rather than risk duplicates. Decoding shows the fields:

```
>>> ids_for([1767225600005, 1767225600005, 1767225600006], 7)
[{'time_ms': 1767225600005, 'machine': 7, 'sequence': 0},
 {'time_ms': 1767225600005, 'machine': 7, 'sequence': 1},
 {'time_ms': 1767225600006, 'machine': 7, 'sequence': 0}]
```

The tests generate 15,000 IDs from three machines sharing a jittery clock, and check that all are unique and that each machine's IDs increase. They also confirm the 69-year lifetime of 41 bits of milliseconds and that a backwards clock raises an error.

Short IDs With Base 62

When IDs appear in URLs, such as a link shortener's codes (Chapter 16), they are written in **base 62**, using digits and upper- and lower-case letters:

```
def encode(n):
    if n == 0:
        return ALPHABET[0]
    out = []
    while n:
        n, r = divmod(n, 62)
        out.append(ALPHABET[r])
    return "".join(reversed(out))


def decode(s):
    n = 0
    for ch in s:
        n = n * 62 + ALPHABET.index(ch)
    return n
```

Seven base-62 characters give 62^7, about 3.5 trillion, distinct codes; the test checks that $62^7 - 1$ is the largest number with seven characters.

Ordering Events Without Trusting Clocks

Machine clocks drift, and synchronization (NTP) can be off by milliseconds or more, so timestamps from different machines can't reliably order events that happen close together. Two logical clocks solve parts of the problem without relying on physical time.

Lamport clocks (Leslie Lamport, 1978) give every event a counter. A process increments its counter for each event; a message carries the sender's counter, and the receiver jumps its counter past it. If event A could have caused event B, A's number is smaller:

```
def lamport(events):
    """events: list of (process, kind, msg) where kind is 'local', 'send'
    or 'recv'; returns the Lamport time of each event."""
    clock, sent, out = {}, {}, []
    for proc, kind, msg in events:
        t = clock.get(proc, 0) + 1
        if kind == "recv":
            t = max(t, sent[msg] + 1)
        if kind == "send":
            sent[msg] = t
        clock[proc] = t
        out.append(t)
    return out
```

```
>>> lamport(
...     [('P', 'local', None),
...      ('P', 'send', 'm1'),
...      ('Q', 'local', None),
...      ('Q', 'recv', 'm1'),
...      ('Q', 'send', 'm2'),
...      ('P', 'recv', 'm2')],
... )
[1, 2, 1, 3, 4, 5]
```

Vector clocks keep one counter per process, and can tell whether two versions are ordered or truly **concurrent**, which is exactly what a replicated store needs to detect conflicting writes. Two replicas of a shopping cart that accept changes while disconnected produce concurrent versions; merging them and recording a new event produces a version that comes after both:

```
def compare(a, b):
    """Compare two vector clocks (dicts): 'before', 'after', 'equal' or
    'concurrent'."""
    keys = set(a) | set(b)
    le = all(a.get(k, 0) <= b.get(k, 0) for k in keys)
    ge = all(a.get(k, 0) >= b.get(k, 0) for k in keys)
    if le and ge:
```

```
        return "equal"
    if le:
        return "before"
    if ge:
        return "after"
    return "concurrent"


def merge(a, b):
    return {k: max(a.get(k, 0), b.get(k, 0)) for k in set(a) | set(b)}
```

```
A's version vs B's version: concurrent
resolved vs A's version:    after
resolved vs B's version:    after
```

Many systems choose the simpler **last write wins**, keeping the version with the latest timestamp. It silently discards one of two concurrent writes, which is fine for a profile picture and wrong for a shopping cart. Saying which you'd use, and why, is the point.

> **Interviewer's View:** You won't be asked to implement vector clocks. You may be asked "two users edit the same record at the same time on different replicas; what happens?" The strong answer names the options (last write wins, vector clocks with application merge, or avoiding the conflict with a single leader per record) and picks one for the use case.

Key Takeaways

- Auto-increment IDs don't scale; random UUIDs don't sort; time-ordered IDs (UUIDv7, Snowflake) do both.
- Snowflake IDs pack time, machine and sequence into 64 bits, and need no coordination per ID.
- Base 62 turns large numbers into short, URL-safe codes.
- Physical clocks can't order close events across machines; Lamport and vector clocks track causality instead.
- Last write wins is simple but loses concurrent updates; choose it deliberately.

CHAPTER 13

Probabilistic Data Structures

Some questions don't need exact answers. "Has this crawler seen this URL before?", "roughly how many times was this search term used today?", "how many distinct users visited?" At large scale, exact answers can need gigabytes of memory, while answers that are almost always right, with a known error, fit in kilobytes. Three structures cover most interview needs.

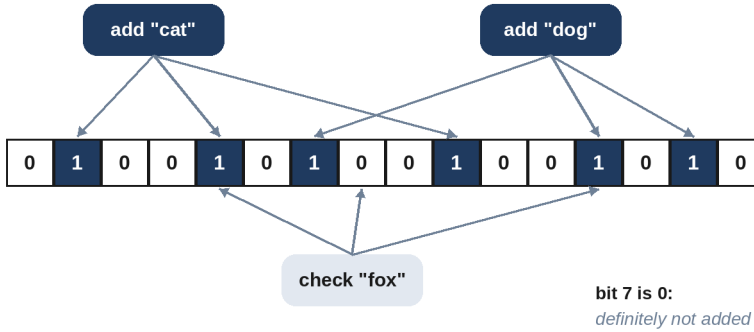
Bloom Filters: "Definitely Not" or "Probably Yes"

A **Bloom filter** (Burton Bloom, 1970) tests set membership. It's an array of bits plus k hash functions. Adding an item sets k bits; checking an item looks at the same k bits. If any is 0, the item was **definitely never added**. If all are 1, it was **probably** added, but those bits might have been set by other items: a false positive. There are never false negatives.

```
class BloomFilter:
    def __init__(self, n_items, fp_rate):
        # Standard sizing formulas for m bits and k hash functions.
        self.m = math.ceil(-n_items * math.log(fp_rate) / math.log(2) ** 2)
        self.k = max(1, round(self.m / n_items * math.log(2)))
        self.bits = bytearray(self.m)

    def _positions(self, item):
        d = hashlib.sha256(item.encode()).digest()
        h1 = int.from_bytes(d[:8], "big")
        h2 = int.from_bytes(d[8:16], "big") | 1
        return [(h1 + i * h2) % self.m for i in range(self.k)]

    def add(self, item):
        for p in self._positions(item):
            self.bits[p] = 1
```



A Bloom filter: each item sets k bits in a bit array; a lookup checks the same bits; any zero means "definitely not present".

```
    def might_contain(self, item):
        return all(self.bits[p] for p in self._positions(item))
```

The size comes from standard formulas: for n items and a target false-positive rate p, use about -n ln p / (ln 2)² bits and (bits / n) ln 2 hash functions. Instead of k independent hash functions, the code derives them from two halves of one SHA-256 hash, a standard trick. For 10,000 items at 1%:

```
95,851 bits (11.7 KB), 7 hash functions
false positive rate: 1.01% (target 1%)
```

Less than 12 KB, and the measured false-positive rate matches the target. Storing 10,000 strings exactly would take far more.

Uses: a database checks a Bloom filter before reading a file that probably doesn't contain the key (the LSM-trees of Chapter 7); a crawler skips URLs it has probably seen (Chapter 20); a browser checks URLs against a list of malicious sites before making a slower exact check. Standard Bloom filters can't delete items; variants such as counting Bloom filters can.

Count-Min Sketch: Approximate Counts

A **Count-Min sketch** (Cormode and Muthukrishnan, 2005) estimates how often each item occurs in a stream, in fixed memory. It's a small table of counters with d rows, each with its own hash function. Adding an item increments one counter in each row; the estimate is the **minimum** of the

item's d counters. Collisions only ever add to counters, so the estimate is never too low, and taking the minimum picks the counter least inflated by collisions.

```
class CountMin:
    def __init__(self, width=1000, depth=4):
        self.w, self.d = width, depth
        self.rows = [[0] * width for _ in range(depth)]

    def _cols(self, item):
        d = hashlib.sha256(item.encode()).digest()
        return [
            int.from_bytes(d[4 * i : 4 * i + 4], "big") % self.w
            for i in range(self.d)
        ]

    def add(self, item, count=1):
        for row, col in zip(self.rows, self._cols(item)):
            row[col] += count

    def estimate(self, item):
        return min(
            row[col] for row, col in zip(self.rows, self._cols(item))
        )
```

The test streams 50,000 items with a heavy-tailed popularity distribution into a 4 × 500 sketch, and checks that no estimate is below the true count and that at most 5% of estimates exceed it by more than the theoretical bound of about e/w of the stream size. Use it to find **heavy hitters**: trending searches (Chapter 24), the most active API clients, top products.

HyperLogLog: Counting Distinct Items

How many distinct users visited today? An exact answer needs to remember every user ID. **HyperLogLog** (Flajolet and colleagues, 2007) estimates it with a few kilobytes. The intuition: hash each item; a hash that starts with many zero bits is rare, so the longest run of leading zeros seen hints at how many distinct items there were. HyperLogLog splits items into many registers and averages their evidence to reduce the error.

```
class HyperLogLog:
    def __init__(self, p=12):
        self.p, self.m = p, 1 << p  # 2^p registers
        self.registers = [0] * self.m

    def add(self, item):
        x = int.from_bytes(
            hashlib.sha256(item.encode()).digest()[:8], "big"
```

```
        )
        idx = x >> (64 - self.p)  # first p bits choose a register
        rest = x & ((1 << (64 - self.p)) - 1)
        rank = (64 - self.p) - rest.bit_length() + 1  # leading zeros + 1
        self.registers[idx] = max(self.registers[idx], rank)

    def count(self):
        alpha = 0.7213 / (1 + 1.079 / self.m)
        raw = alpha * self.m**2 / sum(2.0**-r for r in self.registers)
        zeros = self.registers.count(0)
        if raw <= 2.5 * self.m and zeros:  # small-range correction
            return self.m * math.log(self.m / zeros)
        return raw
```

```
true     1,000  estimate         998  error 0.2%
true   100,000  estimate      99,313  error 0.7%
true 1,000,000  estimate     994,858  error 0.5%
memory: 4096 registers, about 3 KB at 6 bits each
```

With 4,096 registers, about 3 KB, the error stays under 1% from a thousand to a million distinct items. The expected error is about 1.04 / √m for m registers, roughly 1.6% here. Adding duplicates doesn't change the estimate, which the test checks. Registers from different servers can be merged by taking the maximum of each, so each server counts its own traffic and the results combine exactly.

Structure	Answers	Error	Memory
Bloom filter	"Is x in the set?"	False positives only	About 10 bits per item for 1%
Count-Min sketch	"How often has x occurred?"	Overestimates only	Fixed: width × depth counters
HyperLogLog	"How many distinct items?"	About 1.04 / √m relative	Fixed: m small registers

> **Interviewer's View:** Proposing a Bloom filter where exactness isn't required, and stating what a false positive costs ("we skip crawling a few new URLs, which is acceptable"), is a strong signal. Don't use one where a false positive is unacceptable, such as checking whether a username is taken.

Key Takeaways

- Probabilistic structures trade a small, known error for huge memory savings.
- Bloom filters answer membership with no false negatives and a tunable false-positive rate.
- Count-Min sketches estimate frequencies, never underestimating.
- HyperLogLog counts distinct items in kilobytes, and merges across servers.
- Always say what an error costs in your design.

CHAPTER 14

Reliability: Timeouts, Retries and Circuit Breakers

In a large system, something is always failing: a disk, a server, a network link, a dependency in another team. Reliability doesn't come from preventing every failure; it comes from making sure that one failure stays small instead of spreading. This chapter covers the patterns that do that, and that interviewers expect in the wrap-up of any design.

Timeouts

Every network call needs a **timeout**. Without one, a slow dependency makes callers wait, their threads and connections fill up, and soon the callers are as broken as the dependency. Set timeouts from the dependency's measured latency, a little above its 99th percentile (Chapter 15), and make sure the total of nested timeouts fits within the caller's own deadline. Passing the remaining time budget along with each call ("you have 300 ms left") lets every layer give up promptly.

Retries With Backoff and Jitter

Many failures are brief: a dropped packet, a server restarting. Retrying helps, but naive retries make outages worse. If a service struggles under load and every client retries immediately, the load multiplies, and the service never recovers: a **retry storm**.

The standard fix is **exponential backoff**: wait longer after each failure (0.5 s, 1 s, 2 s, 4 s...), up to a cap. But if all clients failed at the same moment, they all back off by the same amounts and retry in synchronized waves. Adding randomness, **jitter**, spreads them out. With "full jitter", popularized by Marc Brooker's 2015 analysis on the AWS Architecture Blog, each wait is a random time between zero and the exponential limit:

```
def backoff_delays(attempts, base=0.1, cap=10.0, rng=None):
    """Full jitter: wait a random time between 0 and the exponential cap."""
    rng = rng or random.Random()
    return [rng.uniform(0, min(cap, base * 2**i)) for i in range(attempts)]
```

The model makes 1,000 clients retry five times after the same outage and counts the busiest 10 milliseconds of retries:

```
busiest 10 ms of retries from 1,000 clients:
  fixed exponential backoff: 1000
  with full jitter:          38
```

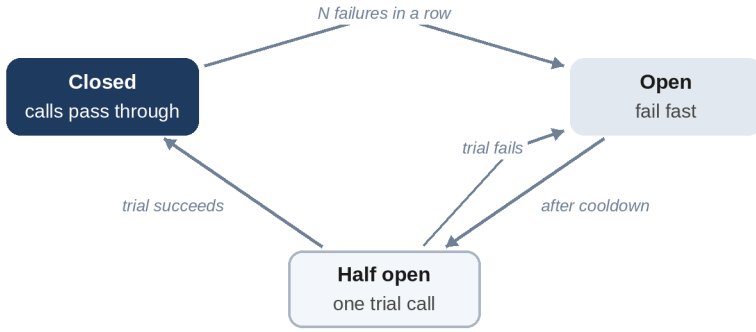
Without jitter, all 1,000 retries arrive in the same instant, five times over. With full jitter, the worst 10 ms sees 38.

Rules for safe retries:

- Retry only **idempotent** operations, or use idempotency keys (Chapter 4).
- Retry only **transient** errors (timeouts, 503), not "bad request" errors.
- Limit attempts, and use a **retry budget**: for example, retries may add at most 10% to normal traffic.
- Retry at one layer only. If three layers each retry three times, one user request can become 27 calls.

Circuit Breakers

When a dependency is clearly down, even well-behaved retries waste time and resources. A **circuit breaker**, a pattern popularized by Michael Nygard's book *Release It!* (2007), stops calling it for a while:



Circuit breaker states: closed (calls pass through), open after repeated failures (calls fail fast), half open after a cooldown (one trial call decides).

```
class CircuitBreaker:
    """closed -> open after `threshold` failures in a row; open -> half
    open after `cooldown` seconds; one trial call closes or reopens it."""

    def __init__(self, threshold, cooldown, clock):
        self.threshold, self.cooldown, self.clock = (
            threshold,
            cooldown,
            clock,
        )
        self.state, self.failures, self.opened_at = "closed", 0, None

    def call(self, fn):
        if self.state == "open":
            if self.clock() - self.opened_at < self.cooldown:
                raise RuntimeError("circuit open: failing fast")
            self.state = "half open"
        try:
            result = fn()
        except Exception:
            self.failures += 1
            if self.state == "half open" or self.failures >= self.threshold:
                self.state, self.opened_at = "open", self.clock()
            raise
        self.state, self.failures = "closed", 0
        return result
```

- **Closed:** calls pass through; consecutive failures are counted.
- **Open:** after `threshold` failures, calls fail immediately without touching the dependency, giving it time to recover and keeping the caller fast.
- **Half open:** after a cooldown, one trial call is allowed. Success closes the breaker; failure opens it again.

The examples step through those transitions with a threshold of three failures and a 30-second cooldown. In the first, the breaker opens after three failures, rejects a call at second 3 while open, then lets a trial call succeed at second 40 and closes. In the second, the trial call fails and it opens again:

```
>>> breaker_states(
...     [False, False, False, True, True, False, True],
...     [0, 1, 2, 3, 40, 41, 80],
... )
['closed', 'closed', 'open', 'open', 'closed', 'closed', 'closed']
>>> breaker_states([False, False, False, False], [0, 1, 2, 40])
['closed', 'closed', 'open', 'open']
```

While the breaker is open, return a **fallback** if one makes sense: cached data, a default, or a degraded feature ("recommendations unavailable") rather than an error page.

Bulkheads, Load Shedding and Graceful Degradation

- **Bulkheads:** give each dependency its own pool of threads or connections, so one slow dependency can't consume them all. Named after the watertight compartments of a ship.
- **Load shedding:** when overloaded, reject some requests early, cheaply and deliberately (lowest priority first), rather than slowing down for everyone.
- **Graceful degradation:** design features to fail partially. A product page without reviews is better than no product page.

Redundancy and Failure Domains

Remove single points of failure by running at least two of everything that matters, in different **failure domains**: different machines, racks, availability zones and, for the most critical systems, regions. Test failover regularly; a backup that has never been restored, or a standby that has never been promoted, is a hope, not a plan.

Key Takeaways

- Put a timeout on every call and propagate deadlines.
- Retry only transient failures of idempotent operations, with exponential backoff and jitter, at one layer.
- Circuit breakers fail fast when a dependency is down and probe gently for recovery.
- Bulkheads, load shedding and graceful degradation keep one failure from spreading.
- Spread redundant components across failure domains, and test failover.

CHAPTER 15

Observability and Security Essentials

Two topics come up at the end of almost every design interview: "how would you know it's working?" and "how would you keep it secure?" Neither needs a long answer, but having a crisp one shows that you think about systems the way operators and security engineers do.

The Three Signals

- **Metrics:** numbers over time, such as request rate, error rate, latency and queue length. Cheap to store and ideal for dashboards and alerts.
- **Logs:** records of individual events, with context. Essential for investigating what happened to one request.
- **Traces:** the path of one request through many services, with the time spent in each. The fastest way to find which service makes a request slow.

For every service, watch the **rate** of requests, the rate of **errors**, and their **duration**. Add **saturation** (how full the service's resources are: CPU, memory, connection pools, queue length) and you can spot most problems before users report them.

Why Averages Lie

Latency should be reported as **percentiles**, not averages. The model below generates 100,000 request latencies where 98% are fast (20 to 40 ms) and 2% take a slow path such as a cache miss to disk (800 to 1,200 ms):

```
def percentile(values, p):
    """Nearest-rank percentile: the smallest value with at least p% of
    values at or below it."""
    ordered = sorted(values)
    rank = max(1, math.ceil(p / 100 * len(ordered)))
    return ordered[rank - 1]
```

```
mean 49 ms
p50   30 ms
p95   39 ms
p99   1002 ms
p99.9 1180 ms
SLO 99.00%: 432.0 minutes of downtime per 30 days
SLO 99.90%: 43.2 minutes of downtime per 30 days
SLO 99.99%: 4.3 minutes of downtime per 30 days
```

The average is 49 ms, which describes no real request. The median (p50) is 30 ms, and so is the experience of most requests, but the 99th percentile is about a second. If a page makes 50 backend calls, the chance that at least one hits the slow path is $1 - 0.98^{5}$, about 64%: most page loads are slow even though the average looks excellent. That's why service objectives are written for p99 or p99.9, and why reducing tail latency matters so much in large systems.

SLIs, SLOs and Error Budgets

- A **service level indicator (SLI)** is a measurement: the fraction of requests that succeed within 300 ms, for example.
- A **service level objective (SLO)** is a target for it: 99.9% over 30 days.
- The **error budget** is what the SLO allows to fail. The demo above computes it for three common targets: 99% allows about 7 hours of total outage in 30 days, 99.9% about 43 minutes, and 99.99% about 4 minutes.

The error budget turns reliability into a decision rule: while budget remains, ship features; when it's spent, focus on reliability. Every extra "nine" costs much more than the last, so choose targets that users actually need.

Alerting

Alert on **symptoms users feel** (error rate, latency, failed checkouts), not on every cause (one server's CPU), and require a condition to last for a few minutes so a single spike doesn't wake someone at night. Chapter 31 designs a metrics and alerting system and shows this rule in code.

Security Essentials for Designers

You won't be expected to design cryptography, but you should cover these basics:

- **Authentication** (who is calling): sessions or short-lived tokens for users; mutual TLS or signed tokens between services. Never put secrets in URLs or logs.
- **Authorization** (what they may do): check permissions on every request, server-side, for every object. "User 42 may read document 7" must be checked, not assumed from the URL.
- **Encryption:** TLS for data in transit, everywhere, including inside the data center; encryption at rest for stored data, with keys in a managed key service.
- **Least privilege:** each service gets only the permissions and network access it needs.
- **Input validation and abuse prevention:** validate everything from clients, rate limit (Chapter 11), and protect expensive endpoints such as sign-up and search from automated abuse.
- **Secrets management:** credentials in a secrets manager, rotated regularly, never in code.
- **Privacy:** collect only what you need, delete it when you no longer need it, and know which data is personal. Regulations such as the EU's GDPR give users rights over their data, which affects designs that copy data into many systems.

> **Interviewer's View:** A thirty-second wrap-up that covers "dashboards on rate, errors and p99 latency; alerts on SLO burn; TLS everywhere; per-object authorization checks; secrets in a manager" is usually all that's needed, unless the prompt is security-sensitive, such as payments or authentication.

Key Takeaways

- Watch rate, errors, duration and saturation for every service; use traces to find slow components.
- Report latency as percentiles; tails dominate the experience in systems with many calls.
- Set SLOs from user needs and manage reliability with error budgets.
- Alert on user-facing symptoms that persist, not on every spike.
- Cover authentication, per-object authorization, encryption, least privilege and secrets in every design.

PART III

Twenty Worked Designs

Each design in this part follows the framework of Chapter 2: requirements, estimates, API, data model, high-level design, deep dives, and a wrap-up with bottlenecks, failure modes and follow-up questions. The estimates are computed with the helpers from Chapter 3, and the core mechanism of each design is a tested Python model. Read the requirements, then try the design yourself for 30 minutes before reading on.

CHAPTER 16

Link Shortener

The prompt: "Design a service that turns long URLs into short links, like `sho.example/aZ3kQ9x`, and redirects visitors to the original URL."

It's the classic first system design question because it's small enough to finish in 45 minutes, yet it touches ID generation, caching, storage, and abuse prevention.

Requirements

Functional:

1. Create a short link for a long URL, optionally with a custom alias.
2. Visiting the short link redirects to the long URL.
3. Links may have an expiry time.
4. Optional: basic click statistics.

Non-functional:

- Redirects must be fast (under about 50 ms server time) and highly available; a broken link is visible to everyone who clicks it.
- Short codes should be hard to guess sequentially only if the product requires it (ask).
- Link creation can be slightly slower and less available than redirects.

Out of scope: user accounts and link editing, unless the interviewer wants them.

Estimates

Assume 100 million new links a month, 100 redirects per link on average, about 500 bytes per stored link, and a peak of five times the average.

```
new links per second (avg)      38.6
redirects per second (avg)      3.86K
redirects per second (peak x5)  19.3K
storage per year (500 B/link)   608 GB
7-char base-62 codes            3.52T
years until codes run out       2,894
```

- **Read-heavy, by 100 to 1:** caching redirects matters far more than write throughput.
- **About 600 GB a year:** a few terabytes over several years. That fits on one large database server, but replication and possibly partitioning will be wanted for availability and growth.
- **Seven base-62 characters** give 3.5 trillion codes, enough for nearly 3,000 years at this rate.

API

```
POST /v1/links    body {"url": "...", "alias": "optional", "ttl":
  seconds}
                  returns 201 {"code": "aZ3kQ9x", "short_url":
                    "..."}
GET  /{code}      returns 301 or 302 with Location: the long URL,
  or 404
```

Creation should accept an idempotency key, so a retried request doesn't create two links.

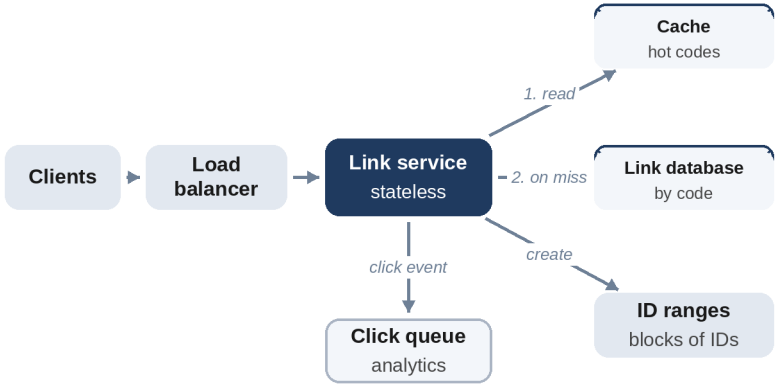
Data Model

One table, keyed by the code:

Field	Notes
code (primary key)	7 characters, or a custom alias
long_url	up to a few KB
created_at, expires_at	expires_at may be empty
owner_id	optional

Lookups are always by code, so a key-value store or a relational table with the code as primary key both work. With a relational database, partition by code if it outgrows one machine.

High-Level Design



Link shortener: a load balancer in front of stateless link services; redirects check a cache, then the database; creation gets a code from an ID generator and writes the database; clicks go to a queue for analytics.

Redirect path: the request reaches a load balancer, then a stateless link service, which looks up the code in a cache and, on a miss, in the database, and responds with a redirect. **Creation path:** the service validates the URL, obtains a new code, stores the row, and returns the short URL. **Analytics:**

each redirect publishes a small click event to a queue; a separate consumer aggregates counts, so redirects never wait for statistics.

Deep Dive 1: Generating Codes

Three options:

1. **Hash the URL** (for example, the first 7 base-62 characters of a SHA-256 hash). The same URL always gets the same code, but different URLs can collide, so every insert must check for a collision and retry with a salt.
2. **Random codes:** generate 7 random characters and insert if unused. Simple and unguessable; collisions are rare while the space is mostly empty, but each insert needs a uniqueness check.
3. **A counter encoded in base 62:** guaranteed unique, no collision checks, but sequential codes are guessable, and a single counter needs care at scale.

The model below uses a counter, starting at 62^6 so every code has exactly 7 characters, and handles custom aliases, which share the namespace and so can collide with future counter codes:

```
class Shortener:
    def __init__(self, clock):
        self.clock = clock
        self.next_id = START
        self.links = {}  # code -> (long_url, expires_at or None)

    def create(self, url, alias=None, ttl=None):
        if alias is not None:
            if alias in self.links:
                raise ValueError(f"alias {alias!r} is taken")
            code = alias
        else:
            code = base62(self.next_id)
            self.next_id += 1
            while code in self.links:  # a custom alias already took it
                code = base62(self.next_id)
                self.next_id += 1
        expires = self.clock() + ttl if ttl else None
        self.links[code] = (url, expires)
        return code

    def resolve(self, code):
        """The long URL for a redirect, or None (404) if unknown/expired."""
        item = self.links.get(code)
        if item is None:
            return None
        url, expires = item
```

```
        if expires is not None and self.clock() >= expires:
            return None
        return url
```

```
1000000 -> https://example.com/articles/2026/10/system-design
sale -> https://example.com/sale
sale after an hour -> None
error: alias 'sale' is taken
```

The test creates 5,000 links and checks that all codes are unique and 7 characters long, then deliberately claims the next counter code as a custom alias and checks that the generator skips it.

To make a counter scale, don't use one global counter. Give each link service a **range of IDs** (a block of 1,000 or a million) from a small coordination service, as in Chapter 19; each server then counts within its own range with no coordination. If guessable codes are a concern, scramble the counter value with a reversible permutation before encoding it.

Deep Dive 2: Fast, Available Redirects

- **Cache:** a small fraction of links receives most clicks (Chapter 6), so a distributed cache of hot codes serves most redirects from memory. Use cache-aside with a TTL.
- **Replicas:** database read replicas serve cache misses; the database's leader only handles creation.
- **Edge caching:** a CDN can cache redirect responses for very popular links.
- **Negative caching:** cache "not found" briefly too, so scans for random codes don't hammer the database.

301 or 302? A 301 (permanent) redirect is cached by browsers, which reduces load but means later clicks never reach you, so they can't be counted and the link can't be changed or expired reliably. A 302 (temporary) redirect sends every click through the service. If analytics or expiry matter, use 302; ask the interviewer.

Deep Dive 3: Abuse

Shorteners are attractive to spammers because short links hide the destination. Defenses: rate limit creation per user and IP (Chapter 11), check destinations against lists of known malicious sites, allow users to report links, and show an interstitial warning for suspicious ones.

Wrap-Up

- **Bottlenecks:** cache capacity for hot links; the ID range service, mitigated by large ranges; database size after several years, handled by partitioning by code.
- **Failures:** if the cache fails, replicas absorb misses, so size them for it; if creation is down, existing links still redirect.
- **Expired links:** delete them lazily on read and with a periodic cleanup job.

Follow-Up Questions

- "Make codes unpredictable." Random codes with a uniqueness check, or a scrambled counter.
- "Show per-link click counts in near real time." Aggregate click events by code in a stream processor; store counts per minute.
- "The same long URL submitted twice: same code?" Only if required; then look up existing URLs through an index on a hash of the long URL.

Key Takeaways

- A link shortener is read-heavy by about 100 to 1, so caching dominates the design.
- Generate codes from per-server ID ranges encoded in base 62; handle custom aliases in the same namespace.
- Choose 301 or 302 based on whether you need analytics and expiry.
- Keep analytics off the redirect path with a queue.

CHAPTER 17

Distributed Rate Limiter

The prompt: "Design a rate limiter for an API used by millions of clients and served by many gateway servers."

Chapter 11 covered the algorithms on one machine. This chapter is about the hard part of the full design: enforcing one limit consistently across many servers, quickly enough that every request can be checked, and safely when parts of the system fail.

Requirements

Functional:

1. Limit requests per client (API key) per endpoint, with configurable rules such as "100 requests per minute, bursts up to 20".
2. Reject excess requests with HTTP 429 and a `Retry-After` header.
3. Change rules without redeploying.

Non-functional:

- Add little latency: about 1 to 2 ms per request.
- Be accurate enough that clients can't exceed limits by much, even across many gateways.
- Never become the cause of an outage.

Estimates

```
checks per second (peak)        500K
store ops per check             1 (an atomic script)
bucket state per client         16 B (tokens + timestamp)
state for all clients           160 MB
store shards (100K ops/s each)  5
added latency budget per check  1-2 ms (same data center)
```

Half a million checks per second is a lot of operations, but the state is tiny: 16 bytes per client, 160 MB for ten million clients. It fits in memory comfortably; the challenge is the operation rate, which a handful of in-memory store shards can handle.

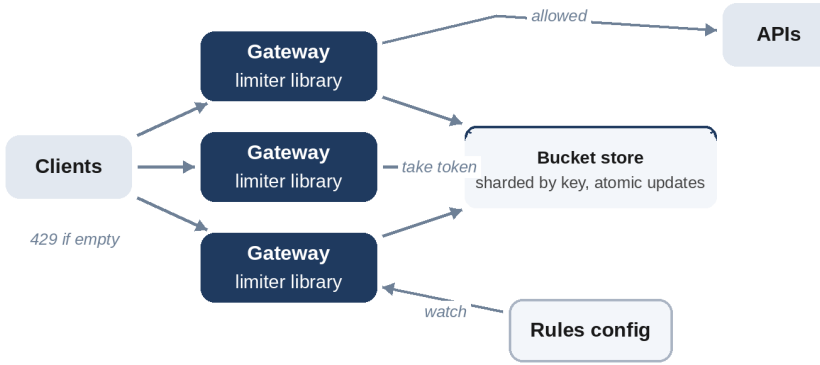
API and Rules

The limiter is a library or sidecar inside each gateway, not a public API. Rules live in a configuration store:

```
rule: endpoint="/v1/search", key="api_key", limit=100,
  window=60s, burst=20
```

Gateways load the rules at startup and watch for changes.

High-Level Design



Distributed rate limiter: each gateway checks a shared in-memory store with one atomic operation per request; rules come from a configuration store; rejected requests get 429.

For each request, the gateway identifies the client and the rule, performs one atomic "take a token" operation on the client's bucket in a shared in-memory store, and either forwards the request or returns 429.

Deep Dive 1: Why Updates Must Be Atomic

The natural implementation reads the bucket, computes the new token count, and writes it back. With many gateways, two can read the same value at the same time, both decide there's a token left, and both write: one update is lost, and an extra request gets through. The model below sends 400 requests through 4 gateways at a bucket of 100 tokens, interleaving the gateways' reads and writes randomly:

```
def run(atomic, gateways=4, requests=400, capacity=100, seed=1):
    """All requests arrive within one second (no refill). Gateways take
    turns in a random interleaving. Returns how many were allowed."""
    rng = random.Random(seed)
    store = Store()
    store.set("bucket", capacity)
    pending = [requests // gateways] * gateways
    reads = [None] * gateways  # value and version each gateway has read
    allowed = 0
    while any(pending) or any(reads):
        g = rng.choice(
            [i for i in range(gateways) if pending[i] or reads[i]]
        )
        if reads[g] is None:  # step 1: read
            reads[g] = store.get("bucket")
            pending[g] -= 1
            continue
        tokens, version = reads[g]  # step 2: decide and write
        reads[g] = None
        if tokens < 1:
            continue  # rejected
        if atomic:
            if store.cas("bucket", tokens - 1, version):
                allowed += 1
            else:
                pending[g] += 1  # lost the race: re-read and retry
        else:
            store.set("bucket", tokens - 1)
            allowed += 1
    return allowed
```

```
limit 100 per second, 400 requests through 4 gateways:
  read-then-write allows 264
  compare-and-set allows 100
```

With read-then-write, 264 requests are allowed against a limit of 100. With **compare-and-set**, a write succeeds only if the bucket hasn't changed since it was read, and otherwise the gateway re-reads and retries, so exactly 100 are allowed. The test checks this for 20 different interleavings.

In practice, the cleanest solution is to run the whole token-bucket update **inside the store** as one atomic operation. Redis, for example, executes a Lua script atomically: the script reads the bucket, refills it based on elapsed time, takes a token if available, and writes it back, all in one round trip. One network call per request, no races.

Deep Dive 2: Latency and Scale

- **Partition buckets** across store shards by client key with consistent hashing (Chapter 9); each check touches exactly one shard.
- **Co-locate** the store with the gateways in each data center; a round trip of well under a millisecond keeps the budget.
- **Local pre-checks:** a gateway can keep a small local token allowance per hot client and only synchronize every few hundred milliseconds. This trades a little accuracy for far fewer store operations, a good choice for very high-volume clients.
- **Multiple regions:** synchronizing buckets across regions on every request would add tens of milliseconds. Instead, give each region its own share of the limit, or synchronize counts asynchronously and accept brief overshoot.

Deep Dive 3: Failure Behavior

If the store is unreachable, the gateway must decide: **fail open** (allow requests) or **fail closed** (reject them). For most APIs, failing open, perhaps with a conservative local limit, is right: a limiter outage shouldn't become an API outage. For limits that protect against fraud or brute-force attacks, such as login attempts, failing closed may be safer. Either way, put a short timeout on the store call (Chapter 14) and alert when the limiter is degraded.

Wrap-Up

- **Bottleneck:** store operations for very hot clients; mitigated by local allowances.
- **Accuracy:** exact within a region with atomic scripts; approximate across regions by design.
- **Observability:** metrics for allowed and rejected requests per rule, store latency, and how often the limiter fails open.

Follow-Up Questions

- "Different limits for free and paid tiers." Rules keyed by plan; the gateway looks up the client's plan with the API key.
- "Limit by cost, not count." Take more tokens for expensive requests.
- "Limit concurrent requests rather than rate." Use a counter incremented on start and decremented on completion, with a lease to recover from crashed requests.

Key Takeaways

- The central problem is sharing one limit across many servers.
- Read-then-write loses updates under concurrency; use compare-and-set or an atomic script in the store.
- Keep the check to one fast operation: shard the state, co-locate it, and use local allowances for hot clients.
- Decide deliberately whether to fail open or closed.

CHAPTER 18

Key-Value Store

The prompt: "Design a distributed key-value store: put(key, value) and get(key), for a large amount of data, highly available."

This design brings together most of Part II: partitioning, replication, quorums, conflict handling and storage engines. It's often asked of senior candidates because it's all deep dives.

Requirements

Functional: `put(key, value)`, `get(key)`, `delete(key)`. Values up to about 1 MB.

Non-functional:

- Store about 100 TB of data, growing.
- About a million operations per second at peak.
- Highly available: reads and writes succeed even when some machines or a whole rack fail.
- Tunable consistency: some clients want the latest value, others prefer lower latency.

Estimates

Assume each node has 4 TB of usable disk and handles about 20,000 operations per second, with three copies of every item.

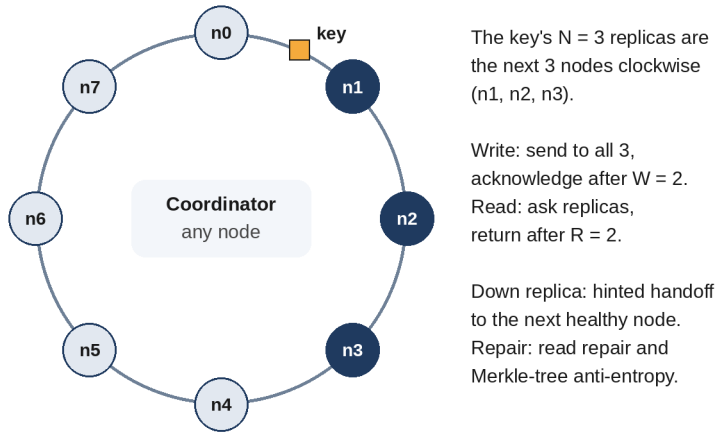
```
data with 3 copies             300 TB
nodes for storage (4 TB each)  75
nodes for 1M ops/s (20K each)  50
cluster size (the larger)      75
```

Storage, not load, decides the cluster size here: about 75 nodes. That tells us partitioning is unavoidable, and that adding nodes must be routine and cheap.

API

```
put(key, value, consistency=QUORUM) -> ok
get(key, consistency=QUORUM) -> value (or several conflicting
  versions)
delete(key) -> ok
```

High-Level Design



Key-value store: keys are placed on a consistent-hashing ring; each key is stored on N consecutive nodes; any node can coordinate a request, writing to W replicas and reading from R.

- **Partitioning:** a consistent-hashing ring with virtual nodes (Chapter 9). Each key is stored on the N = 3 distinct nodes that follow its position clockwise.

- **Replication:** leaderless, with quorums (Chapter 8). Any node can act as **coordinator** for a request: it forwards a write to all N replicas and acknowledges after W = 2 succeed; for a read, it asks the replicas and returns after R = 2 answer. R + W > N, so reads overlap the latest acknowledged write.
- **Membership:** nodes learn about each other and detect failures through a gossip protocol, each periodically exchanging its view of the cluster with a few random peers.
- **Storage engine:** an LSM-tree on each node (Chapter 7), because the workload includes many writes; Bloom filters avoid unnecessary disk reads.

Deep Dive 1: Writes When Nodes Are Down

If one of a key's three replicas is down, a strict quorum can still succeed with the other two. If two are down, the write would fail. A **sloppy quorum** accepts the write on the next healthy node along the ring, with a note, a **hint**, saying which node it really belongs to. When that node returns, the hint is delivered: **hinted handoff**. This keeps writes available during failures, at the cost that a read during the outage might not find the latest write on the "real" replicas.

Deep Dive 2: Conflicting Versions

With leaderless writes, two clients can update the same key concurrently through different coordinators. Options:

- **Last write wins** by timestamp: simple, but silently drops one write, and depends on clocks.
- **Vector clocks** (Chapter 12): each version carries a vector clock, so the store can tell whether one version supersedes another or they are concurrent. Concurrent versions are returned together to the client, which merges them (for example, the union of two shopping carts) and writes back the result.

Offer both: last write wins for clients that don't care, version vectors for those that do.

Deep Dive 3: Repairing Replicas

Replicas drift apart: a hint is lost, a node is down for days. Two repair mechanisms:

- **Read repair:** when a read sees that one replica returned an older version, the coordinator writes the newer one back to it.
- **Anti-entropy:** a background process compares replicas and copies missing data. Comparing every key would mean sending the whole dataset over the network. Instead, each replica builds a **Merkle tree**: keys are grouped into buckets, each bucket is hashed, and each parent hashes its children. Two replicas compare root hashes; only where hashes differ do they descend further.

```
def build(data, buckets=16):
    """Return the tree as a list of levels, leaves first."""
    groups = [[] for _ in range(buckets)]
    for k, v in data.items():  # one pass: put each key in its bucket
        groups[bucket_of(k, buckets)].append((k, v))
    leaves = [digest(repr(sorted(g))) for g in groups]
    levels = [leaves]
    while len(levels[-1]) > 1:
        prev = levels[-1]
        levels.append(
            [digest(prev[i] + prev[i + 1]) for i in range(0, len(prev), 2)]
        )
    return levels


def differing_buckets(a, b, buckets=16):
    """Walk both trees from the root; return differing leaf buckets and
    how many hashes were compared."""
    ta, tb = build(a, buckets), build(b, buckets)
    compared, frontier = 0, [0]
    for depth in range(len(ta) - 1, -1, -1):
        level_a, level_b = ta[depth], tb[depth]
        nxt = []
        for i in frontier:
            compared += 1
            if level_a[i] != level_b[i]:
                nxt.extend([2 * i, 2 * i + 1] if depth else [i])
        frontier = nxt
    return sorted(frontier), compared
```

With 100,000 keys and a single changed value:

```
100,000 keys, 1,024 buckets: 21 hashes compared;
1 bucket differs, holding 94 keys to compare
```

Twenty-one hash comparisons narrow the difference to one bucket of 94 keys, instead of comparing 100,000. The test checks, on thousands of random pairs of replicas, that the keys found through the tree are exactly the keys that differ.

Wrap-Up

- **Adding nodes:** a new node takes over ranges from its neighbors on the ring and streams that data; throttle the transfer so the cluster stays responsive.
- **Hot keys:** cache them at the coordinator, or replicate them more widely.
- **Failure domains:** place a key's replicas on different racks or availability zones, not just different nodes.
- **Consistency per request:** let clients choose R and W, for example R = 1 for low-latency reads that tolerate staleness.

Follow-Up Questions

- "Support range queries." Hash partitioning scatters ranges across nodes; use range partitioning for those tables, and handle hot ranges by splitting.
- "Make it strongly consistent." Use a leader per partition chosen by consensus (Raft), and route all writes and strong reads through it.
- "Delete semantics." Tombstones replicated like writes, removed by compaction after all replicas have seen them; otherwise a deleted value can come back from a stale replica.

Key Takeaways

- Consistent hashing places each key on N replicas; quorums with R + W > N give overlapping reads and writes.
- Sloppy quorums and hinted handoff keep writes available during failures.
- Detect concurrent versions with vector clocks, or accept last write wins deliberately.

- Repair replicas with read repair and Merkle-tree anti-entropy, which finds differences with very few comparisons.

CHAPTER 19

Unique ID Generator

The prompt: "Design a service that generates unique IDs for all the systems in a large company."

It sounds trivial until you add the requirements: millions of IDs per second, no duplicates ever, IDs that sort roughly by time, and no single point of failure.

Requirements

Functional:

1. Generate unique 64-bit integer IDs.
2. IDs generated later should generally be larger (sortable by creation time), so they make good database keys.

Non-functional:

- About 2 million IDs per second at peak across the company.
- Very low latency: ID generation is on the path of almost every write.
- Highly available: no single point of failure.

Estimates

```
IDs per second needed (peak)    2M
max per Snowflake worker        4.1M per second
worker IDs available (10 bits)  1,024
years of 41-bit milliseconds    69.7
64-bit IDs per day at peak      173B
```

One Snowflake-style generator can produce over 4 million IDs per second in theory, so throughput is not the constraint; availability and latency are. That suggests generating IDs as close to the caller as possible.

Options

Chapter 12 compared the main schemes. For this prompt, three are worth presenting:

1. **A database counter** with auto-increment: simple and sortable, but every ID is a database write, and the database is a single point of failure. Even with two databases, one handing out odd and one even numbers, it scales poorly.
2. **Range allocation:** a central service hands out blocks of IDs; servers generate IDs locally within their block.
3. **Snowflake-style IDs:** each generator combines a timestamp, its machine ID and a sequence number, with no coordination per ID.

Design A: Range Allocation

The allocator stores one number, the start of the next free block, in a strongly consistent store. A server asks for a block (say 1,000 IDs), and hands out IDs from it without talking to anyone until it runs out.

```
class Allocator:
    def __init__(self, block=1000):
        self.next_start, self.block = 1, block

    def lease(self):
        start = self.next_start
        self.next_start += self.block  # persisted before replying
        return start, start + self.block


class IdServer:
```

```
    def __init__(self, allocator):
        self.allocator, self.cur, self.end = allocator, 0, 0

    def next_id(self):
        if self.cur >= self.end:
            self.cur, self.end = self.allocator.lease()
        self.cur += 1
        return self.cur - 1
```

With blocks of 5 for illustration: server 0 takes IDs 1 to 5, server 1 takes 6 to 10, then server 0 crashes and restarts. Its unused IDs, 3 to 5, are lost, and its next block starts at 11:

```
>>> run([0, 0, 1, ('crash', 0), 0, 1])
[1, 2, 6, 11, 7]
```

That's the defining trade-off: **gaps are allowed, duplicates are not**. The allocator must persist the new block boundary *before* returning a block, so a crash of the allocator can never hand out the same block twice. The test runs 5,000 requests across three servers with random crashes and checks that no ID repeats.

IDs from range allocation are only roughly time-ordered: a server with a large block may hand out "old" low numbers long after others have moved on.

Design B: Snowflake-Style Generators

Each generator holds a 10-bit machine ID and produces IDs from its clock (Chapter 12). Generators can run inside each application server as a library, so getting an ID is a local function call with no network at all.

The hard part is **assigning machine IDs** uniquely, especially with containers that start and stop all the time. Options: lease machine IDs from a coordination service (such as etcd or ZooKeeper) with a time-to-live that the generator renews; or derive them from a unique deployment slot. Two generators must never share a machine ID, or they will produce duplicates.

The second hard part is **clocks**. If a server's clock jumps backwards, the generator must stop and wait, or fail, rather than reuse timestamps. Keep clocks synchronized, and monitor clock drift.

Choosing

	Range allocation	Snowflake-style
Coordination	One call per block	Once, to get a machine ID
Time ordering	Rough	By millisecond
Clock dependence	None	Yes; backwards clocks must be handled
Information leaked	IDs reveal rough volume	IDs reveal creation time and machine
Failure impact	Lost blocks leave gaps	None, if machine IDs are unique

For a company-wide service, Snowflake-style generators embedded as a library, with machine IDs leased from a coordination service, are the stronger answer: no network hop per ID and time-ordered results. Range allocation is a fine choice for a single product, such as the link shortener of Chapter 16.

Wrap-Up

- **Single points of failure:** none on the hot path; the coordination service is needed only at startup.
- **Ordering across machines:** only approximate (within clock skew), which is enough for database locality but not for strict event ordering.
- **Capacity:** 69 years from the chosen epoch; choose a recent epoch.

Follow-Up Questions

- "We need 128-bit IDs or standard formats." UUIDv7 (RFC 9562) gives time-ordered 128-bit IDs with random bits instead of machine IDs, so no machine-ID assignment is needed.
- "Make IDs non-guessable." Use random UUIDs for public identifiers and keep time-ordered IDs internal.
- "More than 1,024 generators." Change the bit split: fewer sequence bits, more machine bits.

Key Takeaways

- A central counter is simple but a bottleneck and single point of failure.
- Range allocation needs one call per block; it allows gaps but never duplicates, provided the allocator persists before replying.
- Snowflake-style generators need no coordination per ID; the hard parts are unique machine IDs and backwards clocks.
- Generate IDs as close to the caller as possible.

CHAPTER 20

Web Crawler

The prompt: "Design a web crawler that downloads a billion pages a month, for example to build a search index."

A crawler is a long-running pipeline: pick a URL, fetch the page, store it, extract links, add new links to the queue, and repeat, forever. The interesting problems are not the fetching itself but deciding what to fetch next, avoiding duplicates, and being polite to the websites being crawled.

Requirements

Functional:

1. Start from seed URLs and discover new pages by following links.
2. Download and store HTML pages for later processing (indexing is out of scope).
3. Revisit pages periodically to pick up changes.

Non-functional:

- About 1 billion pages per month.
- **Politeness:** never overload a website; respect its `robots.txt` rules.
- Robust to bad pages: huge files, slow servers, infinite link traps, malformed HTML.
- Avoid storing duplicate content.

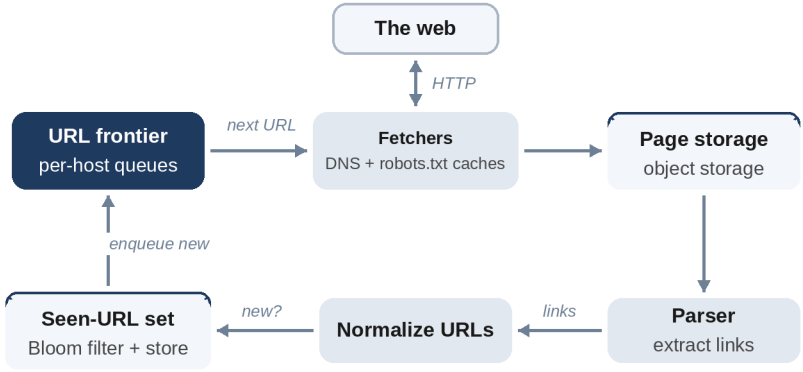
Estimates

Assume 500 KB downloaded per page, 100 KB stored after compression, and about 10 links per page.

```
pages per second                                386
download bandwidth (500 KB/page)                193 MB/s
stored HTML per month (100 KB compressed)       100 TB
links found per month (10 per page)             10B
seen-URL set, 8-byte fingerprints               80 GB
politeness: 1 fetch/s/host, hosts needed busy   386
```

- **About 400 pages per second** and **200 MB per second** of downloads: a fleet of fetchers, but not an enormous one.
- **About 100 TB of compressed HTML per month:** object storage.
- **10 billion links found per month:** the set of URLs already seen grows to tens of billions over time. Even as 8-byte fingerprints, that's tens of gigabytes per month, so the "seen" check needs a partitioned store, or a Bloom filter in front of it (Chapter 13).

High-Level Design



Web crawler: a URL frontier feeds fetchers through DNS resolution; fetched pages go to storage and to a parser; extracted links are normalized, checked against the seen-URL set and added to the frontier.

1. The **URL frontier** decides which URL to fetch next.
2. **Fetchers** resolve DNS (with a cache), check `robots.txt` (also cached), and download the page with a timeout and a size limit.

3. Pages go to **content storage**; a **content fingerprint** (a hash of the page) detects exact duplicates of pages already stored under other URLs.
4. A **parser** extracts links.
5. Links are **normalized**, checked against the **seen-URL set**, and new ones are added to the frontier.

Deep Dive 1: Normalizing and Deduplicating URLs

The same page has many spellings: `HTTPS://Example.com:443/a#top` and `https://example.com/a` are the same resource. Normalizing before the "seen" check avoids fetching it twice:

```
def normalize(url):
    parts = urlsplit(url.strip())
    scheme = parts.scheme.lower()
    host = (parts.hostname or "").lower()
    port = parts.port
    default = {"http": 80, "https": 443}.get(scheme)
    netloc = host if port in (None, default) else f"{host}:{port}"
    path = parts.path or "/"
    return urlunsplit((scheme, netloc, path, parts.query, ""))  # drop #frag
```

```
>>> norm_all(
...     ['HTTPS://Example.com:443/a?x=1#top',
...      'http://example.com',
...      'http://EXAMPLE.com:8080/b'],
... )
['https://example.com/a?x=1',
 'http://example.com/',
 'http://example.com:8080/b']
```

The function lower-cases the scheme and host, removes default ports and fragments, and makes empty paths `/`. Real crawlers go further, removing tracking parameters and following canonical-URL hints in pages.

At tens of billions of URLs, the seen set itself is large. A Bloom filter answers "definitely new" for most URLs cheaply; only "maybe seen" answers check the exact, partitioned store. A false positive means skipping a genuinely new URL, which is acceptable for a crawler.

Deep Dive 2: Politeness and Priority

A naive queue would send hundreds of requests per second to one big site, which is rude and gets the crawler blocked. The frontier therefore keeps **one queue per host**, and a schedule of when each host may next be contacted:

```
class Frontier:
    def __init__(self, delay):
        self.delay = delay
        self.queues = {}  # host -> deque of URLs
        self.ready = []  # heap of (time host may be fetched again, host)
        self.seen = set()

    def add(self, url):
        url = normalize(url)
        if url in self.seen:
            return False
        self.seen.add(url)
        host = urlsplit(url).hostname
        if host not in self.queues:
            self.queues[host] = deque()
            heapq.heappush(self.ready, (0, host))
        self.queues[host].append(url)
        return True

    def next(self, now):
        """The next URL that may be fetched at time `now`, or None."""
        while self.ready and self.ready[0][0] <= now:
            _, host = heapq.heappop(self.ready)
            q = self.queues[host]
            if q:
                heapq.heappush(self.ready, (now + self.delay, host))
                return q.popleft()
            del self.queues[host]  # empty: forget the host for now
        return None
```

`next(now)` takes the host whose waiting time has passed soonest, fetches one URL from its queue, and schedules that host again `delay` seconds later. The test fills the frontier with 500 URLs across 5 hosts and checks two properties over the whole crawl: no host is ever contacted twice within the delay, and every URL is eventually fetched.

Real frontiers add **priority**: important or frequently changing pages (news home pages) go into higher-priority queues, and are re-crawled more often. A common structure is a set of front queues by priority feeding back queues by host.

Deep Dive 3: Robustness

- **Timeouts and size limits** on every fetch.
- **Crawler traps:** sites that generate infinite URLs, such as calendars with a "next month" link forever. Limit URL length, crawl depth per site, and pages per host.
- `robots.txt`**:** fetch and cache it per host; respect disallowed paths and crawl-delay hints.
- **Checkpointing:** the frontier must survive restarts, so persist it (for example as per-host queues in a durable store or log).

Wrap-Up

- **Scaling out:** partition the frontier and the seen set by host, so each crawler machine owns a set of hosts. That keeps politeness local to one machine.
- **Freshness:** revisit pages at a rate based on how often they've changed before.
- **Bottlenecks:** DNS lookups (cache aggressively), the seen-set lookups (Bloom filter plus partitioned store), and slow hosts tying up fetchers (asynchronous I/O and tight timeouts).

Follow-Up Questions

- "Detect near-duplicate pages" (same article, different ads). Use similarity fingerprints such as SimHash rather than exact hashes.
- "Crawl JavaScript-heavy sites." Render pages in headless browsers, which is far more expensive, so reserve it for sites that need it.
- "Prioritize new content." Feed sitemaps and RSS feeds into a high-priority queue.

Key Takeaways

- A crawler is a loop: frontier, fetch, store, parse, normalize, deduplicate, enqueue.

- Normalize URLs before checking whether they've been seen; use a Bloom filter to make the check cheap.
- Politeness needs per-host queues and a schedule of when each host may be contacted next.
- Partition the work by host, so politeness and deduplication stay local.

CHAPTER 21

Notification Service

The prompt: "Design a service that sends push notifications, emails and text messages to users on behalf of other teams' products."

Notification systems look simple, call a provider's API, but users judge them harshly: a missed security alert is dangerous, and ten duplicate "someone liked your photo" messages are annoying enough to make people uninstall the app. The design is about reliability on one side and restraint on the other.

Requirements

Functional:

1. Other services request notifications: recipient, type, content, channel(s), priority.
2. Deliver through push (mobile and web), email and SMS, via third-party providers.
3. Respect user preferences: channels turned off, quiet hours.
4. Avoid duplicates and limit how many non-urgent notifications a user gets.
5. Track delivery status.

Non-functional:

- About 2 billion notifications a day, with bursts of ten times normal (breaking news, a marketing campaign).

- Critical notifications (security alerts, one-time passcodes) must arrive within seconds; marketing can wait.
- At-least-once delivery inside the system, without visible duplicates to users.

Estimates

```
notifications per second (avg)      23.1K
peak (x10, e.g. breaking news)      231K
queue storage, 1 day (1 KB each)    2 TB
dedup keys kept 1 hour (50 B each)  4.17 GB
```

About 23,000 notifications per second on average and 230,000 at peak. The volume is high, but each message is small; the real constraints are provider rate limits, provider failures, and bursts. Deduplication state for an hour fits in a few gigabytes of memory.

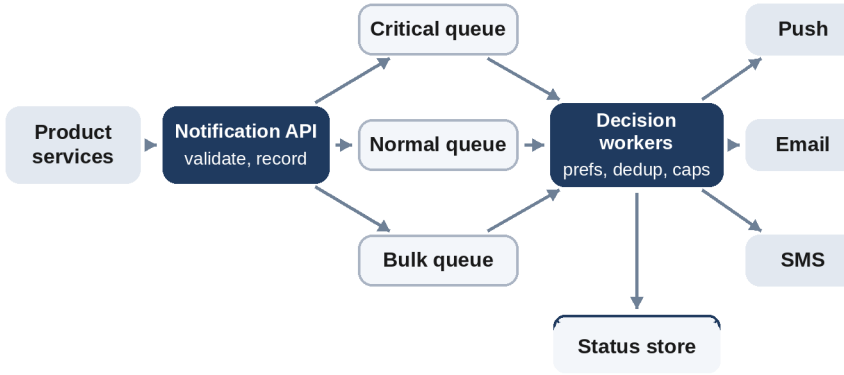
API

```
POST /v1/notifications
  body {"user_id", "type", "channels": ["push", "email"],
    "priority",
        "template_id", "params", "dedupe_key"}
  returns 202 Accepted {"notification_id"}
```

Accept quickly and process asynchronously; the caller doesn't wait for delivery.

High-Level Design

1. The **API** validates the request, records it, and publishes it to a **queue per priority** (critical, normal, bulk), so a marketing blast can't delay a security alert.
2. **Decision workers** load the user's preferences and contact details (cached), apply the rules below, and render the template in the user's language.
3. **Channel workers**, one pool per channel, call the providers: mobile push services, an email provider, an SMS gateway. Each pool has its own rate



Notification service: an API validates requests and writes them to priority queues; workers apply preferences, deduplication and rate caps, render templates and call channel providers; results are recorded in a status store.

limits and circuit breakers (Chapter 14), so a failing SMS provider doesn't affect email.

4. A **status store** records every attempt; providers' delivery callbacks update it.

Deep Dive 1: Deciding Whether to Send

The core logic is a sequence of rules, each of which can stop a notification:

```
def decide(events, prefs, cap=3):
    """events: (time, user, event_key, channel, critical) sorted by time.
    prefs: user -> {"off": set of channels, "quiet": (start_h, end_h)}.
    Returns a list of 'send' or the reason it was skipped."""
    last_sent = {}  # (user, event_key) -> time
    recent = {}  # user -> times of non-critical sends
    out = []
    for t, user, key, channel, critical in events:
        p = prefs.get(user, {})
        hour = (t // HOUR) % 24
        start, end = p.get("quiet", (0, 0))
        quiet = (
            start <= hour < end
            if start <= end
            else hour >= start or hour < end
        )
        times = [x for x in recent.get(user, []) if x > t - HOUR]
        if channel in p.get("off", set()):
            out.append("channel off")
```

```
        elif t - last_sent.get((user, key), -HOUR) < HOUR:
            out.append("duplicate")
        elif quiet and not critical:
            out.append("quiet hours")
        elif not critical and len(times) >= cap:
            out.append("rate cap")
        else:
            out.append("send")
            last_sent[(user, key)] = t
            if not critical:
                times.append(t)
        recent[user] = times
    return out
```

1. Skip channels the user turned off.
2. Drop a **duplicate** of the same event (same user and dedupe key) within an hour. Producers retry; queues redeliver; users must not notice.
3. During the user's **quiet hours**, send only critical notifications.
4. Cap non-critical notifications at a few per user per hour.

```
>>> decide(
...     [(32400, 'ana', 'like:7', 'push', False),
...      (32460, 'ana', 'like:7', 'push', False),
...      (32520, 'ana', 'news', 'email', False),
...      (82800, 'ana', 'promo', 'push', False),
...      (82805, 'ana', 'login-alert', 'push', True)],
...     {'ana': {'off': {'email'}, 'quiet': (22, 7)}},
... )
['send', 'duplicate', 'channel off', 'quiet hours', 'send']
```

The like is sent, its retry a minute later is dropped as a duplicate, email is off, a promotion at 23:00 waits for quiet hours to end (here simply skipped), and the 23:00 login alert goes through because it's critical. The test replays 3,000 random events and checks two properties over everything sent: no event is sent twice within an hour, and no user receives more than three non-critical notifications in any hour.

In production, "skipped for quiet hours" usually means **delayed** until the quiet period ends, and capped notifications may be merged into a digest ("Ana and 12 others liked your photo").

Deep Dive 2: Reliable Delivery

- **Retries** with exponential backoff and jitter for provider errors, using the notification ID as an idempotency key where the provider supports one.

- **Fallback channels:** if push fails because a device token is invalid, fall back to email for important messages, and remove dead tokens.
- **Dead-letter queue** for notifications that keep failing.
- **Provider redundancy:** two email or SMS providers, with routing that shifts traffic when one degrades.

Deep Dive 3: Bursts

A campaign to 100 million users must not starve transactional messages. Separate queues per priority, separate worker pools, and a sending rate for bulk campaigns that stays within provider limits, spreading the campaign over minutes or hours.

Wrap-Up

- **Bottlenecks:** provider rate limits and the preference lookups (cache them).
- **Observability:** delivery rate and latency per channel and provider, failure reasons, opt-out rate after campaigns.
- **Compliance:** honor unsubscribes immediately, include unsubscribe links in marketing email, and follow regional rules for SMS.

Follow-Up Questions

- "Guarantee a one-time passcode arrives in 10 seconds." A dedicated critical queue, the fastest provider, and fallback to a second provider after a short timeout.
- "Let users see a notification inbox in the app." Store notifications per user (a wide-column store partitioned by user ID) and show them with cursor pagination.
- "Avoid notifying users about content they've already seen." Check read state before sending, which requires a fast lookup service.

Key Takeaways

- Accept requests quickly and deliver asynchronously through priority queues.
- Apply preferences, deduplication, quiet hours and rate caps before sending; users notice every duplicate.
- Isolate channels and providers with their own workers, rate limits and circuit breakers.
- Keep critical messages fast even during bulk campaigns.

CHAPTER 22

News Feed

The prompt: "Design the home feed of a social network: users follow accounts and see their recent posts."

The news feed is one of the most frequently asked design questions because it has a famous central trade-off, doing the work when a post is written or when a feed is read, and a famous complication: accounts with millions of followers.

Requirements

Functional:

1. Users publish posts (text, with optional media).
2. Users follow other users.
3. A user's feed shows recent posts from the accounts they follow, newest first (ranking is a follow-up).

Non-functional:

- 300 million daily active users.
- Feed loads must be fast: a couple of hundred milliseconds.
- A new post can take a few seconds to appear in followers' feeds (eventual consistency is fine).

Estimates

Assume half of users post once a day, users load their feed 10 times a day, and accounts have 200 followers on average.

```
posts per second                    1.74K
feed reads per second               34.7K
inbox writes per second (push)      347K
feed cache per user (500 ids x 8 B) 4 KB
feed cache for all users            1.2 TB
```

- Reads (about 35,000 per second) outnumber posts (about 1,700 per second) by 20 to 1.
- But if every post is copied into every follower's feed, the system performs about **350,000 feed writes per second**. The fan-out, not the posts themselves, is the write load.
- A cached feed of 500 post IDs per user costs about 1.2 TB across all users: large, but feasible for a distributed cache.

API

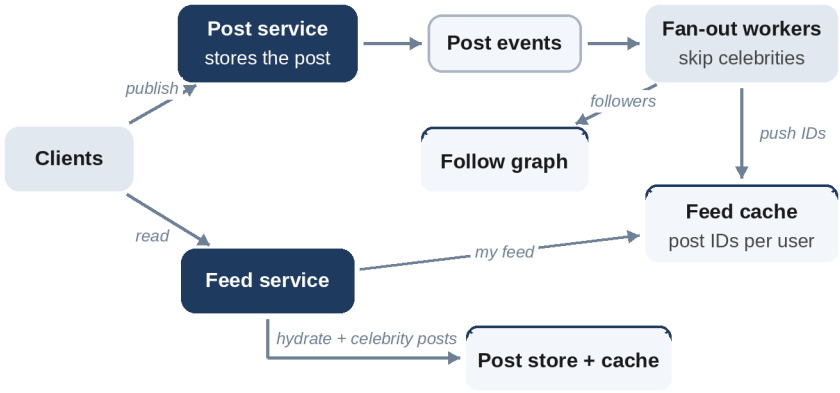
```
POST /v1/posts               body {"text", "media_ids"}  returns
  {"post_id"}
GET  /v1/feed?cursor=...      returns {"posts": [...],
  "next_cursor"}
POST /v1/users/{id}/follow
```

Data Model

- **Posts:** post_id (time-ordered, Chapter 12), author_id, text, media references, created_at. Partitioned by author.
- **Follows:** (follower_id, followee_id), with indexes in both directions.
- **Feed cache:** for each user, a list of recent post IDs.

High-Level Design

Publishing: the post service stores the post and publishes an event. **Fan-out workers** look up the author's followers and push the post ID into each



News feed: a post service stores posts and publishes an event; fan-out workers push post IDs into followers' feed caches; the feed service reads the cache, merges posts from followed celebrities, and hydrates post details.

follower's feed cache. **Reading:** the feed service reads the user's list of post IDs, fetches the posts themselves (from a post cache), and returns them.

Deep Dive 1: Fan-Out on Write or on Read?

- **Fan-out on write (push):** copy each post to every follower's feed when it's published. Feed reads are a single cache lookup, very fast. But writes multiply by the follower count, and work is wasted on followers who never log in.
- **Fan-out on read (pull):** at read time, fetch recent posts from every followed account and merge them. No write amplification, but a user following 500 accounts makes every feed load expensive.

The problem with pure push is the **celebrity**: an account with 50 million followers turns one post into 50 million writes. The standard answer is a **hybrid**: push for ordinary accounts, pull for accounts above a follower threshold. At read time, the user's pushed feed is merged with recent posts from the few celebrities they follow.

```
class FeedService:
    def __init__(self, follows, threshold):
        self.follows = follows  # user -> set of accounts they follow
        self.followers = {}
        for user, accounts in follows.items():
            for a in accounts:
                self.followers.setdefault(a, set()).add(user)
        self.big = {
            a for a, f in self.followers.items() if len(f) > threshold
        }
        self.inbox = {}  # user -> list of (time, author, post)
        self.posts = {}  # author -> list of (time, author, post)
        self.writes = 0  # inbox writes performed

    def publish(self, t, author, post):
        item = (t, author, post)
        self.posts.setdefault(author, []).append(item)
        if author not in self.big:
            for f in self.followers.get(author, ()):
                self.inbox.setdefault(f, []).append(item)
                self.writes += 1

    def feed(self, user, n=10):
        lists = [self.inbox.get(user, [])]
        lists += [
            self.posts.get(a, [])
            for a in self.follows.get(user, ())
            if a in self.big
        ]
        return heapq.nlargest(n, (x for lst in lists for x in lst))
```

The test checks, on thousands of random follow graphs, that the hybrid feed always equals a simple "pull everything" feed. The demo measures what the hybrid saves: 2,000 users each follow 20 random users and one celebrity, and one post in ten comes from the celebrity:

```
push to everyone  218,086 inbox writes for 1,000 posts
hybrid             18,086 inbox writes for 1,000 posts
```

The hybrid does 18,086 inbox writes instead of 218,086: the celebrity's 100 posts would have been 200,000 writes on their own. At real scale, with celebrity accounts of tens of millions of followers, the difference is what makes the system feasible.

Deep Dive 2: Feed Storage and Hydration

Feeds store **post IDs**, not posts: a post appears in thousands of feeds but is stored once, and edits or deletions show up everywhere immediately. Reading a page of the feed means "hydrating" 20 IDs into posts with a batched lookup

from a post cache. Keep only the newest few hundred IDs per feed in the cache; users rarely scroll further, and older pages can be computed by pulling.

Users who haven't logged in for weeks don't need a precomputed feed at all. Skip them during fan-out and build their feed by pulling when they return.

Deep Dive 3: Ranking

Real feeds are ranked, not purely chronological. Ranking fits naturally on top of this design: candidate posts come from the pushed feed and pulled celebrity posts, a ranking service scores them with features such as the relationship to the author and engagement so far, and the top results are returned. Keep ranking within the latency budget by limiting the candidate set to a few hundred.

Wrap-Up

- **Bottlenecks:** fan-out workers during spikes (queue the work, scale workers), the follower lookup for large accounts (pre-split follower lists), the post cache for viral posts (replicate hot entries).
- **Consistency:** a post may take a few seconds to reach all feeds; the author should see their own post immediately (read-your-writes, Chapter 8), so insert it into their own feed synchronously.
- **Deletes:** remove from the post store; feeds containing the dead ID simply skip it at hydration.

Follow-Up Questions

- "Unfollow should remove that account's posts from the feed." Filter at read time against the current follow list, and let old entries age out.
- "Show a 'new posts' indicator." Push a lightweight signal over an open connection, or have the client poll the head of its feed cheaply.
- "Support multiple regions." Replicate posts asynchronously; fan out within each region.

Key Takeaways

- The fan-out, not the posts, is the main write load.
- Push gives fast reads but amplifies writes; pull is the reverse; a hybrid pushes for most accounts and pulls for celebrities.
- Store post IDs in feeds and hydrate at read time.
- Skip inactive users during fan-out, and let authors see their own posts immediately.

CHAPTER 23

Chat and Messaging

The prompt: "Design a chat app supporting one-to-one and small group conversations, with online status and message history."

Chat combines real-time delivery over long-lived connections with the expectation that no message is ever lost, duplicated or shown out of order, even on flaky mobile networks. That combination drives every decision.

Requirements

Functional:

1. Send and receive messages in one-to-one and group conversations (groups up to a few hundred members).
2. Messages appear in real time when the recipient is online, and are delivered when they come back online.
3. Message history on every device.
4. Delivery and read receipts; online presence.

Non-functional:

- 500 million daily active users.
- Low latency: messages appear within a second or so.
- No lost messages; consistent order within a conversation; no visible duplicates.

Estimates

Assume 40 messages per user per day, 60% of daily users online at peak, and about 200 bytes per stored message.

```
messages per second (avg)          231K
open connections (60% online)      300M
gateway servers (100K conns each)  3,000
storage per year (200 B/msg)       1.46 PB
```

- About 230,000 messages per second on average.
- **300 million open connections** at peak: at 100,000 connections per server, about 3,000 gateway servers. Connection management is a central concern.
- About 1.5 PB of messages per year: a partitioned, write-optimized store.

API

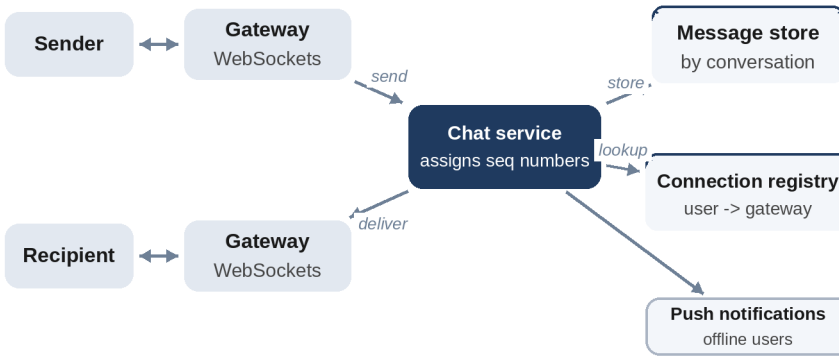
Clients keep a WebSocket open to a gateway (Chapter 4) and exchange small messages:

```
client -> server: send {conversation_id, client_msg_id, text}
server -> client: ack  {client_msg_id, seq}
server -> client: message {conversation_id, seq, sender, text}
client -> server: sync {conversation_id, after_seq}
```

Data Model

- **Messages:** partitioned by conversation_id and sorted by sequence number within it, in a wide-column store: perfect for "the latest messages in this conversation" and "messages after seq 1,042".
- **Conversations and members:** a relational or key-value store.
- **Connection registry:** which gateway each online user is connected to, in a fast in-memory store with short TTLs.

High-Level Design



Chat: clients hold WebSocket connections to gateways; a chat service assigns sequence numbers, stores messages, and routes them to the recipients' gateways using a connection registry; offline recipients get push notifications.

To send a message: the sender's gateway forwards it to the **chat service**, which assigns the next sequence number in the conversation, stores the message, acknowledges the sender, and looks up each recipient in the **connection registry**. Online recipients get the message through their gateway; offline ones get a push notification and fetch it when they reconnect.

Deep Dive 1: Ordering, Gaps and Duplicates

Mobile networks drop connections constantly. The design that survives this is simple:

- The server assigns each message a **sequence number** within its conversation, so there is one agreed order.
- Each client remembers the **last sequence number it has**, and after reconnecting asks for everything after it.
- Clients retry sends after a lost acknowledgement using the same **client message ID**, so the server can drop the duplicate.

```
class ChatServer:
    def __init__(self):
        self.log = []  # (seq, sender, client_msg_id, text)
        self.ids = set()

    def send(self, sender, client_msg_id, text):
        if client_msg_id in self.ids:  # a retry of a message we stored
            return
        self.ids.add(client_msg_id)
        self.log.append((len(self.log) + 1, sender, client_msg_id, text))

    def since(self, seq):
        return self.log[seq:]


class Client:
    def __init__(self, name):
        self.name, self.messages, self.last_seq = name, [], 0

    def sync(self, server):
        for m in server.since(self.last_seq):
            self.messages.append(m)
            self.last_seq = m[0]
```

The test simulates three users sending messages while randomly disconnecting and reconnecting, with 30% of sends retried after a lost acknowledgement. Across 30 random runs, every client ends with exactly the server's log: the same order, no gaps, no duplicates.

Sequence numbers per conversation require one place to assign them. Partition conversations across chat-service instances (by conversation ID), so each conversation's counter lives in one place and different conversations scale independently.

Deep Dive 2: Connections at Scale

- **Gateways are stateful**: each holds many WebSockets. Register each connection in the registry with a TTL refreshed by heartbeats, so stale entries disappear when a server dies.
- **Deployments** must drain connections gradually; clients reconnect to another gateway and sync from their last sequence number, so nothing is lost.
- **Multiple devices:** the registry maps a user to several connections; deliver to all, and sync each device by its own last sequence number.

Deep Dive 3: Groups and Presence

- **Groups:** for groups of a few hundred, the chat service fans out each message to the members' gateways. Very large broadcast channels need a different design, closer to the news feed (pull rather than push).
- **Presence:** online status changes constantly. Update it via heartbeats, broadcast changes only to contacts who are currently online, and tolerate delays of a few seconds. Presence is a cache, not a source of truth.
- **Receipts:** delivery and read receipts are small messages in the same conversation stream, so they inherit the same ordering and retry logic.

Wrap-Up

- **End-to-end encryption:** if required, the server stores and routes only ciphertext; keys stay on devices. That moves features such as search to the client.
- **Bottlenecks:** the connection registry (partition it), hot group conversations, and storage growth (tiered storage for old messages).
- **Failure:** if a chat-service instance dies, another takes over its conversations; clients retry, and duplicates are dropped by client message ID.

Follow-Up Questions

- "Search message history." A search index per user, updated asynchronously; with end-to-end encryption, on-device search.
- "Edit and delete messages." New events in the stream that refer to the original sequence number.
- "Typing indicators." Ephemeral messages, not stored, delivered only to online members.

Key Takeaways

- Real-time delivery uses persistent connections, tracked in a registry with TTLs.

- Per-conversation sequence numbers give one order; clients sync from their last sequence number after reconnecting.
- Client message IDs make retries safe; the server drops duplicates.
- Partition conversations so each one's sequence lives in one place.

CHAPTER 24

Search Autocomplete

The prompt: "Design the suggestions that appear under a search box as the user types."

Autocomplete has an unusual profile: a request on every keystroke, a latency budget measured in tens of milliseconds, and data (popular queries) that changes slowly enough to be precomputed. The design separates a fast, read-only serving path from a slower pipeline that builds the data.

Requirements

Functional:

1. As the user types, return the top 5 to 10 completions for the current prefix.
2. Rank by popularity, with recent trends reflected within about an hour.
3. Filter offensive or unsafe suggestions.

Non-functional:

- Results within about 100 ms end to end, so typing feels instant.
- Highly available; if suggestions fail, search itself must still work.

Estimates

Assume 5 billion searches a day and about six keystroke requests per search.

```
suggestion requests per second           347K
peak (x3)                                1.04M
distinct popular phrases kept            100M
trie + top-5 lists (about 200 B/phrase)  20 GB
latency target per keystroke             under 100 ms end to end
```

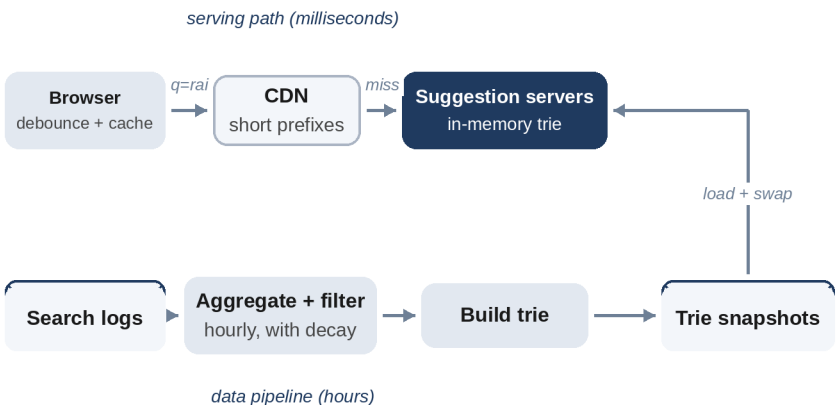
Hundreds of thousands of requests per second, but the data, 100 million popular phrases with their top-completion lists, is about 20 GB: it fits in memory on a single large server, and can be replicated many times to handle the request rate.

API

```
GET /v1/suggest?q=rai&lang=en  returns ["rain", "rainbow",
  "rails", ...]
```

The client waits briefly after each keystroke (debouncing, 50 to 100 ms), cancels outdated requests, and caches results for prefixes it has already seen.

High-Level Design



Autocomplete: the serving path reads an in-memory trie replicated across suggestion servers behind a CDN; a separate pipeline aggregates search logs into counts and builds new tries periodically.

- **Serving path:** suggestion servers keep a trie of popular phrases in memory and answer from it. Responses for common short prefixes are cacheable at the CDN and in the browser.
- **Data pipeline:** search logs are aggregated (for example hourly) into phrase counts, filtered, and used to build a new trie, which is distributed to the servers and swapped in atomically.

Deep Dive 1: A Trie With Top-K at Every Node

A trie stores phrases letter by letter. A naive lookup walks to the prefix's node and then searches the whole subtree for the most popular phrases, too slow for short prefixes like "a" with millions of completions. The fix is to **store the top k completions at every node**, computed when the trie is built. A lookup is then just a walk down the prefix:

```
class Autocomplete:
    def __init__(self, k=3):
        self.root, self.k, self.counts = Node(), k, {}

    def record(self, phrase, times=1):
        count = self.counts.get(phrase, 0) + times
        self.counts[phrase] = count
        node = self.root
        for ch in phrase:
            node = node.children.setdefault(ch, Node())
            entries = [e for e in node.top if e[1] != phrase]
            entries.append((-count, phrase))
            node.top = sorted(entries)[: self.k]

    def suggest(self, prefix):
        node = self.root
        for ch in prefix:
            node = node.children.get(ch)
            if node is None:
                return []
        return [p for _, p in node.top]
```

```
>>> run(
...     ['rain', 'rainbow', 'rain', 'railway', 'rainbow', 'rain', 'rails'],
...     'rai',
... )
['rain', 'rainbow', 'rails']
```

"rain" was searched three times, "rainbow" twice, and "railway" and "rails" once each; ties are broken alphabetically. The test compares the trie with a

brute force that counts and sorts every matching phrase, on thousands of random query streams.

Storing top-k lists multiplies memory by the trie's depth, but for the top 10 phrases per node it's affordable, and lookups become O(length of prefix).

Deep Dive 2: Building and Updating the Data

Updating counts on every search would mean writing to the trie hundreds of thousands of times per second. Instead:

1. Log every completed search.
2. Aggregate counts per phrase over a window (hourly, with older windows decayed so recent trends rise), using a stream or batch job. A Count-Min sketch (Chapter 13) can find heavy hitters cheaply in the stream.
3. Filter: remove offensive phrases, personal information, and phrases searched by too few distinct users (a privacy safeguard).
4. Build a new trie offline and ship it to servers, which load it and switch over without downtime.

For very fresh trends (a breaking news event), merge a small "trending" trie, rebuilt every few minutes, with the main one at query time.

Deep Dive 3: Scaling the Serving Path

- **Replicate** the whole trie on every server if it fits in memory; add servers for throughput.
- If it doesn't fit, **partition by prefix range** (a to f, g to m...), being careful that popular first letters are split finer.
- **Cache** results for the shortest prefixes, which are the most requested and the most expensive.

Wrap-Up

- **Personalization:** blend global suggestions with the user's own recent searches on the client or in a small per-user store.
- **Languages and regions:** one trie per language or region.

- **Failure:** if suggestion servers are unavailable, the client simply shows no suggestions.

Follow-Up Questions

- "Handle typos." Fuzzy matching (edit distance), usually as a second pass when the exact prefix has few results.
- "Rank by more than popularity." Score candidates from the trie with freshness and personalization features before returning them.
- "Remove a harmful suggestion immediately." A blocklist checked at serving time, in addition to filtering in the pipeline.

Key Takeaways

- Separate a read-only, in-memory serving path from an offline pipeline that builds the data.
- Store the top-k completions at every trie node so lookups are a short walk.
- Aggregate counts in batches with decay for trends; never update the serving trie per search.
- Debounce and cache on the client; cache short prefixes everywhere.

CHAPTER 25

Video Streaming

The prompt: "Design a video platform where users upload videos and millions watch them on phones, computers and TVs."

Video platforms are dominated by two numbers: storage and egress bandwidth. Almost every design decision, transcoding into several formats, chunking into segments, pushing them to CDNs, follows from making playback smooth on unreliable networks while keeping those numbers affordable.

Requirements

Functional:

1. Users upload videos.
2. Viewers watch videos on any device, starting quickly and without stalling.
3. Basic metadata: titles, view counts.

Non-functional:

- Hundreds of hours of video uploaded every minute; tens of millions of concurrent viewers.
- Playback starts in about a second and rarely rebuffers, even on poor connections.
- Uploaded videos are never lost.

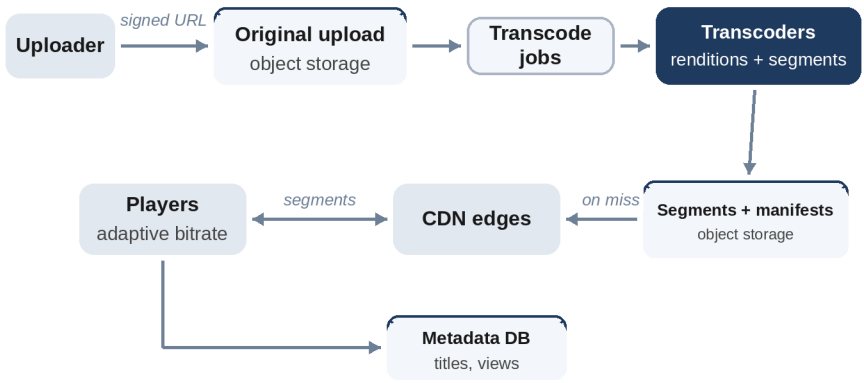
Estimates

Assume 500 hours uploaded per minute, five renditions per video (0.4 to 8 Mbps), 20 million concurrent viewers at peak watching at 3 Mbps on average.

```
hours uploaded per day              720K
storage per day, all renditions     5.48 PB
concurrent viewers (peak)           20M
egress at 3 Mbps average            7.5 TB/s
videos per second (10 min average)  50
```

- About **5 PB of new storage a day** for all renditions. Storage tiers and deleting rarely watched high renditions matter.
- About **7.5 TB per second** of egress at peak. That is far beyond any single data center; it's only possible with CDNs and caches deep inside internet service providers' networks.

High-Level Design



Video platform: uploads go to object storage; a transcoding pipeline splits each video into segments and encodes several renditions; segments and manifests are served through CDNs; metadata lives in a database.

Upload: the client uploads directly to object storage using a pre-signed URL, in resumable chunks, so large uploads survive network drops. A message triggers processing.

Processing: the video is split into short pieces and **transcoded** in parallel into several renditions (resolutions and bitrates) and codecs. Each rendition is cut into **segments** of a few seconds, and a **manifest** lists them (formats such as HLS and DASH). Thumbnails and metadata are generated along the way.

Playback: the player downloads the manifest, then requests segments from the nearest CDN edge, choosing a rendition for each segment.

Deep Dive 1: Adaptive Bitrate Streaming

Network bandwidth changes from second to second. Because every rendition is cut into the same segments, the player can switch quality at each segment boundary. A simple and effective rule is **buffer-based**: if the buffer of downloaded video is low, choose a low bitrate to refill it quickly; when the buffer is healthy, step up.

```
def play(bandwidth, choose):
    """bandwidth: Mbps available for each segment download.
    Returns (seconds spent rebuffering, average bitrate)."""
    buffer, stalled, rates = 0.0, 0.0, []
    for bw in bandwidth:
        rate = choose(buffer)
        rates.append(rate)
        download = rate * SEGMENT / bw  # seconds to fetch this segment
        if download > buffer:
            stalled += download - buffer  # playback froze
            buffer = 0.0
        else:
            buffer -= download
        buffer = min(buffer + SEGMENT, 30.0)  # cap the buffer at 30 s
    return round(stalled, 1), round(sum(rates) / len(rates), 2)


def buffer_based(buffer):
    """Low buffer -> lowest quality; full buffer -> highest."""
    if buffer < 4:
        return RENDITIONS[0]
    step = min(len(RENDITIONS) - 1, int((buffer - 4) / 4) + 1)
    return RENDITIONS[step]
```

The model plays 60 two-second segments over a connection that drops from 12 Mbps to 1.5 Mbps for 40 seconds, like a train going through a tunnel, then recovers:

```
always 8 Mbps  rebuffering 162.0 s, average 8.0 Mbps
buffer-based   rebuffering   0.1 s, average 4.58 Mbps
```

Always streaming at 8 Mbps freezes for 162 seconds; the buffer-based player stalls for a tenth of a second and averages 4.6 Mbps. The test checks over 200 random bandwidth traces that the adaptive player never rebuffers more than the fixed one. Production players combine buffer level with bandwidth estimates, but the principle is the same: quality is worth less to viewers than continuity.

Deep Dive 2: Transcoding at Scale

- **Parallelism:** split each video into chunks and transcode them in parallel across a worker fleet, coordinated through a queue; reassemble the segments.
- **Priority:** popular creators and short videos first; long, rarely watched videos can wait.
- **Cost:** transcoding is expensive. Encode rare formats lazily, on first request, and skip the highest renditions for videos that few people watch.
- **Reliability:** each step is idempotent and retried; the original upload is kept so everything can be re-encoded later with better codecs.

Deep Dive 3: Delivery and Storage

- **CDN caching:** popular videos are served almost entirely from edge caches. The long tail of rarely watched videos is fetched from origin storage on demand.
- **Storage tiers:** recent and popular videos on fast storage; old, rarely watched ones on cheaper, slower tiers, with **erasure coding** instead of full copies (Chapter 26).
- **Startup time:** the first segment at low quality, small segments at the start, and prefetching the next video in a feed.

Wrap-Up

- **Bottlenecks:** egress (CDNs, efficient codecs), transcoding capacity during upload spikes (queue and prioritize), and the long tail of cold content (origin capacity).

- **View counts:** a classic hot counter. Count in a stream and update in batches, accepting slight delays (Chapter 31).
- **Live streaming** is a different design: segments are produced in real time, with a latency target of a few seconds.

Follow-Up Questions

- "Resume watching on another device." Store the playback position per user per video, updated every few seconds.
- "Copyright detection." Compare fingerprints of uploads with a reference database during processing.
- "Reduce bandwidth costs." Newer codecs, per-title encoding that adapts bitrates to content complexity, and deeper CDN caching.

Key Takeaways

- Storage and egress bandwidth dominate; CDNs are essential, not optional.
- Upload directly to object storage; transcode in parallel into several renditions cut into aligned segments.
- Adaptive bitrate players switch quality per segment to avoid stalling; continuity beats quality.
- Treat popular and long-tail content differently for both storage and caching.

CHAPTER 26

File Storage and Sync

The prompt: "Design a cloud file storage service that keeps a user's files in sync across devices."

The interesting parts are not storing files, object storage does that well, but **syncing efficiently** (sending only what changed), **deduplicating** content, and handling **conflicts** when the same file is edited on two offline devices.

Requirements

Functional:

1. Upload, download and delete files and folders.
2. Changes on one device appear on the user's other devices automatically.
3. File version history.
4. Sharing with other users (a follow-up).

Non-functional:

- 200 million users, about 10 GB each on average.
- Never lose a file.
- Sync quickly, and use little bandwidth for small edits to large files.

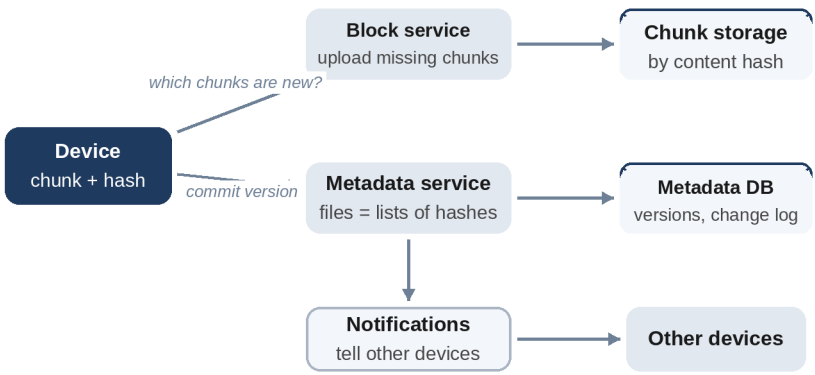
Estimates

Assume deduplication saves 30%, and compare storing three full copies with erasure coding at 1.5 times overhead.

```
logical data                      2,000 PB
after dedup (assume 30% saved)    1,400 PB
stored with 1.5x erasure coding   2,100 PB
vs. 3 full copies                 4,200 PB
metadata rows (1,000 files/user)  200B
```

About 2,000 PB of logical data. The difference between three copies (4,200 PB) and erasure coding (2,100 PB) is around two exabytes of disks: **erasure coding** splits data into fragments plus parity fragments, so it survives the loss of several fragments while storing far less than full copies. It's slower to read and repair, so it suits colder data, while hot data may keep replicas. And about 200 billion metadata rows (one per file) call for a partitioned metadata store.

High-Level Design



File sync: clients split files into chunks and upload only new chunks to block storage; a metadata service records files as lists of chunk hashes; a notification service tells other devices to sync.

- **Client:** watches the local folder, splits changed files into **chunks**, hashes each chunk, and asks which hashes the server lacks.

- **Block storage:** stores chunks, addressed by their hash (content-addressed), in object storage. Identical chunks are stored once: deduplication for free.
- **Metadata service:** records each file version as an ordered list of chunk hashes, plus name, folder, size and owner. Partitioned by user (or by shared namespace).
- **Notification service:** keeps a connection open to each device (long polling or WebSockets) and tells it when its files change; the device then pulls the new metadata and any missing chunks.

Deep Dive 1: Chunking

Uploading only changed chunks makes small edits cheap, but it depends on **where chunk boundaries fall**. With fixed-size chunks, inserting one byte near the start of a file shifts every later boundary, so every chunk looks new. **Content-defined chunking** cuts where a rolling hash of the last few bytes matches a pattern, so boundaries are tied to the content, not to positions, and move along with it:

```
def cdc_chunks(data):
    """Cut where a rolling hash over the last WINDOW bytes has its low
    bits all zero (bounded by MIN and MAX chunk sizes)."""
    base, mod = 257, (1 << 61) - 1
    drop = pow(base, WINDOW, mod)
    chunks, start, h = [], 0, 0
    for i, byte in enumerate(data):
        h = (h * base + byte) % mod
        if i - start >= WINDOW:
            h = (h - data[i - WINDOW] * drop) % mod  # slide the window
        size = i - start + 1
        if (size >= MIN and h & MASK == 0) or size >= MAX:
            chunks.append(data[start : i + 1])
            start, h = i + 1, 0
    if start < len(data):
        chunks.append(data[start:])
    return chunks
```

The model inserts one byte near the start of a 1 MB file and counts how many chunks the server doesn't already have:

```
fixed 2 KiB      upload  489 of 489 chunks after a 1-byte insert
content-defined  upload    1 of 383 chunks after a 1-byte insert
```

Fixed 2 KiB chunks force all 489 chunks to be uploaded again; content-defined chunks need just 1 of 383. The tests check that chunks always reassemble into the original data and stay within the minimum and maximum sizes. Real services use larger chunks (several megabytes) to keep metadata small, but the principle is the same.

Deep Dive 2: Metadata and Consistency

The metadata store is the source of truth for "what is in my folder". Each change is a new **version** of a file, with a version number used for optimistic concurrency (Chapter 30): a client uploads its change saying "this replaces version 7". If the server is already at version 8, the client is out of date and must resolve a conflict. Metadata changes for a user are also appended to a per-user **change log**, so a device can ask "what changed since cursor X?" and catch up after being offline.

Deep Dive 3: Conflicts

Two devices edit the same file while offline; both try to upload. The second upload fails its version check. Automatically merging arbitrary files is impossible, so file sync services keep both: the second version is saved as a **conflicted copy** next to the original, and the user decides. For specific formats (documents edited online), real-time collaboration avoids conflicts entirely (Chapter 34).

Wrap-Up

- **Durability:** chunks in object storage with replication or erasure coding across failure domains; metadata in a replicated database with backups and point-in-time recovery.
- **Bottlenecks:** metadata operations for users with millions of files, and the notification fan-out for widely shared folders.
- **Garbage collection:** chunks referenced by no file version (after deletions and version expiry) are removed by a background job using reference counts.

Follow-Up Questions

- "Sharing folders between users." Shared folders are their own metadata namespace with an access list; notifications go to every member's devices.
- "Large files over slow connections." Resumable, chunked uploads, in parallel.
- "End-to-end encryption." Encrypt chunks on the client; server-side deduplication then only works within one user's data.

Key Takeaways

- Store files as lists of content-addressed chunks: deduplication and incremental sync come naturally.
- Content-defined chunking keeps most chunks unchanged after insertions; fixed-size chunking doesn't.
- The metadata service is the source of truth; version numbers detect conflicting updates.
- Keep both versions on conflict, and let the user resolve it.
- Erasure coding saves enormous amounts of storage for colder data.

CHAPTER 27

Nearby Places

The prompt: "Design a service that finds restaurants, shops or other places near a user, such as 'cafes within 2 km'."

The core problem is a **spatial index**: ordinary database indexes sort values in one dimension, but "near" is two-dimensional. The design turns two-dimensional locations into keys that a normal index can search.

Requirements

Functional:

1. Search for places within a radius of a location, optionally filtered by category.
2. Show place details.
3. Business owners add and update places (changes may take minutes to appear).

Non-functional:

- 200 million places worldwide; searches return in about 100 ms.
- Read-heavy: searches vastly outnumber updates.

Estimates

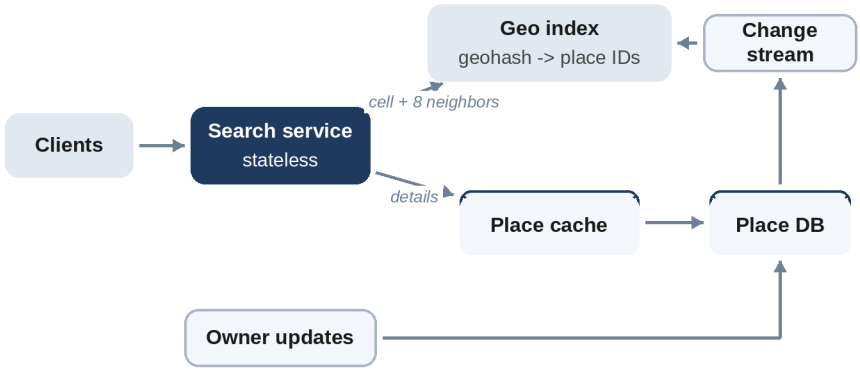
```
places                                  200M
place records (1 KB each)               200 GB
geo index entries (geohash + id, 24 B)  4.8 GB
searches per second                     1.93K
place updates per day (1%)              2M
```

Place records total about 200 GB, but the **geo index**, a location key and a place ID per place, is under 5 GB. It fits in memory on a single server, and can be replicated across many servers for about 2,000 searches per second, and far more at peak.

API

```
GET /v1/places/search?lat=51.50&lon=-0.12&radius_km=2&category=ca
  fe&cursor=...
GET /v1/places/{id}
POST /v1/places   (owners)
```

High-Level Design



Nearby places: a search service queries an in-memory geo index (geohash to place IDs) replicated across servers, then loads place details from a cache backed by the place database; updates flow to the index asynchronously.

Searches go to a stateless **search service**, which queries the **geo index** for candidate place IDs, filters them by exact distance and category, and loads

details from a **place cache** backed by the **place database**. Owner updates write to the database; a change stream updates the index within minutes.

Deep Dive 1: Geohash

A **geohash** (introduced by Gustavo Niemeyer in 2008) encodes a latitude and longitude as a short string. It repeatedly halves the world, alternating between longitude and latitude, and records each choice as a bit; the bits are written in base 32. Each additional character narrows the cell, and nearby points usually share a prefix, so a plain sorted index on geohash strings becomes a spatial index:

```
def encode(lat, lon, precision=6):
    lat_r, lon_r = [-90.0, 90.0], [-180.0, 180.0]
    bits, out, even = [], [], True
    while len(out) < precision:
        rng, val = (lon_r, lon) if even else (lat_r, lat)
        mid = (rng[0] + rng[1]) / 2
        if val >= mid:
            bits.append(1)
            rng[0] = mid
        else:
            bits.append(0)
            rng[1] = mid
        even = not even
        if len(bits) == 5:
            out.append(BASE32[int("".join(map(str, bits)), 2)])
            bits = []
    return "".join(out)
```

The test checks the implementation against a well-known reference value (57.64911, 10.40744 encodes to `u4pruydqqvj`) and checks the size of a five-character cell: about 4.9 km by 4.9 km at the equator.

To search, find the cell containing the user and its **eight neighbors**, because a nearby place can sit just across a cell boundary, and then filter candidates by exact distance:

```
def precision_for(lat, radius_km):
    """The finest precision whose cells are at least radius_km tall and
    wide at this latitude. Cells get narrower away from the equator, by
    a factor of cos(latitude), so a fixed precision is not safe."""
    for precision in range(9, 0, -1):
        h, w = cell_size(precision)
        width_km = w * 111.32 * math.cos(math.radians(abs(lat) + h))
        if min(h * 110.57, width_km) >= radius_km:
            return precision
    return 1
```

```
def nearby(places, point, radius_km):
    """places: list of (name, lat, lon). Search the 3 x 3 block of cells
    around the point, then keep places within radius_km."""
    precision = precision_for(point[0], radius_km)
    index = {}
    for name, lat, lon in places:
        index.setdefault(encode(lat, lon, precision), []).append(
            (name, lat, lon)
        )
    found = []
    for cell in neighbors(*point, precision):
        for name, lat, lon in index.get(cell, []):
            if km(point, (lat, lon)) <= radius_km:
                found.append(name)
    return sorted(found)
```

```
>>> nearby(
...     [('cafe', 51.5007, -0.1246),
...      ('museum', 51.5194, -0.127),
...      ('park', 51.6, -0.3)],
...     (51.5033, -0.1196),
...     3.0,
... )
['cafe', 'museum']
```

A Bug the Tests Found

The first version of `nearby` always used five-character cells, "about 4.9 km wide", for any radius up to 4.9 km. The random tests, which compare the search with a brute-force distance check, failed at latitude 59: a 3 km search missed a place. The reason is geometric: lines of longitude converge toward the poles, so a cell's **width shrinks by a factor of cos(latitude)**. At latitude 59, a "4.9 km" cell is only about 2.5 km wide, so the eight neighbors don't cover a 3 km radius.

The fix, `precision_for`, chooses the finest precision whose cells are at least as tall and as wide as the radius at that latitude. It's a good example of why tests against a simple, obviously correct version are worth having, and of an edge case worth mentioning in an interview: geohash cells are not squares everywhere.

Deep Dive 2: Alternatives

- **Quadtrees:** recursively split regions with too many places into four. Adapts to density, so dense cities get small cells and oceans large ones. Usually kept in memory.
- **S2 and H3 cell systems:** project the Earth onto a cube (S2) or a hexagonal grid (H3) to get more uniform cells and better neighbor handling.
- **Database support:** many databases provide spatial indexes (R-trees), which are enough for moderate scale.

In an interview, geohash is the easiest to explain; mention quadtrees or S2 as ways to handle uneven density.

Wrap-Up

- **Dense areas:** a cell in central Tokyo may hold thousands of places; return the closest and best rated, and paginate.
- **Ranking:** combine distance with rating, popularity and whether the place is open now.
- **Updates:** places change rarely; asynchronous index updates within minutes are fine.

Follow-Up Questions

- "Search along a route." Cover the route with cells and query them.
- "Very large radius." Use coarser cells, and rank by quality rather than listing everything.
- "Show places on a moving map." Query by the visible bounding box, cache tiles of results.

Key Takeaways

- Convert two-dimensional locations into one-dimensional keys (geohash, S2, H3) that ordinary indexes can search.
- Search the user's cell and its eight neighbors, then filter by exact distance.

- Choose cell size from the radius and the latitude; cells narrow toward the poles.
- The geo index is small: keep it in memory and replicate it.

CHAPTER 28

Ride Matching

The prompt: "Design the system that matches riders with nearby drivers in a ride-hailing app."

Compared with nearby places, everything moves: millions of drivers report their positions every few seconds, and a match must be made in about a second, without giving the same driver to two riders.

Requirements

Functional:

1. Drivers send their location every few seconds while online.
2. A rider requests a ride; the system finds a nearby available driver and offers the trip.
3. The rider sees the driver approach in real time.

Non-functional:

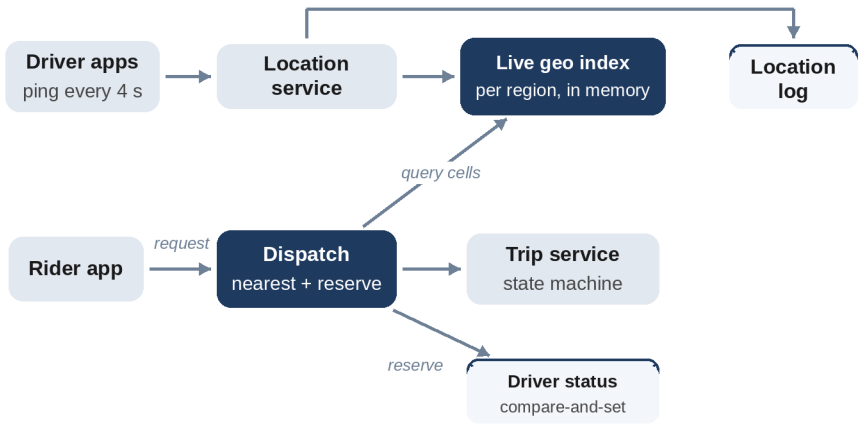
- 5 million drivers online at peak; tens of thousands of ride requests per second at peak.
- A match within a second or two.
- A driver is never assigned to two rides at once.

Estimates

```
drivers online (peak)                  5M
location pings per second (every 4 s)  1.25M
ping size                              about 100 B
ingest bandwidth                       125 MB/s
live driver index (100 B each)         500 MB
ride requests per second (peak)        50K
```

- **1.25 million location updates per second**, but tiny: about 125 MB per second in total.
- The **live index** of driver positions is only about 500 MB: it fits in memory, so the challenge is the update rate, not the size. Partition by city or region.
- Location history for analytics is written to a log, not to the live index.

High-Level Design



Ride matching: driver apps send location pings to location services that update an in-memory geo index partitioned by region; a dispatch service queries the index for nearby available drivers, reserves one atomically and sends the offer.

- **Location service:** receives pings, updates the **in-memory geo index** for that region, and appends the ping to a log for history and analytics.

- **Dispatch service:** for each ride request, queries the index for nearby available drivers, ranks them (by estimated time to pick up, not straight-line distance), **reserves** one atomically, and sends the offer.
- **Trip service:** stores the trip and its state machine (requested, accepted, arrived, in progress, completed), and streams the driver's position to the rider.

Deep Dive 1: The Live Index

Drivers move constantly, so the index must handle many more updates than queries. A simple grid works well: divide the map into cells of about 1 km and keep a set of drivers per cell. An update moves a driver between cells only when they cross a boundary. A query searches rings of cells outward from the rider and stops as soon as no unsearched cell could contain a closer driver:

```
class Dispatcher:
    def __init__(self):
        self.grid, self.where = {}, {}

    def update(self, driver, lat, lon):  # a driver's location ping
        if driver in self.where:
            self.grid[cell(*self.where[driver])].discard(driver)
        self.where[driver] = (lat, lon)
        self.grid.setdefault(cell(lat, lon), set()).add(driver)

    def remove(self, driver):
        self.grid[cell(*self.where.pop(driver))].discard(driver)

    def match(self, lat, lon, max_rings=50):
        """Nearest driver (ties: smallest name), removed from the pool."""
        cy, cx = cell(lat, lon)
        best = None
        for r in range(max_rings + 1):
            if best and (r - 1) * CELL > best[0]:
                break  # every unsearched cell is farther than best
            for dy in range(-r, r + 1):
                for dx in range(-r, r + 1):
                    if max(abs(dy), abs(dx)) != r:
                        continue  # only the ring at distance r
                    for d in self.grid.get((cy + dy, cx + dx), ()):
                        cand = (dist((lat, lon), self.where[d]), d)
                        best = min(best, cand) if best else cand
        if best:
            self.remove(best[1])
            return best[1]
        return None
```

```
>>> run(
...     [('d1', 40.7, -74.0), ('d2', 40.75, -73.99)],
...     [(40.74, -73.99), (40.74, -73.99), (40.74, -73.99)],
... )
['d2', 'd1', None]
```

The first request gets d2, the closest; the second gets d1, because d2 is no longer available; the third finds no one. The test compares the grid search with a brute force that checks every driver, over thousands of random driver and rider positions.

Deep Dive 2: Never Double-Book a Driver

Two dispatch servers may pick the same driver for two riders at the same moment. The reservation must be **atomic**: a compare-and-set on the driver's status from "available" to "offered to trip T" in the store that owns that driver (Chapter 17 shows why). If the set fails, try the next candidate. If the driver declines or doesn't respond within a few seconds, release them and offer the trip to the next driver.

Partitioning by region makes this simpler: all drivers in a city are owned by one partition, so the reservation is a local operation.

Deep Dive 3: Matching Quality

Nearest by straight line isn't the best match. Real dispatch estimates **arrival time** using road networks and traffic, and may batch requests for a second or two to make better assignments overall: matching two riders and two drivers jointly can beat matching each greedily. Mention this as an improvement after the basic design works.

Wrap-Up

- **Hot spots:** stadiums and airports concentrate requests and drivers; split hot cells finely.
- **Failure:** if a location server dies, drivers' next pings (within seconds) rebuild its part of the index on a replacement; positions are soft state.

- **Surge pricing:** compute supply and demand per cell in a stream processor, separate from the matching path.

Follow-Up Questions

- "Shared rides." Match a request to drivers whose existing route can absorb it with little detour.
- "Show nearby cars before requesting." Query the index for a few drivers' approximate positions; cache per cell for a second or two.
- "Prevent location spoofing." Detect impossible jumps and cross-check with other signals.

Key Takeaways

- The live driver index is small but updated constantly; keep it in memory, partitioned by region.
- Use a grid or geohash cells and search outward until no closer driver is possible.
- Reserve drivers atomically so concurrent dispatchers can't double-book them.
- Treat locations as soft state that the next pings rebuild after failures.

CHAPTER 29

Payment System

The prompt: "Design the payment system for an online marketplace: customers pay, the marketplace keeps a fee, sellers are paid out."

Payment systems are where "eventually consistent" and "at least once" stop being acceptable on their own. The design is shaped by one rule: **never lose money and never move it twice**, even when every network call can fail halfway.

Requirements

Functional:

1. Charge a customer's payment method for an order, through an external payment processor.
2. Record money owed to sellers and the marketplace's fee.
3. Pay sellers out periodically.
4. Refunds.

Non-functional:

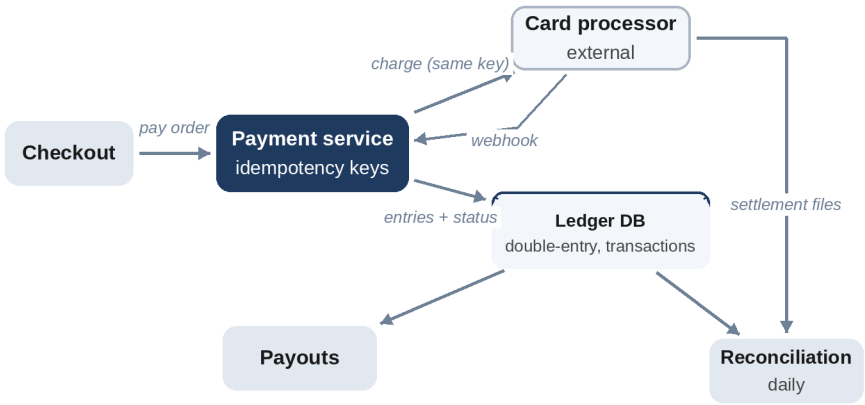
- About 50 million payments a day, with holiday peaks of ten times the average.
- Correctness over speed: no double charges, no lost payments, a complete audit trail.
- Reconcile every day against the processor's and the bank's records.

Estimates

```
payments per second (avg)            579
peak (x10, holiday sales)            5.79K
ledger rows per day (2 per payment)  100M
ledger storage, 7 years (200 B/row)  51.1 TB
```

Hundreds to thousands of payments per second: not an extreme rate. The difficulty is correctness, not throughput. Seven years of ledger history (a common retention requirement) is about 50 TB.

High-Level Design



Payments: the payment service records an intent with an idempotency key, calls the external processor, and writes balanced double-entry ledger entries in one transaction; webhooks and daily reconciliation catch anything missed.

1. The checkout service calls the **payment service** with an order ID and an **idempotency key**.
2. The payment service records a **payment intent** (status "pending") and calls the external **processor** with the same key.
3. On success, it writes **ledger entries** and marks the payment "succeeded", in one database transaction.
4. The processor also sends **webhooks** to confirm outcomes, and a daily **reconciliation** job compares the ledger with the processor's settlement files.

5. A **payout service** periodically transfers seller balances to their bank accounts.

Deep Dive 1: Idempotency

The most dangerous moment is a timeout: the processor may have charged the card even though no response arrived. Retrying blindly could charge twice; giving up could lose a valid payment. The answer is idempotency at every step:

- The client sends an idempotency key with each logical payment; retries reuse it.
- The payment service stores the key with the result, and returns the stored result for any repeat.
- The processor call uses the same key, so the processor also refuses to charge twice.

```
class Ledger:
    def __init__(self):
        self.entries = []  # (transfer_id, account, amount in cents)
        self.results = {}  # idempotency key -> result

    def balance(self, account):
        return sum(a for _, acct, a in self.entries if acct == account)

    def transfer(
        self, key, source, dest, cents, allow_overdraft=("world",)
    ):
        if key in self.results:
            return self.results[key]  # a retry: same answer, no new money
        if cents <= 0:
            result = "rejected: amount must be positive"
        elif source not in allow_overdraft and self.balance(source) < cents:
            result = "rejected: insufficient funds"
        else:
            tid = len(self.results) + 1
            self.entries += [(tid, source, -cents), (tid, dest, cents)]
            result = f"ok: transfer {tid}"
        self.results[key] = result
        return result
```

```
>>> run(
...     [('k1', 'world', 'ana', 5000),
...      ('k2', 'ana', 'shop', 3000),
...      ('k2', 'ana', 'shop', 3000),
...      ('k3', 'ana', 'shop', 3000)],
... )
(['ok: transfer 1',
```

```
  'ok: transfer 2',
  'ok: transfer 2',
  'rejected: insufficient funds'],
 {'ana': 2000, 'shop': 3000})
```

Ana tops up 50.00, then pays 30.00. The client's retry with key `k2` returns the original result without moving money again. A genuinely new payment with key `k3` is rejected, because only 20.00 is left. Amounts are stored in **integer cents**, never floating point, which can't represent most decimal amounts exactly.

Deep Dive 2: Double-Entry Bookkeeping

Each transfer writes **two entries that sum to zero**: money leaves one account and enters another. Balances are never updated in place; they are sums of entries, so the ledger is an append-only, auditable history. For a sale of 100.00 with a 10% fee, the entries move 100.00 from the customer's payment account to an escrow account, then 90.00 to the seller and 10.00 to the marketplace's revenue.

The test performs 2,000 random transfers, many of them retries of the same keys, and checks the invariants: all entries sum to zero (money is conserved), no customer balance goes negative, and every transfer has exactly two entries.

Deep Dive 3: Reconciliation and Failure Recovery

Even with idempotency, systems drift: a webhook is lost, a crash happens between the processor call and the ledger write. Two safety nets:

- **A recovery job** finds payments stuck in "pending" and asks the processor for their real status, using the idempotency key.
- **Daily reconciliation** compares every ledger entry with the processor's and bank's records; any mismatch creates an alert and a manual review case.

Assume something will go wrong, and design so it's detected and corrected, not silently accumulated.

Wrap-Up

- **Database:** a relational database with transactions, replicated synchronously within a region. Partition by account if needed, keeping both entries of a transfer in one transaction.
- **Security and compliance:** never store raw card numbers; use the processor's tokens, which keeps most of the system out of the scope of card-industry security standards. Encrypt sensitive data and audit access.
- **Asynchronous parts:** payouts, receipts and analytics are driven by events from the ledger (with the outbox pattern of Chapter 10).

Follow-Up Questions

- "Multiple currencies." Store currency per entry; convert through explicit exchange entries with recorded rates.
- "Partial refunds." New transfers in the opposite direction, linked to the original payment.
- "Fraud checks." A risk-scoring step before charging, with its own latency budget and a decision to fail open or closed.

Key Takeaways

- Every operation that moves money is idempotent, end to end, with the same key.
- Use double-entry bookkeeping: append-only entries that sum to zero, with balances computed from entries.
- Store money as integer minor units, never floating point.
- Recover stuck payments by asking the processor, and reconcile daily against external records.

CHAPTER 30

Ticket Booking

The prompt: "Design a ticket booking system for concerts, where a popular event sells out in minutes."

The design must handle an extreme spike, many thousands of buyers per seat, while guaranteeing that no seat is ever sold twice, and while being reasonably fair to the people waiting.

Requirements

Functional:

1. Browse events and see available seats.
2. Select seats, which are held for a few minutes while the buyer pays.
3. Pay to confirm; unpaid holds expire and the seats become available again.

Non-functional:

- A popular event: 60,000 seats and 3 million fans arriving within minutes.
- **No double booking**, ever.
- Fairness: roughly first come, first served.
- The rest of the site stays up during the rush.

Estimates

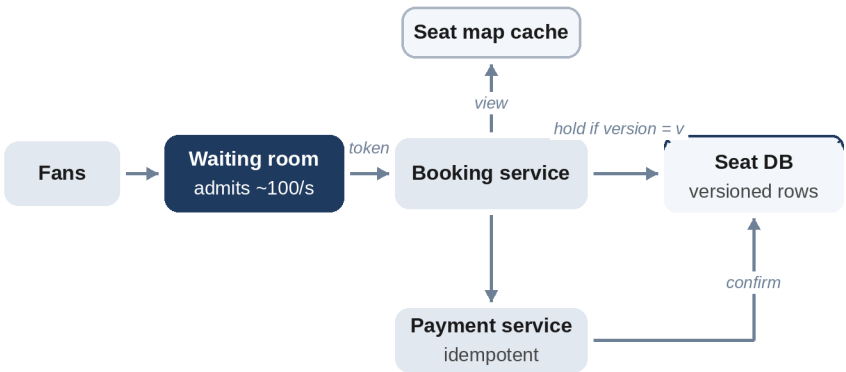
```
seats on sale                          60K
fans arriving in 10 minutes            3M
arrivals per second                    5K
buyers per seat                        50
queue admission rate (sell in ~30 min)  100 per second
```

Five thousand new arrivals per second, 50 buyers per seat. Letting everyone hit the seat database at once would overload it, and most would fail anyway. The numbers point to a **virtual waiting room** that admits buyers at a controlled rate, about 100 per second here, which still sells the event in about half an hour.

API

```
GET  /v1/events/{id}/seats            returns availability
  (cached, may be slightly stale)
POST /v1/events/{id}/holds  {seats}   returns {hold_id,
  expires_at} or 409
POST /v1/holds/{id}/purchase  {payment_token, idempotency_key}
```

High-Level Design



Ticket booking: a waiting room admits users at a controlled rate; the booking service holds seats with conditional updates in the seat database; holds expire automatically; payment confirms the hold.

- **Waiting room:** users join a queue and receive a token when admitted; only token holders can reach the booking pages.
- **Booking service:** holds and confirms seats in the **seat database**, using conditional updates.
- **Seat map cache:** serves availability views, refreshed every second or so; it may be slightly stale, but the hold itself is always checked against the database.
- **Payment service:** charges with idempotency (Chapter 29); success confirms the hold.

Deep Dive 1: Holding Seats Safely

Two buyers click the same seat at the same moment. Both read "free", and both try to hold it. Only one may succeed. Two standard approaches:

- **Pessimistic locking:** lock the seat row while checking and updating it. Correct, but locks are held across network calls and can pile up under heavy contention.
- **Optimistic concurrency control:** each seat row has a **version number**. A hold succeeds only if the version hasn't changed since the seat was read; otherwise the buyer is told the seat was taken. In SQL, that's `UPDATE seats SET state = 'held', ..., version = version + 1 WHERE id = ? AND version = ?`, followed by a check of how many rows were updated.

```
class Seats:
    def __init__(self, n, hold_seconds=300):
        self.rows = {
            s: {"state": "free", "who": None, "until": 0, "v": 0}
            for s in range(n)
        }
        self.hold_seconds = hold_seconds

    def read(self, seat, now):
        row = dict(self.rows[seat])
        if row["state"] == "held" and now >= row["until"]:
            row.update(state="free", who=None)  # the hold expired
        return row

    def write_if_unchanged(self, seat, expected_v, **changes):
        row = self.rows[seat]
        if row["v"] != expected_v:
            return False  # someone else changed the seat first
```

```
        row.update(changes)
        row["v"] += 1
        return True

    def hold(self, seat, user, now):
        row = self.read(seat, now)
        if row["state"] != "free":
            return False
        return self.write_if_unchanged(
            seat,
            row["v"],
            state="held",
            who=user,
            until=now + self.hold_seconds,
        )

    def confirm(self, seat, user, now):
        row = self.read(seat, now)
        if row["state"] != "held" or row["who"] != user:
            return False  # the hold expired or belongs to someone else
        return self.write_if_unchanged(seat, row["v"], state="sold")
```

Expired holds are treated as free at read time, so no background job is needed to release them, though one can tidy up. The test runs 50 random races of 30 users holding and buying 5 seats and checks that no seat is ever sold twice, and checks directly that when two users read the same free seat, only the first write wins.

Deep Dive 2: The Waiting Room

The waiting room protects everything behind it:

- Users arriving before the sale starts get a **random position**, which is fairer than rewarding the fastest connection; later arrivals join the end.
- The queue admits users at a rate the booking path can handle, and adjusts it as holds expire or seats sell out.
- Admission tokens are signed and short-lived, so they can't be shared or forged.
- The waiting room itself is mostly static content served from a CDN, so it can absorb millions of visitors cheaply.

Deep Dive 3: Bots and Abuse

Popular events attract scalpers' bots. Defenses: per-account purchase limits, verified accounts or presale codes for high-demand events, rate limiting and bot detection at the edge, and holds that require a logged-in session.

Wrap-Up

- **Hot rows:** contention on one event's seats is inherent; keep each hold a single, fast conditional update, and partition by event.
- **General admission** (no assigned seats) is simpler: a counter decremented with a conditional update, or tickets pre-split into buckets to spread contention.
- **Failure:** if payment fails after the hold, the hold simply expires; if payment succeeds but confirmation fails, the idempotency key lets the confirmation be retried safely.

Follow-Up Questions

- "Let users pick the best available seats automatically." Search free seats by section and price, then attempt holds on the top candidates.
- "Show live seat availability." Push updates over SSE from the seat cache, at most once a second per event.
- "Resale marketplace." A transfer of ownership with the same version checks and idempotent payments.

Key Takeaways

- Protect the booking path with a waiting room that admits users at a sustainable rate.
- Hold seats with conditional updates (optimistic concurrency) so concurrent buyers can't both win.
- Let holds expire by time; treat expired holds as free when reading.
- Serve availability from a cache, but always check the database when holding.

CHAPTER 31

Metrics and Alerting

The prompt: "Design a monitoring system that collects metrics from all of a company's servers, shows dashboards, and alerts engineers when something is wrong."

It's a write-heavy design with a long tail of storage: millions of time series written every few seconds, read mostly as aggregates over time ranges, and kept for years at decreasing resolution.

Requirements

Functional:

1. Collect numeric metrics (CPU, request counts, latencies) from many services, each with labels such as service, host and region.
2. Query and graph them over time ranges, with aggregation (sum, average, percentiles) across labels.
3. Alert rules that notify on-call engineers.

Non-functional:

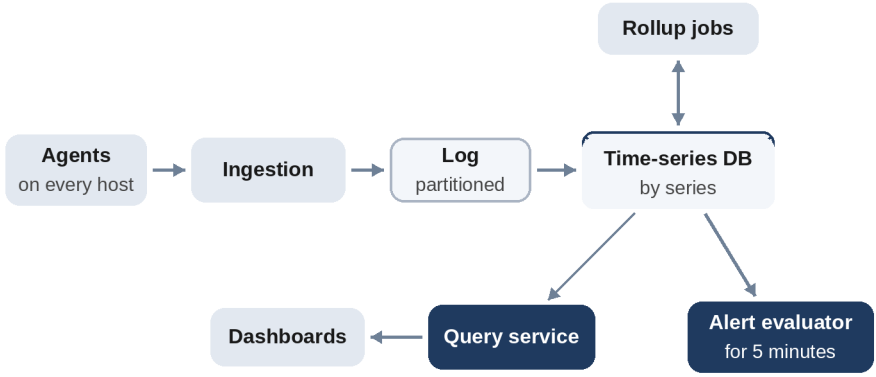
- 100 million active time series, one point every 10 seconds.
- Recent data queryable within seconds of being written.
- Raw data for about two weeks; downsampled data for years.
- The monitoring system must stay up when the systems it monitors are failing.

Estimates

```
points per second                    10M
raw, 15 days (2 B/point compressed)  25.9 TB
1-minute rollups, 1 year (16 B)      841 TB
1-hour rollups, 5 years (16 B)       70.1 TB
```

Ten million points per second. Time-series compression is very effective (consecutive timestamps and values are similar, so a point can compress to a couple of bytes), which keeps two weeks of raw data around 26 TB. Notice that one year of one-minute rollups (841 TB) costs far more than the raw data, so keep fine-grained rollups for months, not years, and hourly rollups for the long term (70 TB for five years).

High-Level Design



Metrics system: agents send points to ingestion services that write to a log; storage nodes consume the log into a partitioned time-series database; rollup jobs downsample; the query service serves dashboards; the alert evaluator runs rules and sends notifications.

1. **Agents** on each host collect metrics and send them in batches.
2. **Ingestion services** validate and write points to a partitioned **log** (Chapter 10), which buffers bursts and lets several consumers read.
3. **Storage nodes** consume the log into a **time-series database**, partitioned by series (a hash of the metric name and labels) and organized in time blocks.

4. **Rollup jobs** compute downsampled series.
5. The **query service** serves dashboards, choosing raw or rolled-up data based on the time range.
6. The **alert evaluator** runs rules every minute and sends alerts to the notification service (Chapter 21).

Deep Dive 1: Downsampling Without Losing Meaning

Averaging raw points into one-minute averages loses information: a spike disappears into an average, and an average of averages is wrong when buckets have different numbers of points. Keep **count, sum, minimum and maximum** per bucket instead; averages, totals and extremes can then be computed exactly at any coarser level:

```
def rollup(points, bucket=60):
    """points: (timestamp, value). Returns {bucket_start: (count, sum,
    min, max)}."""
    out = {}
    for t, v in points:
        b = t - t % bucket
        c, s, lo, hi = out.get(b, (0, 0, v, v))
        out[b] = (c + 1, s + v, min(lo, v), max(hi, v))
    return dict(sorted(out.items()))
```

```
>>> rollup([(0, 5), (10, 7), (65, 1), (70, 3)])
{0: (2, 12, 5, 7), 60: (2, 4, 1, 3)}
```

The test compares the rollup with a brute-force version that rescans the raw points for each bucket, on thousands of random series. Percentiles can't be combined this way; for latency, store a histogram or a quantile sketch per bucket instead.

Deep Dive 2: Alerts That Don't Cry Wolf

An alert that fires on every spike trains engineers to ignore alerts. Good rules require a condition to **hold for a duration** and target **symptoms** (Chapter 15):

```
def alert_times(series, threshold, hold):
    """series: (minute, value) per minute. Return the minutes at which an
    alert starts firing: value > threshold for `hold` minutes in a row."""
    fired, run = [], 0
    for minute, value in series:
        run = run + 1 if value > threshold else 0
        if run == hold:
            fired.append(minute)
    return fired
```

A CPU series with a one-minute spike at minute 3, and a real, sustained problem from minute 10:

```
alert on any minute > 90%:   [3, 10]
alert after 5 minutes > 90%: [14]
```

Alerting on any minute above 90% fires twice, once for the harmless spike; requiring five consecutive minutes fires once, at minute 14, for the real problem. Real systems add error-budget burn-rate alerts, which fire quickly for fast budget burn and slowly for slow burn.

Deep Dive 3: Cardinality

Each unique combination of labels is a separate series. A label with unbounded values, such as user ID or request ID, can turn thousands of series into billions and overwhelm the system. Enforce limits per service, reject or drop high-cardinality labels at ingestion, and put per-request detail in logs and traces instead.

Wrap-Up

- **Independence:** run monitoring on separate infrastructure, and monitor the monitoring (a "dead man's switch" alert that fires if no data arrives).
- **Hot queries:** dashboards refreshing every few seconds; cache recent query results and precompute common aggregates.

- **Retention tiers:** raw for weeks on fast storage, rollups for years on cheaper storage.

Follow-Up Questions

- "Pull or push collection?" Pull (the server scrapes targets) makes it easy to see which targets are down; push suits short-lived jobs. Many systems support both.
- "Global view across regions." Store per region; federate queries or replicate rollups centrally.
- "Anomaly detection." Compare current values with the same time last week, or with a forecast, and alert on large deviations.

Key Takeaways

- Metrics are write-heavy: buffer ingestion through a log and partition storage by series.
- Downsample with count, sum, min and max so aggregates stay exact; use histograms for percentiles.
- Alert on persistent, user-facing symptoms, not on single spikes.
- Control label cardinality; it's the most common way to break a metrics system.

CHAPTER 32

Distributed Job Scheduler

The prompt: "Design a service that runs scheduled and delayed jobs, such as 'send this reminder at 9:00' or 'run this report every night', reliably across a fleet of workers."

The challenge is reliability: jobs must run at about the right time, at least once, even when workers crash mid-job, and ideally never twice at the same time.

Requirements

Functional:

1. Submit one-off jobs at a time, delayed jobs, and recurring jobs (cron-style schedules).
2. Workers execute jobs; results and failures are recorded.
3. Retries with backoff for failed jobs.

Non-functional:

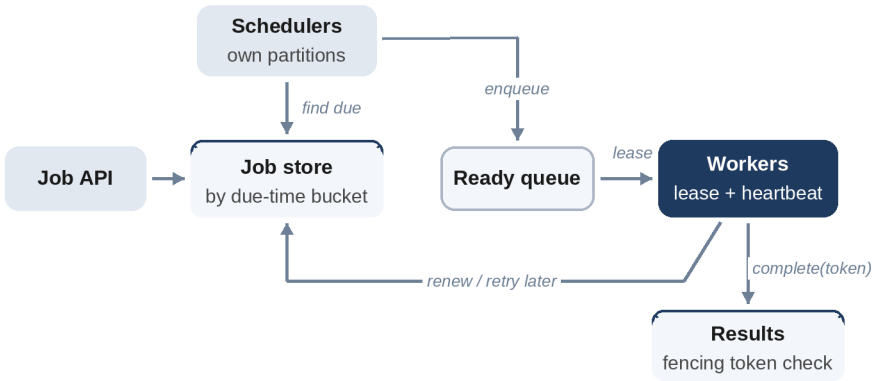
- About a billion job executions a day, with spikes when many jobs are due at the same moment (on the hour).
- Jobs start within a few seconds of their due time.
- Every job runs at least once; jobs should be idempotent so a rare second run is harmless.

Estimates

```
jobs per second (avg)                          11.6K
jobs due on the hour (x20 spike)               231K
heartbeats per second (50K running, every 5 s)  10K
job rows kept 30 days (500 B)                  15 TB
```

The average rate is modest, about 12,000 jobs per second, but schedules cluster: twenty times that at the top of the hour is common, because people schedule "every hour" on the hour. Spreading recurring jobs across a few minutes with jitter, when the schedule allows it, reduces the spike.

High-Level Design



Job scheduler: jobs are stored with their next run time, partitioned by time bucket; schedulers move due jobs to a ready queue; workers lease jobs, send heartbeats, and complete them with a fencing token.

1. **Job store:** each job with its next run time, partitioned so that "jobs due in this minute" is a cheap query, for example by time bucket and a hash.
2. **Schedulers:** each owns some partitions, and every second moves due jobs into a **ready queue**. Recurring jobs get their next run time computed and stored.
3. **Workers:** take jobs from the queue under a **lease**, run them, heartbeat while running, and report completion.

Deep Dive 1: Leases, Heartbeats and Crashes

If a worker takes a job and crashes, the job must not be lost. So a worker doesn't simply take a job; it **leases** it for a period, say 10 seconds, and renews the lease with **heartbeats** while running. If the heartbeats stop, the lease expires and another worker can take the job.

There's a subtle danger: a worker that pauses (a long garbage-collection pause, a network hiccup) may lose its lease, wake up, and finish the job after a new worker has started it. Its late result could overwrite the new one. The fix is a **fencing token**: each lease gets an increasing number, and completion is accepted only with the current token.

```
class Scheduler:
    def __init__(self, jobs, lease):
        self.lease = lease
        self.jobs = {
            j: {"state": "ready", "owner": None, "until": 0, "token": 0}
            for j in jobs
        }
        self.results = {}

    def acquire(self, worker, now):
        for job, info in self.jobs.items():
            expired = info["state"] == "running" and now >= info["until"]
            if info["state"] == "ready" or expired:
                info.update(
                    state="running", owner=worker, until=now + self.lease
                )
                info["token"] += 1
                return job, info["token"]
        return None

    def heartbeat(self, job, token, now):
        info = self.jobs[job]
        if (
            info["token"] == token
            and info["state"] == "running"
            and now < info["until"]
        ):
            info["until"] = now + self.lease
            return True
        return False

    def complete(self, job, token, result):
        info = self.jobs[job]
        if info["token"] != token:
            return False  # a newer lease exists: reject the stale write
        info["state"] = "done"
        self.results[job] = result
        return True
```

The simulation runs 20 jobs on 4 workers that randomly crash, pause without heartbeating, and resume. Across 40 random runs, the test checks that every job finishes, despite crashes and expired leases; stale completions from workers that lost their lease are rejected by the token check.

Deep Dive 2: Exactly Once Is Not Free

Leases give **at least once** execution: a job can run twice when a lease expires while the original worker is still working. Fencing tokens stop the *stale result* from being recorded, but they can't undo side effects the old worker already performed, such as sending an email. So jobs must be **idempotent**: use the job ID as an idempotency key for external effects (Chapter 4), and design steps that are safe to repeat.

Deep Dive 3: Scheduling Accuracy and Scale

- **Time buckets:** store jobs by due minute; a scheduler reads the current bucket, which keeps "find due jobs" fast even with billions of future jobs.
- **Partitioning schedulers:** assign partitions to scheduler instances with leases too, so a crashed scheduler's partitions are taken over.
- **Priorities and isolation:** separate queues for latency-sensitive jobs and big batch jobs, so a nightly report flood can't delay a 9:00 reminder.
- **Retries:** failed jobs return to the store with a later run time (exponential backoff), and move to a dead-letter state after a maximum number of attempts.

Wrap-Up

- **Clock dependence:** schedulers depend on time; keep clocks synchronized and tolerate small skew.
- **Observability:** queue depth, scheduling delay (due time vs. start time), failure rate per job type.
- **Very long jobs:** split into steps with checkpoints, so a crash doesn't restart hours of work.

Follow-Up Questions

- "Jobs with dependencies" (run B after A). A workflow engine: store the dependency graph and schedule each job when its inputs are complete (topological order).
- "Run a job on exactly one machine at a time." Leases plus fencing tokens, as above; for strict guarantees, a lock service built on consensus.
- "Pause and cancel jobs." State changes in the job store, checked by workers at heartbeat time.

Key Takeaways

- Store jobs by due time, move due jobs to a ready queue, and let workers lease them.
- Leases with heartbeats recover jobs from crashed workers; fencing tokens reject stale results.
- Execution is at least once; make jobs idempotent.
- Isolate priorities, and spread recurring jobs away from the top of the hour.

CHAPTER 33

Real-Time Leaderboard

The prompt: "Design a leaderboard for an online game with millions of players: show the top 100 and each player's own rank, updated in real time."

The top 100 is easy; **"what is my rank among 50 million players?"** is the interesting part, because a database query that counts everyone with a higher score is far too slow to run on every request.

Requirements

Functional:

1. Update a player's score after each game.
2. Show the top N players.
3. Show any player's rank, and the players just above and below them.
4. Leaderboards per season (and optionally per region or among friends).

Non-functional:

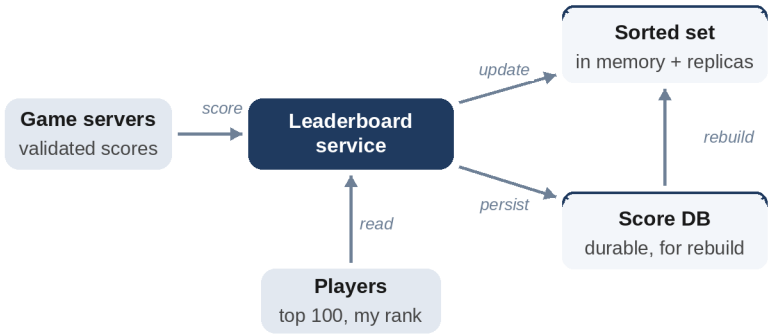
- 50 million players; about 100,000 score updates and 300,000 rank reads per second at peak.
- Ranks visible within a second or so of a game ending.

Estimates

```
players                                 50M
score updates per second (peak)         100K
rank reads per second (peak)            300K
sorted-set memory (about 100 B/player)  5 GB
Fenwick tree for scores 0..1,000,000    8 MB
```

A sorted set holding every player needs only about 5 GB, so the whole leaderboard fits in memory on one machine; the challenge is the operation rate and availability, not size.

High-Level Design



Leaderboard: game servers send score updates to a leaderboard service backed by an in-memory sorted set (with replicas for reads); scores are also written durably to a database for recovery.

Game servers report results to the **leaderboard service**, which updates an **in-memory sorted structure** and writes the score durably to a **database** for recovery and history. Reads of the top N, a player's rank and their neighbors are served from memory, with read replicas for scale.

Deep Dive 1: Data Structures for Rank

- **A database with an index on score:** `SELECT COUNT(*) WHERE score > ?` scans a large part of the index for low-ranked players. Too slow at this scale.

- **A sorted set** (a skip list or balanced tree that tracks subtree sizes, as Redis's sorted sets do): updates, top-N and rank queries all in O(log n). The standard answer.
- **Counting by score:** when scores are bounded integers, such as 0 to 1,000,000 points, keep a **count of players per score** in a Fenwick tree (a binary indexed tree). A player's rank is one plus the number of players with a higher score, a prefix sum in O(log max_score), whatever the number of players.

```
class Leaderboard:
    def __init__(self, max_score):
        self.n = max_score + 1
        self.tree = [0] * (self.n + 1)  # Fenwick tree of player counts
        self.score = {}

    def _add(self, s, delta):
        i = s + 1
        while i <= self.n:
            self.tree[i] += delta
            i += i & -i

    def _count_upto(self, s):  # players with score <= s
        i, total = s + 1, 0
        while i > 0:
            total += self.tree[i]
            i -= i & -i
        return total

    def set_score(self, player, s):
        if player in self.score:
            self._add(self.score[player], -1)
        self.score[player] = s
        self._add(s, 1)

    def rank(self, player):
        """1 + number of players with a strictly higher score."""
        s = self.score[player]
        return 1 + len(self.score) - self._count_upto(s)
```

```
>>> run(
...     100,
...     [('ana', 50), ('ben', 70), ('cy', 50), ('ana', 80)],
...     ['ana', 'ben', 'cy'],
... )
[1, 2, 3]
```

Ana's score improves from 50 to 80, so she's first; Ben with 70 is second; Cy with 50 is third. The test compares ranks with a brute-force count on thousands of random update sequences. The Fenwick tree for a million possible scores is only 8 MB, whatever the number of players.

Deep Dive 2: Ties and Ordering

Players with equal scores need a rule: same rank for everyone with that score (as above), or a tie-breaker such as "who reached the score first". To encode a tie-breaker in a sorted set, combine score and time into one sortable key, for example score $\times 10^{1}$ minus the timestamp, so that among equal scores the earlier one ranks higher.

Deep Dive 3: Scaling Beyond One Machine

If one machine can't handle the update rate:

- **Partition by score range:** each shard holds a range of scores and knows the count of players in all higher ranges, so rank = higher-range count + rank within the shard. Shards for crowded score ranges must be split finer.
- **Approximate ranks** for players far down the board: "top 37%" is often enough, computed from a histogram of scores, which is cheap to maintain.
- **Batch updates:** apply score updates in small batches every few hundred milliseconds instead of one by one.

Wrap-Up

- **Recovery:** rebuild the in-memory structure from the database on restart; replicas take over meanwhile.
- **Seasons:** a new leaderboard key per season; archive old ones.
- **Friends leaderboards:** small sets, so compute them on demand from friends' scores rather than maintaining a separate structure.
- **Cheating:** validate scores on game servers, never trust clients.

Follow-Up Questions

- "Show the players just above and below me." In a sorted set, fetch by rank range around the player's rank.
- "Daily, weekly and all-time boards." Separate structures, each updated on every game; expire daily boards.

- "Global board across regions." Regional boards plus a merged global top N, updated every few seconds.

Key Takeaways

- Counting higher scores in a database is too slow at scale; keep a sorted structure in memory.
- Sorted sets give O(log n) updates and ranks; with bounded scores, a Fenwick tree of counts per score is even smaller.
- Decide on a tie rule and encode it in the sort key.
- Persist scores durably and rebuild memory from them after failures.

CHAPTER 34

Collaborative Document Editing

The prompt: "Design a collaborative document editor where several people edit the same document at the same time and see each other's changes live."

The defining problem is **concurrent edits**: two people type at the same moment in different places, or the same place, and both must end up with exactly the same document, without locking anyone out.

Requirements

Functional:

1. Several users edit a document simultaneously; changes appear for everyone within a fraction of a second.
2. Everyone converges on the same final text.
3. Show other users' cursors; keep version history.
4. Edits made offline sync when the user reconnects (a follow-up).

Non-functional:

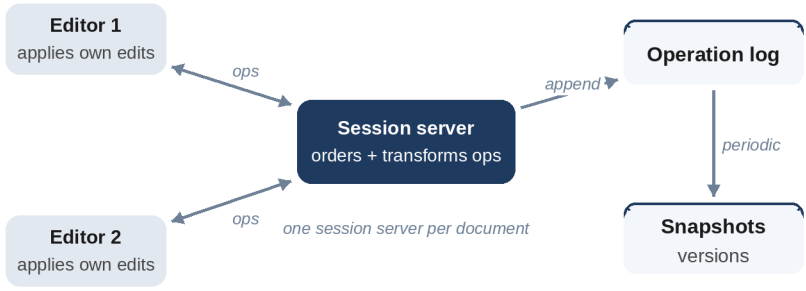
- Millions of documents open at once; typically a few editors per document, occasionally hundreds.
- Local typing must feel instant, without waiting for the server.

Estimates

```
documents open at once                      5M
editors per active document (avg)           2
operations per second per editor (typing)   5
operations per second, total                50M
operation size                              about 50 B
op log per day (8 busy hours, all kept)     72 TB
```

Tens of millions of small operations per second in total, but each document's traffic is tiny and independent. That's the key to scaling: **partition by document**.

High-Level Design



Collaborative editing: each client applies its own edits immediately and sends operations over a WebSocket to the document's session server, which transforms them against concurrent operations, assigns an order, broadcasts them and appends them to the operation log.

- Each open document is assigned to one **session server** (by consistent hashing on the document ID). All editors of that document connect to it over WebSockets.
- Clients apply their own edits immediately and send **operations** ("insert 'h' at position 1").
- The session server establishes one order of operations, transforms concurrent ones, broadcasts them, and appends them to an **operation log**.

- Periodic **snapshots** of the document let new editors load quickly without replaying the entire log, and serve as version history.

Deep Dive 1: Why Concurrent Operations Need Transforming

Start with "cat". User 1 inserts "h" at position 1 ("chat"); at the same moment, user 2 deletes position 2, the "t" ("ca"). If user 1 then applies user 2's operation unchanged, it deletes position 2 of "chat", which is now "a":

```
without transforming, site 1 gets: 'cht'
with OT, both sites get:           ('cha', 'cha')
```

User 1 would see "cht" while user 2 sees "cha". **Operational transformation** (OT), introduced by Ellis and Gibbs in 1989, rewrites each incoming operation to account for the concurrent ones already applied: since user 1's insert came before position 2, user 2's delete must shift to position 3.

```
def transform(a, b):
    """Rewrite op `a` so it can be applied after concurrent op `b`."""
    if a[0] == "noop" or b[0] == "noop":
        return a
    if a[0] == "ins" and b[0] == "ins":
        if a[1] < b[1] or (a[1] == b[1] and a[3] < b[3]):
            return a
        return ("ins", a[1] + 1, a[2], a[3])
    if a[0] == "ins" and b[0] == "del":
        return a if a[1] <= b[1] else ("ins", a[1] - 1, a[2], a[3])
    if a[0] == "del" and b[0] == "ins":
        return a if a[1] < b[1] else ("del", a[1] + 1)
    if a[1] == b[1]:  # both deleted the same character
        return ("noop",)
    return a if a[1] < b[1] else ("del", a[1] - 1)
```

The four cases are insert-insert (shift if the other insert came first; break ties by site ID), insert-delete, delete-insert, and delete-delete (deleting the same character twice becomes a no-op). The test generates 20,000 random pairs of concurrent operations on random short texts and checks that both sites always converge.

This model handles two concurrent operations. Full OT systems handle long sequences of concurrent operations by having the server define a single order and transform each incoming operation against everything the client hadn't

seen; that's where most of the complexity, and many of the historical bugs, of OT lives.

Deep Dive 2: OT or CRDTs?

CRDTs (conflict-free replicated data types, formalized by Shapiro and colleagues in 2011) take a different approach: every character gets a unique, ordered identifier, so concurrent inserts and deletes commute by construction and replicas converge without a central server transforming operations.

	OT	CRDT
Needs a central server	Typically yes, to order operations	No; works peer to peer and offline
Metadata	Small	Per-character identifiers; larger documents
Offline editing	Harder	Natural
Track record	Long, in centralized editors	Growing, especially for local-first apps

For a centralized editor with a server per document, OT is a natural fit. For offline-first or peer-to-peer collaboration, CRDTs are the usual choice. Naming the trade-off is what interviewers look for.

Deep Dive 3: Session Servers and Persistence

- **One owner per document** keeps ordering simple. If the session server fails, clients reconnect, a new server loads the latest snapshot plus the log, and clients resend unacknowledged operations, deduplicated by operation ID.
- **Popular documents** with hundreds of editors still fit on one server; documents viewed by thousands can stream updates through a separate read-only fan-out.
- **Cursors and presence** are ephemeral messages, not stored in the log.

Wrap-Up

- **History:** snapshots plus the operation log give version history and undo for free.
- **Permissions:** checked when joining the session and on every operation.
- **Rich text:** formatting is an additional attribute on ranges, with its own transformation rules.

Follow-Up Questions

- "Offline editing for hours." Buffer operations locally; on reconnect, transform them against everything that happened meanwhile, or use a CRDT.
- "Comments anchored to text." Anchor to stable positions that move with edits (transformed like cursors).
- "Search across documents." Index snapshots asynchronously.

Key Takeaways

- Clients apply their own edits instantly and exchange operations; everyone must converge.
- Operational transformation rewrites concurrent operations so they apply correctly; ties need a deterministic rule.
- CRDTs converge without a central server and suit offline editing; OT suits centralized editors.
- Partition by document: one session server per document orders its operations.

CHAPTER 35

Distributed Message Queue

The prompt: "Design a distributed message queue: producers publish messages to topics, consumers read them, at very high throughput and without losing data."

This design turns Chapter 10 inside out: instead of using a queue, you build one. The model is a **partitioned, replicated, append-only log**, the architecture popularized by Kafka.

Requirements

Functional:

1. Producers publish messages to named **topics**.
2. Consumers subscribe in **consumer groups**: each message is processed by one consumer per group, and different groups each see every message.
3. Messages with the same key are delivered in order.
4. Messages are retained for a configurable time (days), so consumers can replay.

Non-functional:

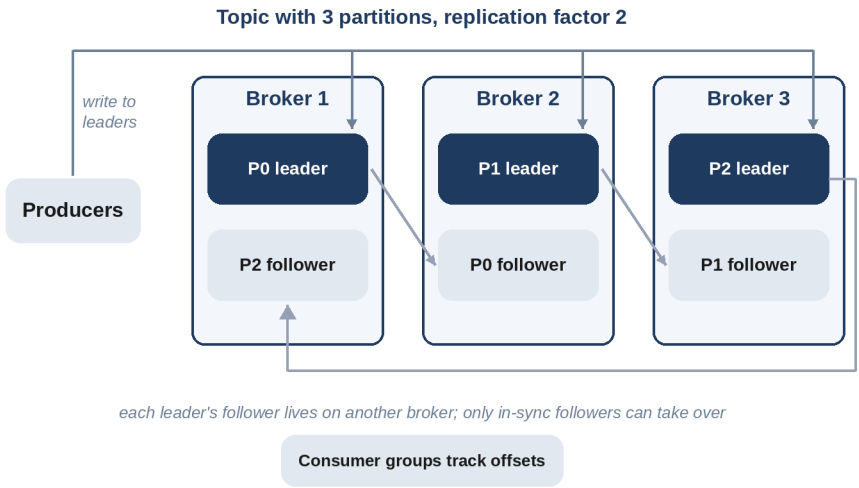
- Millions of messages per second; low latency (milliseconds).
- Durable: acknowledged messages survive machine failures.
- Horizontally scalable.

Estimates

```
messages per second (peak)   2M
ingest                       2 GB/s
average rate (peak / 3)      667K
retained 7 days, 3 replicas  1.21 PB
partitions at 10 MB/s each   200
```

About 2 GB per second at peak, and over a petabyte retained for a week with three replicas. Per-partition throughput of about 10 MB per second suggests a couple of hundred partitions for the largest topics.

High-Level Design



Distributed message queue: topics are split into partitions spread across brokers; each partition has a leader and followers that replicate it; producers write to leaders; consumer groups track offsets per partition.

- A **topic** is split into **partitions**; each partition is an append-only log of messages numbered by **offset**.
- **Brokers** store partitions on disk. Each partition has a **leader** broker and **follower** replicas on other brokers.
- **Producers** send a message to the leader of its partition, chosen by hashing the message key.

- **Consumer groups** divide a topic's partitions among their members, and record a committed offset per partition.
- A **controller**, backed by a consensus system, tracks brokers, partition leaders and group membership.

Deep Dive 1: Partitions, Keys and Offsets

```
class Log:
    def __init__(self, partitions):
        self.parts = [[] for _ in range(partitions)]
        self.committed = {}  # (group, partition) -> next offset

    def produce(self, key, value):
        p = zlib.crc32(key.encode()) % len(self.parts)
        self.parts[p].append((key, value))
        return p, len(self.parts[p]) - 1

    def poll(self, group, partition, max_n=10):
        start = self.committed.get((group, partition), 0)
        return start, self.parts[partition][start : start + max_n]

    def commit(self, group, partition, next_offset):
        self.committed[(group, partition)] = next_offset
```

Hashing the key picks the partition, so all messages for one key stay in order. Consumers poll from their committed offset and commit after processing. If a consumer crashes after processing a batch but before committing, the group re-reads from the last commit:

```
12 events, consumer crashed once: 15 deliveries
duplicates: ['event 5', 'event 8', 'event 9']
```

Twelve events produced 15 deliveries: a batch of three was processed, the consumer crashed before committing, and the batch was delivered again. That's **at-least-once** delivery, the normal guarantee; consumers deduplicate or make processing idempotent (Chapter 10). The test checks the full set of properties on 60 messages: without crashes, every message is delivered exactly once; per-key order is preserved; a second pass delivers nothing new; and after a crash, every message is still delivered, some more than once.

Deep Dive 2: Durability and Replication

A message is **acknowledged** to the producer once it's written to the leader and replicated to enough followers. The set of followers that are fully caught up is the **in-sync replica set**; the producer can require acknowledgement from all of them. If the leader fails, the controller promotes an in-sync follower, so no acknowledged message is lost. Trade-off: waiting for followers adds latency; acknowledging after the leader alone is faster but can lose recent messages on failover.

Deep Dive 3: Why It's Fast

- **Sequential disk I/O:** appending to a log and reading it in order is fast even on spinning disks, and very fast on SSDs.
- **Batching and compression:** producers send batches, compressed; brokers write and replicate whole batches.
- **The operating system's page cache** serves recent messages from memory, and data can go from disk to network without being copied through the application.
- **Partitions** spread load across brokers and allow consumers to work in parallel.

Wrap-Up

- **Rebalancing:** when consumers join or leave a group, partitions are reassigned; consumers resume from committed offsets.
- **Retention:** delete old log segments by time or size, or **compact** them, keeping only the latest message per key (useful for change streams).
- **Ordering limits:** order is per partition; adding partitions later changes which partition a key maps to, so plan partition counts ahead.
- **Exactly-once processing:** combine idempotent producers (deduplicating retries by producer sequence numbers) with transactional writes of results and offsets.

Follow-Up Questions

- "Delayed messages." A separate delay topic or a timer wheel that republishes messages when due.
- "Very slow consumers." Lag monitoring and alerts; longer retention; separate consumer groups for slow analytics.
- "Multi-region." Replicate topics asynchronously between regions, accepting that consumers in the second region see messages slightly later.

Key Takeaways

- A partitioned, replicated, append-only log provides high throughput, ordering per key, retention and replay.
- Consumers track offsets per partition; committing after processing gives at-least-once delivery.
- Acknowledge writes after in-sync replicas have them, and promote only in-sync followers.
- Speed comes from sequential I/O, batching, compression and partition parallelism.

PART IV

Practice

CHAPTER 36

Four Full Mock Interviews

These four interviews replay complete system design rounds as conversations. The prompts are new, not repeats of Part III, so you can practice on them first: read only the prompt, set a timer for 40 minutes, and design out loud on paper. Then read the conversation and compare your clarifying questions, estimates, design and trade-offs.

Each interview targets a different level, and ends with the interviewer's notes, the kind of written feedback that goes to a hiring committee or debrief.

Mock 1: Photo Sharing (Mid-Level)

The prompt: "Design a photo-sharing app: users upload photos and see photos from people they follow."

Candidate: Before I start, I'd like to pin down scope. I'm thinking: upload a photo with a caption, follow other users, and a home feed of photos from people you follow, newest first. I'll leave out comments, stories, messaging and search unless you want them. Does that work?

Interviewer: Yes. Add likes, but only as a count.

Candidate: Got it. How many users should I design for?

Interviewer: You decide.

Candidate: I'll assume 10 million uploads a day, about 2 MB each, 100 views per photo on average, and photos kept for five years with three copies. That gives about 116 uploads per second and about 11,600 views per second on

average, roughly 35,000 at peak. Storage is about 110 PB over five years with three copies. View bandwidth is around 23 GB per second. So it's very read-heavy, the images can't live in a database, and we need a CDN.

Interviewer: Good. What does the API look like?

Candidate: Three main calls: `POST /photos` to upload, which returns a photo ID; `GET /feed?cursor=` for the feed, cursor-paginated so new photos don't make pages repeat; and `POST /photos/{id}/like`. For uploads, I'd have the client request a pre-signed URL and upload the image straight to object storage, so our servers don't carry the bytes.

Interviewer: Walk me through the design.

Candidate: Clients talk to a load balancer and stateless API servers. Photo metadata, owner, caption, storage key and creation time, goes in a database partitioned by photo ID; the follow graph in its own table with indexes in both directions. Images go to object storage, served through a CDN, with a few resized versions generated by a processing worker after upload. For the feed, when a photo is posted I'd publish an event, and fan-out workers push the photo ID into each follower's feed list in a distributed cache. Reading the feed is then one cache read plus a batched lookup of photo metadata.

Interviewer: What about someone with 50 million followers?

Candidate: Pushing to 50 million feeds per photo would be a huge write spike. I'd use a hybrid: above some follower threshold, say 100,000, don't fan out. When a user reads their feed, merge their pushed feed with recent photos from the few big accounts they follow. That keeps reads fast and avoids the write explosion.

Interviewer: And likes?

Candidate: A popular photo can get thousands of likes per second, so updating one counter row would be a hot spot. I'd record likes in a likes table keyed by photo and user, so each user can like only once, and maintain the count asynchronously: aggregate like events in a stream and update the count every few seconds. Users won't notice a count that's a few seconds behind.

Interviewer: What would break first if traffic grew ten times?

Candidate: The CDN scales; the fan-out workers and the feed cache would be first. I'd watch fan-out lag and cache memory, add workers, and partition the cache further. The metadata database also needs read replicas and a cache for hot photos.

Interviewer's notes. *Hire at mid-level. Scoped the problem crisply and made explicit assumptions. Estimates were correct and, importantly, used: object storage, CDN, read-heavy design all followed from them. Clean API with cursor pagination and direct uploads. Knew the push/pull hybrid for large accounts without prompting beyond the question. Asynchronous like counts was a good touch. Deep dives were adequate rather than deep; failure modes were not discussed until asked.*

Mock 2: Distributed Cache (Senior)

The prompt: "Design a distributed in-memory cache that application servers across the company can use."

Candidate: Let me clarify the use: a shared key-value cache with get, set with a TTL, and delete, used by many services to reduce database load. Values up to, say, 1 MB. Is persistence required?

Interviewer: No, it's a cache. But a node failure shouldn't send a flood of traffic to the databases.

Candidate: Understood. Scale?

Interviewer: About 10 million operations per second and 30 TB of cached data.

Candidate: If each node has 256 GB of usable memory, 30 TB needs about 120 nodes for memory alone. At, say, 200,000 operations per second per node, 10 million needs about 50 nodes, so memory decides: around 120 nodes, more with replicas. Clients need to find the right node for each key without a central lookup on every request.

Interviewer: How?

Candidate: Consistent hashing with virtual nodes. Each client library holds the ring and hashes keys to nodes directly, so a request is one network hop. When a node is added or removed, only about 1/N of keys move, instead of

almost all of them with hash mod N. Virtual nodes keep the load even; with around a hundred per node, the busiest node is within about 15% of its fair share. Ring membership comes from a configuration service backed by consensus, and clients watch it for changes.

Interviewer: A node dies. What happens?

Candidate: Its keys now map to the next nodes on the ring, which don't have them, so those requests miss and go to the databases. With 120 nodes that's under 1% of keys, which may be fine. If even that's too much for some databases, two options: replicate each key to two nodes, so a failure doesn't cause misses, at twice the memory cost; or, more cheaply, protect the databases with request coalescing, so concurrent misses for the same key cause only one database read.

Interviewer: Let's talk about hot keys.

Candidate: A key that's extremely popular, like a homepage configuration, overloads its node regardless of hashing. I'd detect hot keys by sampling requests, perhaps with a Count-Min sketch, and then either replicate them to several nodes and let clients pick one at random, or cache them briefly inside the client library itself. A local cache of even a second absorbs most of the load.

Interviewer: How do you keep it consistent with the database?

Candidate: I'd recommend cache-aside to application teams: read from the cache, fall back to the database, and on writes update the database then delete the key. Deleting rather than setting avoids races between writers. TTLs bound any staleness that slips through. For stampedes when a hot key expires, add jitter to TTLs and use coalescing.

Interviewer: Memory management on each node?

Candidate: LRU eviction per node, with memory allocated in size classes to limit fragmentation. Large values should be discouraged; I'd cap them and expose metrics on value sizes, hit rates and evictions per client, so teams can tune their usage.

Interviewer's notes. *Strong hire, senior. Drove the conversation and quantified everything: node count from memory versus throughput was*

exactly the right analysis. Clear explanation of consistent hashing and why virtual nodes matter, with numbers. Handled node failure with two options and their costs. Good depth on hot keys and stampedes. Thought about the platform's users (guidance, limits, metrics), which is a senior signal.

Mock 3: Online Auctions (Senior)

The prompt: "Design an online auction site. Sellers list items; buyers bid until the auction ends; the highest bid wins."

Candidate: Key requirements as I understand them: list items with an end time; place bids; show the current highest bid in near real time; at the end, determine the winner and start payment. A bid must be higher than the current highest by a minimum increment. Is that right?

Interviewer: Yes. Popular auctions get very busy in the last minute.

Candidate: So the hard parts are correctness of bids under heavy concurrency, especially at the end, and fairness of the end time. Let me estimate. Say a million active auctions and 5,000 bids per second normally. A hot auction might receive a few thousand bids in its final minute, so per-auction bursts of 50 to 100 bids per second.

Interviewer: OK.

Candidate: For correctness, each auction's state, current highest bid and bidder, version and end time, lives in one row or one partition. Placing a bid is a conditional update: "set highest to X if X ≥ current + increment and now < end time and version = v". If the version changed, re-read and re-check. That guarantees no two bids both win, the same optimistic approach as booking a seat. Every accepted and rejected bid is also appended to a bid log, for audit and disputes.

Interviewer: Couldn't contention at the end make most bids fail and retry?

Candidate: At 100 bids per second on one auction, conditional updates on a single row are fine for a modern database. If an auction were much hotter, I'd route all bids for that auction to a single owner process that applies them in order from an in-memory queue, writing results durably. That serializes them

without lock contention. Partitioning auctions across owners by auction ID keeps it scalable.

Interviewer: How do bidders see the current price?

Candidate: Clients on an auction page subscribe over WebSockets or Server-Sent Events. When a bid is accepted, an event goes to a pub/sub channel for that auction, and gateways push it to subscribers. A slow or reconnecting client fetches the current state on reconnect. Price updates may lag by a fraction of a second, but the decision about a bid is always made against the database, never against what the client saw.

Interviewer: What about the end time?

Candidate: The server's clock decides, and bids are timestamped on arrival at the bidding service, not by clients. To prevent last-second sniping, many auction sites extend the end time by a minute or two whenever a bid arrives near the end; that's a product decision I'd raise with you. Closing is a scheduled job per auction, using leases so exactly one worker closes it, with an idempotent close operation: determine the winner from the final state, mark the auction closed, and start payment with an idempotency key.

Interviewer: What if the closing job runs twice?

Candidate: The close is a conditional update from "open" to "closed", so the second run sees it's already closed and does nothing. Payment uses the auction ID as its idempotency key, so it can't charge twice.

Interviewer's notes. *Hire, senior. Immediately identified correctness under contention and end-time fairness as the hard parts. Conditional updates with versions, plus the single-owner serialization option for extreme hot spots, were well reasoned. Separated the real-time display path from the authoritative decision path. Good handling of closing with leases and idempotency. Could have estimated storage and discussed search for listings, but the focus on the hard parts was right.*

Mock 4: Global Configuration Service (Staff)

The prompt: "Design a service that delivers configuration and feature flags to all services in a company, across several regions."

Candidate: Let me understand the users and the risks. Engineers change configuration, such as feature flags, timeouts and rollout percentages, and thousands of services read it. The biggest risk with configuration is outages caused by bad changes, so safety matters as much as speed. Requirements I'd propose: changes reach all services within seconds; reads are local and never block a service; changes are versioned, reviewed and can be rolled back; and staged rollout by region. Does that match your intent?

Interviewer: Yes. Assume 50,000 service instances and a few hundred changes a day.

Candidate: Then the write rate is trivial and the read rate is enormous but can be served locally. Each instance reads flags on every request, maybe millions of reads per second in total, so reads must come from memory inside each process, not from a network call. The design is about distribution and safety, not throughput.

Interviewer: Go ahead.

Candidate: The source of truth is a strongly consistent store, replicated with consensus across regions, holding versioned configuration. Every change goes through an API that validates it against a schema, records who made it, and assigns a new version number. Each region has a distribution layer: a set of servers that receive changes and push them to a lightweight agent on every host, over long-lived connections. The agent writes the configuration to local disk and exposes it to the process through a library that keeps it in memory. If everything upstream is down, services keep running on the last configuration they have, read from local disk at startup.

Interviewer: Why not let services read from the central store directly?

Candidate: Fifty thousand instances polling a consensus store would overload it, and a configuration outage would become an outage of every service. Pushing through regional caches and local agents makes reads independent of the central store's availability. That's the most important design decision here: the configuration system must never be on the critical path of a request.

Interviewer: How do you make changes safe?

Candidate: Several layers. Validation: schemas and type checks, plus custom checks such as "a timeout can't be zero". Review: two-person approval for critical keys. Staged rollout: a change applies to a canary set of hosts, then one region, then everywhere, with automatic checks of error rates between stages, and automatic rollback if they rise. Every change is a new version, so rollback is just "make version N-1 current again". And a kill switch per feature flag.

Interviewer: Two engineers change the same flag at once.

Candidate: The API uses optimistic concurrency: each change states the version it was based on, and the store rejects it if the version has moved on, so the second engineer sees the conflict and re-applies their change knowingly. No silent overwrites.

Interviewer: How would you migrate the company onto this from scattered config files?

Candidate: Gradually. First, ship the client library with a fallback to the existing files, and mirror existing configuration into the new store read-only. Then move teams one at a time to make changes through the new system, starting with low-risk services. Measure propagation time and staleness per host, and only remove the old files when every reader is on the new path. I'd also define ownership: each configuration namespace belongs to a team, which controls approvals.

Interviewer: What would you monitor?

Candidate: Propagation delay per region, the version each host is running, so we can see stragglers, validation failures, rollbacks, and the error-rate checks between rollout stages. And a dashboard that answers "what changed in the last hour?", the first question in most incidents.

Interviewer's notes. *Strong hire, staff. Reframed the problem around its real risk, configuration-caused outages, before designing. Correctly identified that reads must be local and the system off the critical path; designed push distribution with local caching and last-known-good fallback. Safety mechanisms were comprehensive and practical. Covered concurrency, migration, ownership and incident response, which is exactly the breadth expected at staff level.*

What the Four Interviews Have in Common

- Every candidate **agreed on scope** and stated assumptions before designing.
- **Estimates drove decisions**: object storage and a CDN; node counts from memory; a write rate too small to matter.
- Each identified **the hard part of that particular system** early and spent their time there.
- **Trade-offs were explicit**: replication versus memory cost, push versus pull, conditional updates versus single-owner queues.
- The more senior the level, the more the conversation covered **failure, safety, operations and migration**.

Key Takeaways

- Practice full interviews out loud, with a timer, on prompts you haven't studied.
- Scope first, estimate next, and use the numbers.
- Find the hard part of the system and go deep there.
- At senior and staff levels, discuss failures, safety, operations and rollout without being asked.

CHAPTER 37

Common Mistakes and How to Recover

Interviewers see the same mistakes again and again. Most are not gaps in knowledge but habits under pressure, which means they can be fixed with practice. Here are the most common ones, what they look like from the interviewer's side, and how to recover if you catch yourself making one mid-interview.

1. Designing Before Understanding

What it looks like: drawing a load balancer and three databases thirty seconds after hearing the prompt.

Why it hurts: you may design the wrong system, and you lose the chance to show product judgment.

Recovery: stop and say, "Let me step back and confirm the requirements before I go further." Interviewers respect the correction far more than they penalize the false start.

2. Estimates That Go Nowhere

What it looks like: five minutes of arithmetic, then a design that would be identical with numbers a thousand times larger or smaller.

Recovery: after each estimate, say what it implies: "So it's read-heavy, so caching matters most." If you realize later that a number should change a decision, go back and change it, out loud.

3. Buzzword Architecture

What it looks like: "We'll use Kafka, Redis, Cassandra and Kubernetes," with no explanation of what each solves.

Why it hurts: naming a technology isn't the same as understanding it, and interviewers will ask "why?"

Recovery: describe the mechanism and the need first ("we need a durable, partitioned log so several consumers can read the same events"), then name an example technology if useful.

4. Over-Engineering

What it looks like: microservices, multi-region active-active replication and a service mesh for a system with 100 requests per second.

Recovery: start simple and say how you'd evolve: "One database handles this load comfortably; if writes grow tenfold, I'd partition by user ID." Simplicity justified by numbers is a senior signal.

5. Going Deep Too Early

What it looks like: twenty minutes on the database schema, and no time left for the rest of the system.

Recovery: watch the clock. At about 25 minutes, make sure a complete high-level design is on the board. If you notice you're stuck in a detail, say, "Let me finish the overall design and come back to this."

6. Ignoring Failures

What it looks like: a design in which nothing ever goes wrong.

Recovery: in the wrap-up, walk through the main components and ask yourself, out loud, "What if this fails?" For each: redundancy, timeouts and retries, graceful degradation. Chapter 14 is your checklist.

7. One-Sided Answers

What it looks like: "We should use NoSQL" with no alternative or downside.

Recovery: for every important decision, name at least one alternative and the trade-off: "A relational database gives us transactions; a wide-column store gives cheaper writes at this volume. Since messages need no cross-row transactions, I'd choose the wide-column store."

8. Not Listening

What it looks like: the interviewer says "let's assume the cache can fail" and the candidate continues with their plan.

Why it hurts: hints and constraints are the interviewer steering you toward what they want to evaluate. Ignoring them wastes the interview.

Recovery: when a hint comes, follow it, even if you planned something else: "Good point, let's look at cache failure now."

9. Silence

What it looks like: long pauses while the candidate thinks privately.

Recovery: think out loud, even when unsure: "I'm deciding between pushing to followers and pulling at read time. Push makes reads fast but..." The interviewer can only evaluate reasoning they hear.

10. Bluffing

What it looks like: confidently describing how a technology works, incorrectly.

Recovery: say what you know and reason from principles: "I haven't operated that database. If it's log-structured, I'd expect fast writes and reads that may

touch several files." Honest reasoning is always better than a confident mistake that the interviewer has to correct.

A Recovery Phrasebook

Situation	What to say
You started drawing too early	"Let me step back and confirm the requirements first."
You're stuck	"Here's what I'm considering and what's blocking me..."
You realize an earlier choice was wrong	"Given this new constraint, I'd change my earlier choice, because..."
You don't know a technology	"I'm not sure how that system implements it; from first principles, I'd expect..."
Time is running out	"Let me summarize the full design, then the two things I'd do next."

Key Takeaways

- Most mistakes are habits under pressure: rushing, over-building, going too deep, staying silent.
- Every mistake is recoverable if you name it and correct course out loud.
- Justify every component, consider failures, and present alternatives.
- Follow the interviewer's hints; they show you what is being evaluated.

CHAPTER 38

Your Study Plan

System design skill grows from two things: understanding the building blocks well enough to explain why they work, and practicing complete designs out loud often enough that the framework becomes automatic. These plans combine both. Adjust them to the time you have; they assume an hour or two on weekdays and a longer session on the weekend.

How to Practice a Design

1. **Pick a prompt** from Part III or from the list below, and read only the prompt.
2. **Set a timer for 40 minutes** and design out loud, on paper or in a drawing tool, following the seven steps of Chapter 2.
3. **Compare** with the worked design: did you ask the important questions? Did your estimates point to the same bottlenecks? Did you find the hard part?
4. **Write down** the one or two things you missed, and revisit them a week later.
5. **Run the code** for the building blocks you're least sure about, and change a parameter to see what happens.

The Two-Week Plan

Days	Focus
1	Chapters 1 to 3: what's evaluated, the framework, estimation
2	Chapters 5 and 6: scaling out, caching
3	Chapters 7 and 8: databases, replication
4	Chapters 9 and 10: partitioning, queues
5	Practice: Link Shortener (Chapter 16), timed
6	Chapters 11 and 14: rate limiting, reliability
7	Practice: News Feed (Chapter 22) and Chat (Chapter 23)
8	Chapters 12, 13 and 15: IDs, probabilistic structures, observability
9	Practice: Notification Service (21) and Key-Value Store (18)
10	Practice: Payment System (29) and Ticket Booking (30)
11	Mock 1 and Mock 2 from Chapter 36, timed, before reading them
12	Practice two designs you haven't read, from the list below
13	Mock 3 or 4, ideally with a partner as interviewer; Chapter 37
14	Review Appendix A, your notes and the template in Appendix C. Rest

The Six-Week Plan

Week	Building blocks	Designs (practice first, then read)
1	Chapters 1 to 6	Link Shortener, Distributed Rate Limiter
2	Chapters 7 to 10	Key-Value Store, Unique ID Generator, Web Crawler
3	Chapters 11 to 15	Notification Service, News Feed, Chat
4	Revisit weak blocks	Autocomplete, Video Streaming, File Sync, Nearby Places
5	Run and modify the models	Ride Matching, Payments, Ticket Booking, Metrics, Job Scheduler
6	Chapter 37	Leaderboard, Collaborative Editing, Message Queue; all four mocks

More Prompts for Practice

Apply the framework to prompts not covered in this book:

- A pastebin (share text snippets by link)
- A hotel or restaurant reservation system
- A food delivery platform
- An online code judge that runs untrusted code
- A distributed cache (Mock 2 shows one approach)
- A stock exchange order matching engine
- An ad click aggregation pipeline
- A recommendation feed ("people you may know")
- An email service for millions of mailboxes
- A global feature-flag service (Mock 4 shows one approach)

For each, write down which Part III designs it resembles. Most new prompts are combinations of building blocks and patterns you've already seen.

Practicing With Other People

The design interview is a conversation, so practice it as one. Take turns as interviewer with a friend or colleague: the interviewer reads a prompt, answers clarifying questions, adds a constraint halfway through ("now make it multi-region"), and writes notes like the ones at the end of each mock interview. Being the interviewer teaches you what interviewers notice.

The Week Before

- **Ask your recruiter** whether your round is infrastructure-focused or product-focused, and which tool you'll draw in.
- **Practice drawing** in that tool until it's effortless.
- **Review**, don't cram: Appendix A's numbers, Appendix B's building blocks, the template in Appendix C.
- **Sleep.** Design interviews reward clear thinking more than recall.

Key Takeaways

- Practice complete designs out loud and timed, before reading the worked answers.
- Alternate building-block study with full designs; each makes the other stick.
- Run and modify the models to build intuition for the numbers.
- Practice with a partner, and take turns as interviewer.

Appendix A: Numbers Every Designer Should Know

These figures are for back-of-the-envelope estimates. They are orders of magnitude, not benchmarks: real values depend on hardware, configuration and workload, and they change over time. State them as assumptions in an interview.

Time and Rates

Fact	Value
Seconds in a day	86,400 (about 10^5)
Seconds in a month	about 2.6 million
Seconds in a year	about 31.5 million
1 million per day	about 12 per second
100 million per day	about 1,200 per second
1 billion per day	about 12,000 per second
Peak versus average	2 to 10 times; state your assumption

Sizes

Unit	Bytes
1 KB	10^3
1 MB	10^6
1 GB	10^9
1 TB	10^{12}
1 PB	10^{15}
2^1 , 2^2 , 2^3 , 2^4	about 10^3, 10^6, 10^9, 10^{12}

Item	Typical size
An ID (64-bit integer)	8 bytes
A UUID	16 bytes
A short string (name, URL)	tens to a few hundred bytes
A database row with a few fields	a few hundred bytes to 1 KB
A compressed photo	100 KB to a few MB
A minute of video	about 3 to 60 MB, depending on quality

Latency, by Order of Magnitude

These are the classic "latency numbers every programmer should know", popularized by Jeff Dean and Peter Norvig, rounded to orders of magnitude. Modern hardware has improved some (SSDs especially), but the ratios still guide design.

Operation	Order of magnitude
Read from CPU cache	about 1 nanosecond
Read from main memory	about 100 nanoseconds
Read 1 MB sequentially from memory	tens to hundreds of microseconds
Random read from an SSD	tens to about 100 microseconds
Round trip within a data center	hundreds of microseconds
Read 1 MB sequentially from an SSD	about a millisecond

Operation	Order of magnitude
Disk seek on a spinning hard drive	about 10 milliseconds
Round trip across a continent	tens of milliseconds
Round trip between continents	100 to 200 milliseconds

The lessons: memory is about a thousand times faster than a network round trip in the data center, and a cross-continent round trip is about a hundred times slower again. Avoid sequential network calls on the request path, and keep data close to the users who read it.

Rough Capacities for Planning

Assumptions used in this book's estimates; adjust for your hardware and workload.

Component	Planning figure
Stateless web or API server	about 1,000 to 10,000 simple requests per second
Relational database, single node	thousands to tens of thousands of simple queries per second
In-memory cache node	100,000 or more operations per second
Log partition (message queue)	about 10 MB per second
Long-lived connections per server	about 100,000
Usable memory per cache server	about 64 to 256 GB
Usable disk per storage node	a few to tens of TB

Availability

Availability	Downtime per year	Downtime per 30 days
99%	about 3.7 days	about 7.2 hours
99.9%	about 8.8 hours	about 43 minutes
99.99%	about 53 minutes	about 4.3 minutes
99.999%	about 5 minutes	about 26 seconds

Components in series multiply their availabilities: two services at 99.9% each, both required for a request, give about 99.8%. Redundant components in parallel fail only if all fail, which is why redundancy raises availability.

Little's Law

Requests in the system = arrival rate × time in the system. At 2,000 requests per second and 50 ms each, about 100 requests are in progress at any moment, so you need at least 100 concurrent workers or connections.

Appendix B: Building Blocks Reference

A one-page reminder of each building block: what it's for, its main trade-off, and where it appears in this book.

Building block	Use it for	Main trade-off	Chapters
Load balancer	Spreading requests; removing unhealthy instances	Another hop; balancers must be redundant	5
Stateless service	Easy horizontal scaling and deployment	State must live elsewhere	5
CDN	Static and media content near users	Staleness; purge and versioning needed	4, 25
Cache (cache-aside)	Read-heavy data	Staleness, invalidation, stampedes, hot keys	6, 16, 22
Relational database	Structured data, transactions, joins	Write scaling beyond one node	7, 29
Key-value / wide-column store	Huge volumes of reads and writes by key	Limited queries; no cross-row transactions	7, 18, 23
Object storage	Large binary files	High per-request latency	7, 25, 26
Search index	Full-text search and ranking	Updated asynchronously	7, 24
Replication (leader-follower)	Read scaling, availability	Replication lag; failover	8
Quorums	Tunable consistency without a leader	Latency of waiting for replicas	8, 18

Building block	Use it for	Main trade-off	Chapters
Partitioning (sharding)	More data or writes than one node	Cross-partition queries; hot keys	9
Consistent hashing	Adding and removing nodes cheaply	Uneven load without virtual nodes	9, 18
Message queue	Decoupling, absorbing bursts, task distribution	Asynchronous; duplicates	10, 21
Partitioned log	Event streams, replay, several consumers	Order only per partition	10, 35
Outbox pattern	Reliable events alongside database writes	Extra table and publisher	10, 29
Rate limiter (token bucket)	Protecting services, fairness	Shared state across servers	11, 17
Time-ordered IDs	Unique, sortable keys without coordination	Clock dependence	12, 19
Vector clocks	Detecting concurrent updates	Metadata size; merge logic	12, 18
Bloom filter	Fast "definitely not present" checks	False positives	13, 20
Count-Min sketch	Approximate counts, heavy hitters	Overestimates	13, 24
HyperLogLog	Counting distinct items	About 1 to 2% error	13
Timeouts, retries, backoff, jitter	Surviving transient failures	Duplicates; retry storms if done badly	14
Circuit breaker	Failing fast when a dependency is down	Tuning thresholds	14
Merkle tree	Finding differences between replicas	Rebuild cost	18
Geohash / spatial index	"Near me" queries	Cell sizes vary with latitude	27, 28
Idempotency keys	Safe retries of non-read operations	Storage for keys and results	4, 29
Double-entry ledger	Correct, auditable money movement	More rows; balances are sums	29

Building block	Use it for	Main trade-off	Chapters
Optimistic concurrency (versions)	Preventing lost updates and double booking	Retries under contention	17, 26, 30
Leases and fencing tokens	Recovering work from crashed workers safely	At-least-once execution	32
Operational transformation / CRDTs	Concurrent editing that converges	Complexity; metadata	34

Appendix C: One-Page Design Template

Use this template to practice, and keep it in your head during interviews. The times are for a 45-minute round.

1. Requirements (5 minutes)

- **Users and core actions:** three to five functional requirements, phrased as user actions.
- **Out of scope:** say it explicitly.
- **Scale:** users, daily active users, data volume.
- **Latency:** which operations must be fast?
- **Consistency:** where is stale data acceptable, and where not?
- **Durability and availability:** what must never be lost; what must never be down?
- **Geography:** one region or global?

2. Estimates (5 minutes)

- Reads and writes per second, average and peak.
- Storage over the data's lifetime, with replication.
- Bandwidth for large objects.
- Memory for caches; connections for real-time features.

- **One sentence on what the numbers imply.**

3. API (3 to 5 minutes)

- Main endpoints or functions, with key parameters and responses.
- Cursor pagination; idempotency keys for writes; authentication.

4. Data Model (3 to 5 minutes)

- Main entities and key fields.
- Access patterns, and the store chosen for each.
- Partition key, if partitioned.

5. High-Level Design (10 minutes)

- Five to ten boxes: clients, load balancer, services, stores, caches, queues, CDN.
- **Trace one read and one write end to end.**

6. Deep Dives (10 to 15 minutes)

For each of the two or three hardest parts:

1. The problem, with numbers.
2. Two or three options.
3. The comparison on the requirements that matter.
4. The choice, and what you'd monitor.

Common deep dives: hot keys, fan-out, consistency and conflicts, idempotency, ordering, failover, cache invalidation, rate limiting, search and ranking.

7. Wrap-Up (3 to 5 minutes)

- **Bottlenecks** at ten times the traffic.

- **Failure modes:** each major component and a region.
- **Observability:** what you'd measure and alert on.
- **Security and privacy:** authentication, authorization, encryption, sensitive data.
- **Next steps:** what you'd build or improve with more time.

Questions to Ask Yourself When Stuck

- What's the read/write ratio, and what does it imply?
- Does the data fit on one machine? In memory?
- What's the hottest key, and what happens to it?
- What happens if this component is slow? If it's down?
- Where can I make it asynchronous?
- What must be strongly consistent, and what can be eventual?
- Which operations need to be idempotent?

Appendix D: Design Index

The twenty worked designs of Part III, with the building blocks each relies on and the behavior its tested model demonstrates, followed by the four mock interviews of Chapter 36.

Chapter	Design	Key building blocks	Tested model
16	Link Shortener	Base-62 IDs, cache-aside, 301 vs 302	Code generator with aliases and expiry
17	Distributed Rate Limiter	Token bucket, atomic updates, fail open/closed	Lost updates vs compare-and-set
18	Key-Value Store	Consistent hashing, quorums, hinted handoff	Merkle-tree anti-entropy
19	Unique ID Generator	Snowflake IDs, range allocation	Block allocator with crashes
20	Web Crawler	URL normalization, politeness, Bloom filter	Polite per-host frontier
21	Notification Service	Priority queues, dedup, quiet hours	Send/skip decision rules
22	News Feed	Fan-out on write vs read, hybrid	Hybrid feed vs pull-all
23	Chat and Messaging	WebSockets, sequence numbers, client IDs	Sync after reconnect, no gaps
24	Search Autocomplete	Trie with top-k, offline pipeline	Top-k trie vs brute force
25	Video Streaming	Transcoding, segments, CDN, ABR	Buffer-based bitrate selection

Chapter	Design	Key building blocks	Tested model
26	File Storage and Sync	Chunking, dedup, versions, erasure coding	Content-defined chunking
27	Nearby Places	Geohash, neighbor cells	Latitude-aware geohash search
28	Ride Matching	Live grid index, atomic reservation	Ring search vs brute force
29	Payment System	Idempotency, double-entry ledger, reconciliation	Ledger with retries
30	Ticket Booking	Waiting room, optimistic concurrency, holds	No double booking under races
31	Metrics and Alerting	Time-series storage, rollups, cardinality	Exact rollups; sustained alerts
32	Distributed Job Scheduler	Leases, heartbeats, fencing tokens	Crash-tolerant job leasing
33	Real-Time Leaderboard	Sorted sets, rank by counting	Fenwick-tree ranks
34	Collaborative Document Editing	Operational transformation, CRDTs	OT convergence on 20,000 pairs
35	Distributed Message Queue	Partitioned replicated log, offsets	Consumer groups and replay

Mock Interviews

Mock	Prompt	Level
Mock 1	Photo Sharing	Mid-level
Mock 2	Distributed Cache	Senior
Mock 3	Online Auctions	Senior
Mock 4	Global Configuration Service	Staff

Appendix E: Glossary

ACID. Atomicity, consistency, isolation, durability: the guarantees of database transactions (Chapter 7).

Adaptive bitrate (ABR). Switching video quality per segment to match network conditions (Chapter 25).

Anti-entropy. Background comparison and repair of replicas, often with Merkle trees (Chapter 18).

At-least-once delivery. Every message is delivered, possibly more than once (Chapter 10).

Backpressure. Signals that slow producers when consumers can't keep up (Chapter 10).

Bloom filter. A compact set membership test with false positives but no false negatives (Chapter 13).

CAP theorem. During a network partition, a system must choose between consistency and availability (Chapter 8).

CDN (content delivery network). Edge servers that cache content close to users (Chapter 4).

Circuit breaker. A wrapper that stops calling a failing dependency for a while (Chapter 14).

Compaction. Merging storage files and discarding overwritten or deleted values (Chapter 7).

Consensus. Agreement among nodes on a value despite failures, as in Paxos and Raft (Chapter 8).

Consistent hashing. Placing keys and nodes on a ring so adding a node moves about 1/N of keys (Chapter 9).

Count-Min sketch. A fixed-size structure that estimates item frequencies, never underestimating (Chapter 13).

CRDT. A conflict-free replicated data type: replicas converge without coordination (Chapter 34).

Cursor pagination. Fetching "items after this one" instead of by page number (Chapter 4).

Dead-letter queue. Where messages go after repeated processing failures (Chapter 10).

Double-entry bookkeeping. Recording every transfer as two entries that sum to zero (Chapter 29).

Erasure coding. Storing data as fragments plus parity, surviving lost fragments with less overhead than full copies (Chapter 26).

Error budget. The amount of failure an SLO allows (Chapter 15).

Eventual consistency. Replicas agree eventually if writes stop (Chapter 8).

Fan-out. Delivering one event to many recipients, on write or on read (Chapter 22).

Fencing token. An increasing number that lets a resource reject requests from an outdated lease holder (Chapter 32).

Geohash. A string encoding of a location in which nearby points usually share a prefix (Chapter 27).

Hinted handoff. Storing a write temporarily on another node until the intended replica returns (Chapter 18).

Hot key. A key that receives a disproportionate share of traffic (Chapters 6 and 9).

HyperLogLog. An estimator for the number of distinct items, using a few kilobytes (Chapter 13).

Idempotency key. A client-supplied ID that makes retries of an operation safe (Chapters 4 and 29).

Lease. Time-limited ownership of a resource or job, renewed by heartbeats (Chapter 32).

Little's law. Items in a system = arrival rate × time in the system (Chapter 3).

LSM-tree. A log-structured merge tree: writes to memory, flushed to sorted files, merged by compaction (Chapter 7).

Merkle tree. A tree of hashes used to find differences between datasets efficiently (Chapter 18).

Object storage. Storage of immutable blobs by key, built for durability and capacity (Chapter 7).

Offset. A message's position in a log partition (Chapters 10 and 35).

Operational transformation (OT). Rewriting concurrent edits so all replicas converge (Chapter 34).

Optimistic concurrency control. Updating only if a version is unchanged since it was read (Chapter 30).

Outbox pattern. Writing events to a table in the same transaction as data changes, then publishing them (Chapter 10).

PACELC. Even without partitions, systems trade latency against consistency (Chapter 8).

Partition (shard). A subset of data stored on its own node or group of nodes (Chapter 9).

Percentile (p99). The value below which 99% of measurements fall (Chapter 15).

Quorum. The number of replicas that must respond for a read (R) or write (W) to succeed (Chapter 8).

Rate limiter. A component that caps requests per client per period (Chapter 11).

Read repair. Fixing stale replicas when a read notices them (Chapter 18).

Replication lag. How far a follower is behind its leader (Chapter 8).

SLI, SLO, SLA. Service level indicator (a measurement), objective (a target), agreement (a contract) (Chapter 15).

Snowflake ID. A 64-bit, time-ordered ID made of timestamp, machine and sequence bits (Chapter 12).

Stateless service. A service that keeps no request-to-request state in its own memory (Chapter 5).

Thundering herd (stampede). Many requests missing a cache at once and overloading the source (Chapter 6).

Token bucket. A rate limiter that refills tokens at a fixed rate and allows bounded bursts (Chapter 11).

Vector clock. Per-node counters that reveal whether versions are ordered or concurrent (Chapter 12).

Virtual nodes. Multiple ring positions per physical node to spread load evenly (Chapter 9).

WebSocket. A persistent, two-way connection between client and server (Chapter 4).

About the Author

Aadhavan Muthurengan is an automation and artificial intelligence professional with more than 15 years of experience in the automotive industry.

Throughout a career focused on product development and validation, Aadhavan has designed and built a wide range of in-house tools that automate complex engineering workflows, speed up testing, and help teams deliver reliable, high-quality results. Validation engineering teaches a particular habit: don't trust a design until you've tested how it behaves when things go wrong. That habit runs through this book, in which every building block is a model you can run and every claim about its behavior is measured.

Aadhavan is also the author of *Prompt Engineering Mastery*, a beginner-to-advanced guide to getting better results from AI; *Vibe Coding for Beginners*, a guide to building real software with AI coding tools; *Building AI Agents with Claude*, a guide to agents that use tools, remember, and get real work done; *Python Programming Hands-On*, a practical, tested introduction to programming; *How Large Language Models Are Made*, a clear explanation of how today's AI models are built; and *The Coding Interview Playbook*, 120 original coding problems with tested Python solutions.

Thank you for reading *The System Design Interview Playbook*. Good luck with your interviews: think out loud, use your numbers, and enjoy the conversation.